

Seasons in Teamder — a complete technical reading

Prepared for external code review. Repository `~/Projects/soccer`, branch `perf/firestore-read-costs`. Every line number is against commit `e35394a` (2026-09-18 21:40 +0300), `HEAD` of that branch. All production figures were read live from Firestore project `soccer-app-52b6b` on **19.09.2026**.

How to read this document

Two words you must not confuse

Hebrew	Transliteration	What it is
מחזור	<i>machzor</i>	One game-night . A club plays one a week. This is the unit a season is measured in.
משחקון	<i>mischakon</i>	One mini-game inside that night — typically four to six of them.

משחקונים exist **only in "advanced mode"**. The single club that has ever run a season is on the plain timer and has recorded **zero** משחקונים across all ten of its sealed evenings. A great many defects in this document are the same mistake in different clothes: a screen reads one of these two units and labels it with the other, or divides one by the other.

The population this feature runs on

This matters more than usual, because the feature is almost entirely untried in production:

- 195 groups documents scanned on 19.09.2026.
- 2 of them carry a seasons block at all.
- Both use the rounds cadence. **No club has ever run a date season.** Every statement in §4 about date seasons is read from code, not observed.
- 1 club — `HhzIwmjMl1i5HS0GHt3p`, מועדון שכחת שושי — has ever closed a season. It is the worked example throughout.
- That club is **timer-only**: `totals.rounds` is 0 for every sealed evening it has.

The three counters, named once here

The question "how many מחזורים were there" has **three** independent answers in this codebase, and they are allowed to disagree. Sections 1 and 3 develop this; the short version:

Counter	Where	Who writes it	Decrements?
<code>clubRecords.eveningsSealed</code>	<code>/clubRecords/{groupId}</code>	server only, since 26.08.2026	never
<code>seasons.playedRounds</code>	<code>groups.seasons</code> map	server, per sealed evening	on reopen only

Counter	Where	Who writes it	Decrements?
a live scan of <code>games</code>	client	nobody — derived	n/a

The scan filters on the `games.seasonId` stamp, which is **write-once** and was introduced after most games already existed. **An unstamped game is read as belonging to season 1.** On the worked club, 19 of 22 evenings are unstamped. This single fact is the root of the discrepancy the owner reported (22 evenings on one screen, 19 on the next, 0 on a third).

Deployed ≠ HEAD

The Cloud Functions running in production were built from source as it stood at **16:04Z on 2026-09-18**. Commits `bc6a562` (17:03), `5531a21` (19:02) and `8235851` (21:14, all +0300) landed after that. Neither store binary contains most of the seasons UI: both 1.1.9 artifacts — Android versionCode 235, iOS build 103 — were built from `4de80d0` (18.09 08:54 +0300), and every seasons UI commit is newer. Where production and `HEAD` differ, the sections say which is being quoted.

Contents

- 1. The data model
 - 2. Turning seasons on and off
 - 3. Counting a מחזור
 - 4. Cadence
 - 5. Closing a season
 - 6. Undoing a close
 - 7. The nine titles
 - 8. Season vs lifetime
 - 9. The personal summary
 - 10. The screens
 - 11. Plumbing
 - 12. Testing and open questions
 - 13. Defect register — every named defect in this document, in one table
-

The data model

Every line number in this section is against commit `e35394a` (2026-09-18 21:40 +0300), which is `HEAD` of branch `perf/firestore-read-costs`. Production numbers were read live from project `soccer-app-52b6b` on 2026-09-19.

Scale, first, because it changes how you read everything else

Collection	Documents in production
<code>groups</code>	194
<code>groups</code> with a <code>seasons</code> block at all	2
<code>groups</code> with <code>seasons.enabled == true</code>	1 (<code>HhzIwmjMl1i5HS0GHt3p</code> , מועדון שכחת שושי, "Shoshi's Forgetting Club")
<code>seasonSummary</code>	1 (<code>HhzIwmjMl1i5HS0GHt3p__s1</code>)
<code>seasonCards</code>	1 (same season)
<code>roundSummaries</code> for that club	10, against 23 finished games
<code>communityPairStats</code> for that club	314, of which 293 are guest pairs

There is exactly one archived season in the entire database. Every "worked example" below is that season, because there is no other. The feature has never run anywhere else, which is why so many defects survived to production: the code paths that only fire on season 2, 3 and 4 have literally never executed on real data.

`groups/{groupId}.seasons` — the only mutable lifecycle record

This sub-map is the whole state machine. It is the only seasons data a client may read cheaply, and it is the only seasons data the client may not write: the rules lock the field, and every mutation goes through one of five callables (`enableClubSeasons` , `disableClubSeasons` , `updateSeasonTarget` , `endSeasonNow` , `reopenLastSeason`) or two server paths (`performSeasonClose` at `functions/src/index.ts:15884-15925` , and the per-seal increment at `functions/src/index.ts:5219-5232`).

The declared type is `GroupSeasons` , `src/types/index.ts:959-1000` :

```

export interface GroupSeasons {
  enabled: boolean;
  currentNo: number;
  currentId: string;
  startedAt: number;
  roundsAtStart?: number;
  playedRounds?: number;
  cadence: {
    type: SeasonCadenceType; // 'date' | 'rounds'
    months?: number;
    endsAt?: number;
    startsOn?: string; // 'YYYY-MM-DD', club calendar
    endsOn?: string; // 'YYYY-MM-DD', LAST valid day
    targetRounds?: number; // TOTAL finished evenings, not a remainder
  };
  targetHistory?: SeasonTargetChange[];
  count: number;
}

```

The live production document:

```

{
  "enabled": true,
  "currentNo": 2,
  "currentId": "s2",
  "startedAt": 1789679141952, // 2026-09-17 21:05:41Z
  "roundsAtStart": 10,
  "playedRounds": 0,
  "reopenedAt": 1789727847680, // 2026-09-18 10:37:27Z
  "count": 1,
  "cadence": { "type": "rounds", "targetRounds": 24,
    "months": null, "endsAt": null, "endsOn": null, "startsOn": null },
  "targetHistory": [ { "at": 1789727874911, "by": "YIZlKBvvjae3oqgIoAMr9nzQEi1",
    "byName": "Eliran Tzabari",
    "from": { "type": "rounds", "targetRounds": 2, ... },
    "to": { "type": "rounds", "targetRounds": 24, ... } } ]
}

```

Field by field:

Field	Type	Written by	Read by	Absent means	Absent ≠ 0?
enabled	bool	all five callables	everything; the sweep's query is <code>where('seasons.enabled','==',true)</code> (<code>index.ts:16078</code>)	feature off	n/a
currentNo	int, 1-based	enable / close / reopen	id derivation, <code>eveningInSeason</code> fallback	treated as 1 by the reader	no
currentId	string <code>s{N}</code>	same	the stamp written onto games; the scope filter; the archive doc id	the client reader returns undefined for the whole block (<code>firestore.ts:671</code>)	n/a
startedAt	epoch ms	<code>close(now)</code> , <code>enable(now)</code> , reopen (archive's <code>startsAt</code>)	hall-of-fame date range, <code>rebaseCadence</code>	0	no
roundsAtStart	int	<code>seasonSeed()</code> = <code>clubRecords.eveningsSealed</code> at open	<code>completedRoundsFrom</code> fallback; <code>roundSummaries.seasonEvenings</code>	0	no
playedRounds	int	seeded at open, then <code>FieldValue.increment(1)</code> per seal	the card, the sweep, <code>endSeasonNow</code> , the running personal summary	falls back to <code>eveningsSealed</code> – <code>roundsAtStart</code>	yes — <code>completedRoundsFrom(seasonCounters.ts:110)</code> tests <code>typeof playedRounds === 'number'</code> , so 0 is honoured and absent is not
cadence.type	'date' \ 'rounds'	enable / retarget / <code>rebaseCadence</code>	<code>seasonFinishLine</code>	client reader defaults to 'date' (<code>firestore.ts:695</code>)	no
cadence.months	int	enable (length) / retarget (remaining)	<code>rebaseCadence</code>	no length recorded	no
cadence.endsAt	epoch ms	legacy date seasons	<code>seasonFinishLine</code> kind 'epoch'	prefer <code>endsOn</code>	yes (null vs absent both falsy)

Field	Type	Written by	Read by	Absent means	Absent ≠ 0?
<code>cadence.startsOn / endsOn</code>	YYYY-MM-DD	date cadence	<code>seasonFinishLine</code> kind 'date', DST-safe	no calendar boundary	yes
<code>cadence.targetRounds</code>	int	enable / retarget	<code>seasonFinishLine</code> kind 'rounds'	<code>{kind: 'none'}</code> → a season that can never end	yes
<code>targetHistory</code>	array	<code>updateSeasonTarget</code> via <code>arrayUnion</code> ; reset to <code>[]</code> by every open	nothing on any screen	never moved	<code>[]</code> and absent are indistinguishable
<code>count</code>	int	+1 on close, -1 on reopen	which season the undo button targets; whether enable seeds history	0	no

Three further fields are written by the server and **do not exist in the TypeScript type, are not named by the client deserializer, and therefore do not exist on any client**:

- `reopenedAt: number` — stamped by `reopenLastSeason` (`index.ts:16905`), zeroed by every other opener. Grants a 48-hour immunity from re-closing (`REOPEN_GRACE_MS` , checked at `index.ts:15979` and `index.ts:16179`). The live club carries `reopenedAt = 2026-09-18 10:37Z` right now.
- `targetMovedToClose: {at, by, byName}` — set by `updateSeasonTarget` when the new target is already behind the club (`index.ts:16815`), read by `performSeasonClose` to stamp `endedEarly` .
- `dueBlockedSince: number` — set when a season came due and could not be closed (`index.ts:16206`), deleted when it finally is.
- `seasonStampRetry` is the same class of problem one collection over (see games, below).

`src/firebase/firestore.ts:667-714` rebuilds the block field by field, so **a field the reader does not name does not exist on the client, whatever Firestore holds**. This is a recurring failure mode in this codebase, and `targetHistory` was its most recent victim: the server wrote it on every change, the type documented it as "kept and shown... never quietly", and the reader dropped it. The reader was fixed; nothing renders it even now (ledger `target-history-never-shown-to-anyone`).

`SeasonTargetChange` (`src/types/index.ts:1005-1013`) is also already behind its writer. The writer at `index.ts:16802-16812` adds two more fields per entry:

```
targetHistory: admin.firestore.FieldValue.arrayUnion({
  at: now, by: uid, byName,
  from: seasons.cadence ?? null,
  to: next,
  playedAtMove: played,      // not in the type
  endsTheSeason,             // not in the type
}),
```

and no entry carries a `seasonId`. That omission is what allows ledger finding `reopen-inherits-the-successor-seasons-target-history`: an array element cannot say which season it belongs to, so a reopen that restores the wrong array cannot be detected.

`games/{gameId}.seasonId` — the membership stamp

One optional string field on the game document. Written once by `onGameRosterChanged` (`functions/src/index.ts:5802-5836`) on the `→ active` or `→ finished` transition, guarded by `!after.seasonId` so it is effectively write-once. A failed group read writes `seasonStampRetry: true` (`index.ts:5842`) so the next write to the game tries again.

The rule that reads it exists in two copies that are deliberately identical — `src/utils/seasonScope.ts:28` (client) and `functions/src/seasonCounters.ts:53` (server):

```
export function eveningInSeason(game, seasonId, seasonNo): boolean {
  const stamp = typeof game.seasonId === 'string' ? game.seasonId : '';
  return stamp ? stamp === seasonId : seasonNo === 1;
}
```

Absence is load-bearing and means "season 1". That is the whole of "your history becomes season 1". It is also, structurally, a one-shot promise: it can only ever be told once per club, and it is wrong for any club that disables and re-enables the feature.

Every finished game of the only club that runs seasons:

Evenings	seasonId
19 evenings, 2026-06-28 → 2026-09-15	<i>absent</i> → resolved to season 1
3 evenings, 2026-09-16 → 2026-09-17	"s1"
1 evening, 2026-09-17 21:23 (DTNsco1R...)	"s3"
any	"s2" — none

The last row is a dead stamp. Season 3 was opened when season 2 closed at 21:12Z, the game was stamped `s3` eleven minutes later, and then season 2 was reopened on 2026-09-18, which decrements `count` and deletes `s2`'s archive — so `s3` now names a season that does not exist and will not exist again until the club closes `s2`. `eveningInSeason(game, 's2', 2)` is false (stamp present, mismatched), so that evening belongs to **no** season on either side of the wire: it is in no archive, in no live scope, and `seasons.playedRounds` is 0 despite it

having been played inside s2's window. The stamp is write-once, finished games are client-read-only, and the trigger only fires on a status transition a finished game has no more of — so there is no repair path. Ledger: `undo-close-orphans-the-season-stamps`, `orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id`.

`seasonId` appears **nowhere in** `firestore.rules` (verified by grep: the only two hits are comment lines 2097 and 2111, both about other collections). A club member may therefore create a game pre-stamped with any season id, and the server's write-once check then refuses to correct it (ledger `games-seasonid-unpinned`, emulator-proven with four successful writes).

The three live tables a close winds back

These are the club's running counters. They are not seasons documents — they predate the feature — but a season close subtracts its archive out of them and a reopen adds it back, so their exact field sets are part of the seasons contract. The lists are pinned in one place, `functions/src/seasonRollover.ts:184-244`:

```
const PLAYER_SEASON_FIELDS = [
  'goals', 'assists', 'rounds', 'wins', 'losses', 'ties', 'games', 'cleanSheets', 'ownGoals',
  'penTaken', 'penScored', 'penMissed', 'penFaced', 'penSaved', 'penConceded',
  'csRounds', 'asRounds',           // coverage denominators
  'eveningScoreSum', 'eveningScoreCount',
] as const;
const PAIR_SEASON_FIELDS = [
  'assists', 'sameTeam', 'winsTogether', 'lossesTogether', 'cleanSheetsTogether',
  'against', 'winsA', 'winsB', 'assistsAToB', 'assistsBToA',
] as const;
const CLUB_SEASON_FIELDS = [
  'rounds', 'goals', 'guestGoals', 'ownGoals', 'tiedRounds', 'shootoutRounds', 'scorelessRounds',
] as const;
```

Anything **not** on these lists survives a close: `bestEvening`, `lastEveningScore`, `kingGoalsSum / kingGoalsCount` on player rows, `chemistrySince` and the `king*` accumulators on the club row. The module comment at the top of `seasonRollover.ts` explains why — resetting `kingGoalsSum` would make the first top scorer of every new season a perfect 10 by construction.

`communityPlayerStats/{groupId}__{uid}`. Doc id is the composite; `groupId` and `userId` are also fields, because a `list` does not bind the path wildcard (`firestore.rules:2070`). It carries **no display name** — that is why `closeSeason` issues a separate `getAll` over `/users` (`seasonRollover.ts:317-333`) to freeze names into the archive. Three season stamps live here:

- `seasonWoundBack`: `string` — written by the close, cleared by the reopen. Makes the subtraction idempotent.
- `seasonReopened`: `string` — written by the reopen, deleted by the close. Makes the addition idempotent.
- Both are read-only signals; neither is in any client type.

The real row for `מתן לוי` (`1IdtNEjbEXfIRSqvLrJVn99NsFI2`) today, trimmed:

```
{ "groupId": "HhzIwmjMl1i5HS0GHt3p", "userId": "1IdtNEjbEXfiRSqvLrJVn99Nsfi2",
  "goals": 0, "assists": 0, "rounds": 0, "wins": 0, "losses": 0, "ties": 0, "games": 0,
  "cleanSheets": 0, "csRounds": 0, "asRounds": 0,
  "eveningScoreSum": 0, "eveningScoreCount": 0,
  "bestEvening": { "goals": 0, "assists": 0, "involvement": 0, "cleanSheets": 2, "wins": 2 },
  "lastEveningScore": 6,
  "seasonWoundBack": "s1",
  "updatedAt": 1789661526768 }
```

Every season-owned counter is zero. That is correct behaviour — the season was archived — and it is also the single fact behind the owner's original complaint: the club screen scans games and says 22 evenings, and every screen that reads this row says 0 (ROUND4 §2, ledger `evenings-22-on-one-screen-0-on-the-next`). Note the stamp still reads `s1` even though `s2` has since been closed and reopened; the `s2` close skipped this row because a row with `rounds == 0 && games == 0` is not archived as a participant (`seasonRollover.ts:345`).

`communityPairStats/{groupId}__{a}__{b}`. Live `a / b` are *not* sorted; the archive key is `seasonPairKey()` (`seasonRollover.ts:104–110`) which sorts them and returns a `flip` flag so the directional counters (`winsA`, `assistsAToB`) can be re-oriented. A live row today:

```
{ "groupId": "HhzIwmjMl1i5HS0GHt3p",
  "a": "CEKrfDs20xSuiW6xmPKlMcx3NoV2", "b": "JoLFRxFr0tTaYfVAXBPwkvT0et62",
  "sameTeam": 0, "against": 0, "winsTogether": 0, "lossesTogether": 0,
  "cleanSheetsTogether": 0, "winsA": 0, "winsB": 0,
  "assists": 0, "assistsAToB": 0, "assistsBToA": 0,
  "seasonWoundBack": "s2", "updatedAt": 1789679528192 }
```

`seasonWoundBack: "s2"` while **s2 is the running season**. All 314 pair rows read this. The stamp is the close's idempotency latch, so the next close of `s2` will archive this pair's chemistry *and* skip winding it back — sealing it into a write-once archive while leaving it standing live for `s3` to inherit, double-counted. The clearing sweep at `seasonRollover.ts:1174–1200` exists in `HEAD` but ran after this reopen. Ledger: `stale-pair-woundback-stamp-skips-next-wipe`, ROUND4 §9. This is armed in production today.

`assists` versus `assistsAToB` / `assistsBToA` is a real ambiguity, not a duplication: the legacy undirected `assists` counts a **wider window** than the directional pair, so the two disagree. The archive keeps both; the awards use directional only (`seasonRollover.ts:527`), the club chemistry card uses legacy, and `seasonArchive.topDuo` (client) uses directional. On `s1` they are 7 and 8.

`communityStats/{groupId}`. One doc per club, `id = groupId`, with `groupId` also as a field for the same wildcard reason. Season-owned: the seven `CLUB_SEASON_FIELDS`. Not season-owned: `kingGoalsSum/Count`, `kingAssistsSum/Count`, `chemistrySince`. The close **sets the seven to absolute 0** (`seasonRollover.ts:1004–1021`) while the player and pair rows are *subtracted* — an inconsistency the module's own header calls out as forbidden ("SUBTRACT, do not write zeroes") and then violates fifty lines from the bottom. Ledger `club-totals-zeroed-not-subtracted`. Live:

```
{ "groupId": "HhzIwmjMl1i5HSOGHt3p", "rounds": 0, "goals": 0, "guestGoals": 0, "ownGoals": 0,
  "tiedRounds": 0, "shootoutRounds": 0, "scorelessRounds": 0,
  "kingGoalsSum": 6, "kingGoalsCount": 5, "kingAssistsSum": 4, "kingAssistsCount": 3,
  "chemistrySince": 1789679528192, "seasonReopened": "s2", "updatedAt": 1789727846661 }
```

clubRecords/{groupId} — eveningsSealed, and why nobody can read it

```
match /clubRecords/{groupId} { allow read, write: if false; } // firestore.rules:1813
```

Server-only in **both** directions, by design: it is the baseline the evening summary measures records against, and a second readable copy would become a drifting source of truth for "the best evening ever". The consequence for seasons is concrete: the counter a rounds target is measured against is unreadable by the app, which is the entire reason `groups.seasons.playedRounds` exists as a client-readable mirror (`index.ts:5210–5232`).

```
{ "groupId": "HhzIwmjMl1i5HSOGHt3p",
  "eveningsSealed": 10,
  "since": 1787739600000, // 2026-08-26 10:20Z
  "lastEveningAt": ..., "updatedAt": 1789655711795,
  "goals": {"value":2,"userIds":[...], "assists": ..., "wins": ..., "cleanSheets": ..., "involvement": ...
  "firstEverSeen": ["every_team_won","multiple_personal_records"] }
```

`eveningsSealed` is `FieldValue.increment(1)` per sealed evening (`index.ts:5194`), atomic specifically because an absolute write lost evenings when three sealed within a minute. But **it began on 2026-08-26**. The club has played 23 evenings and the counter says 10. Any arithmetic that subtracts `roundsAtStart` from it — `completedRoundsFrom`'s fallback (`functions/src/seasonCounters.ts:111–117`), `roundSummaries.seasonEvenings` (`index.ts:5157`) — is answering for the era of the counter, not for the club. On this club that fallback yields $10 - 7 = 3$ for a season the card says held 22. That is the arithmetic that `endSeasonNow` used to perform (two arguments instead of three) and would have sealed into a write-once archive.

seasonSummary/{groupId}__{seasonId} — the archive

Written exactly once, with `create()` (`seasonRollover.ts:584`), which is the feature's entire idempotency story. Deleted only by `reopenSeason`. Two read statements in the rules (`firestore.rules:2121–2164`): members list it by field; a **participant** — a uid that is a key in the `players` map with `games > 0 || rounds > 0` — may `get` it for ever, even after leaving the club.

Top-level shape:

Field	Type	Present when	Read by
<code>groupId</code> , <code>seasonId</code> , <code>no</code>	string/int	always	everything

Field	Type	Present when	Read by
groupName	string	always (frozen)	personal summary, for readers who can no longer read /groups
startsAt	epoch	always	hall of fame date range; reopenLastSeason restores seasons.startedAt from it
endsAt , closedAt	epoch	always, both = now	display
completedRounds	int	always	the season's length. reopenLastSeason fallback; seasonSummaryService.fromArchive (:382)
roundsAtStartOfSeason	int	always	server only — reopenLastSeason:16893 restores the offset
zeroedAt	epoch	only after the wipe finished	server only — distinguishes "already closed" from "resume a half-finished close" (seasonRollover.ts:667)
endedEarly	true	omitted unless set	seasonHistoryService:89 , badge on the card
partialData	true	omitted unless set	SeasonHistoryScreen:286 , the "נתונים חלקיים" (partial data) chip
closedBy , closedByName	string	manual close only	nothing — write-only, costs one extra /users read per close
originalTarget	cadence map	when the caller passed one	reopenLastSeason:16955 re-bases the cadence from it
targetHistory	array	omitted when empty	nothing
totals	map	always	club table
players	map uid → row	always	everything
pairs	map lo__hi → row	always	duo title, personal summary peers
awards	map of 9 keys → {winners:string[], value:number} \ null	always	card, titles, personal summary

The production document, trimmed:

```
{ "groupId": "HhzIwmjMl1i5HS0GHt3p", "groupName": "מועדון שכחת שושי",
  "seasonId": "s1", "no": 1,
  "startsAt": 1789547794676,          // 2026-09-16 08:36:34Z
  "endsAt": 1789661526768,          // 2026-09-17 16:12:06Z
  "closedAt": 1789661526768,
  "zeroedAt": 1789661533425,
  "completedRounds": 22,
  "roundsAtStartOfSeason": 7,
  "originalTarget": { "type": "rounds", "targetRounds": 22 },
  "totals": { "rounds": 37, "goals": 27, "assists": 13, "cleanSheets": 75,
    "guestGoals": 4, "ownGoals": 0, "tiedRounds": 9,
    "shootoutRounds": 6, "scorelessRounds": 16 },
  "awards": { "topScorer": {"winners":["alsobLSA..."],"value":10},
    "topAssister": {"winners":["K5rSGB4J..."],"value":5},
    "mvp": {"winners":[ ... all seven uids ... ],"value":6},
    "topWinner": {"winners":["B5KpY04I..."],"value":13},
    "mostLoyal": {"winners":["1IdtNEjb..."],"value":19},
    "cleanSheetKing": {"winners":["alsobLSA..."],"value":13},
    "penaltyKing": {"winners":["alsobLSA..."],"value":1},
    "penaltyKeeper": {"winners":["alsobLSA..."],"value":1},
    "deadlyDuo": null } }
```

Read that document against the club's own history and three things are visibly wrong:

1. **startsAt is the moment the admin switched the feature on**, not the season's first evening. The season contains 22 evenings from 2026-06-28; the archive says it began on 2026-09-16 and ended on 2026-09-17. **endsAt – startsAt** is 31.6 hours. The hall of fame renders this as "2026 ספט – 2026 ספט" ("Sep 2026 – Sep 2026") above "22 מחזורים" (22 evenings). Ledger `season1-startsat-is-the-enable-moment`, `season1-range-prints-a-31-hour-lie`. Worse, the client's `VOID_SPAN` guard treats a sub-48h season as a bookkeeping artefact; only `players !== 0` stops this club's entire hall of fame collapsing into a grey dashed line.
2. **completedRounds (22) and the awards disagree**. The awards were decided against a *different* denominator — `awardsDenominatorOf(players.games) = max(games) = 19` (`seasonRollover.ts:133-139, 556`). `minPenaltyAttempts(19) = 2` but `minPenaltyAttempts(22) = 3` (`src/utls/seasonAwards.ts:126`). `הלן צברי` took exactly 2 penalties and scored both; Nofar took 3 and scored 2. At 19 the title is `הלן's` on a rate of 1.0; at 22 `הלן` is not eligible and the title is Nofar's. The archive as it stands publishes the 22 and the 19-derived winner side by side, for ever, because `create()` makes it write-once and nothing recomputes. Ledger `awards-denominator-derivation-untested`, `repaired-completedrounds-silently-contradicts-the-sealed-titles`. The `updateTime` of this document (2026-09-18 11:41Z) is a day after its `createTime` (2026-09-17 16:12Z) — the length was hand-repaired; the awards were not.

3. `partialData` is **absent** on a season that is unambiguously partial: 19 of its 22 evenings predate the feature, assists were only collected from 21.06 and clean sheets from 17.08. The flag is written only on the `sealNow` activation branch (`index.ts:16483`), never on a rollover close and never on `endSeasonNow` . Three independent agents found this. Ledger `partialdata-only-on-the-sealnow-path` .

Also note `mvp.winners` has **seven of seven** members at `value: 6` . `eveningScoreCore` floors the scale at 6.0 and also returns 6.0 as its "no data" sentinel; every archived row has `eveningScoreSum / eveningScoreCount` exactly 6.0 (12/2 and 18/3). `leaders()` uses a floor of 0, so the whole club won שחקן העונה ("player of the season"). This is verifiable from a third source: a collection-group query over `users/*/seasonTitles` returns seven `..._s1__mvp` documents.

THE `PLAYERS` MAP ROW — AND THE ONE PLACE ABSENCE IS PRESERVED

```
"1IdtNEjbEXfiRSqvLrJVn99Nsfi2": {
  "displayName": "מחנן לוי",
  "games": 19, "rounds": 26, "goals": 2, "assists": 1,
  "wins": 11, "ties": 6, "losses": 9,
  "cleanSheets": 12, "ownGoals": 0,
  "penTaken": 1, "penScored": 0, "penMissed": 1, "penFaced": 1, "penSaved": 1, "penConceded": 0,
  "csRounds": 15, "asRounds": 24,
  "eveningScoreSum": 18, "eveningScoreCount": 3
}
```

`games` counts **evenings** (מחזורים); `rounds` counts **mini-games** (משחקונים). Both are in the same row with adjacent names and the code conflates them constantly. All seven rows here have `games` of 17–19 and `rounds` of 21–27. For a club on the plain timer — the common case — `rounds` would be 0 for every player of every season, which is why 23 of production's 87 stat rows have `games > 0` and no `rounds` at all.

Two fields, and only two, deliberately preserve the difference between absent and zero (`seasonRollover.ts:351–356`):

```
for (const f of PLAYER_SEASON_FIELDS) {
  // ABSENT is not zero for the two coverage denominators.
  if ((f === 'csRounds' || f === 'asRounds') && typeof x[f] !== 'number') continue;
  row[f] = num(x[f]);
}
```

and the reader mirrors it (`src/utils/seasonArchive.ts:102–103`):

```
...(typeof x.csRounds === 'number' ? { csRounds: x.csRounds } : {}),
...(typeof x.asRounds === 'number' ? { asRounds: x.asRounds } : {}),
```

The reason is arithmetic, not taste. `csRounds` and `asRounds` are *coverage denominators*: how many mini-games the clean-sheet and assist metrics were actually being recorded in. They arrived later than the metrics they divide. A stored `0` asserts "measured across zero rounds", which is a different statement from "we do not

know how many rounds this was measured across" — and the efficiency tab's fallback (divide by `rounds` instead) only fires on the second. Writing 0 made every long-standing player's clean-sheet percentage read about ten points low. Every *other* field in the archive defaults to 0 on read (`num()` at `seasonArchive.ts:9`), which is the ledger finding `older-archive-degrades-into-plausible-zeros`: an archive written by an older build yields confident wrong numbers rather than blanks, with no version field anywhere to detect it.

THE PAIRS MAP

Keyed "`<lo>__<hi>`" by sorted uid, so `a / b` and every directional counter are re-oriented on the way in. Guests (`id.startsWith('guest:')`) are dropped, and so is any pair whose counters sum to zero; the cap is `MAX_ARCHIVED_PAIRS = 1200` (`seasonRollover.ts:181`), chosen to keep the document under Firestore's 1 MB limit. On the real close the log read "*kept 0, dropped 293 guest, 21 empty*" — the close read 314 pair documents and archived 21, and the 293 guest documents it read are never deleted by anything except the wipe added in `HEAD`.

```
"CEkRfDs20xSuiW6xmPKLMcx3NoV2__JoLFRxFr0tTaYfVAXBPwkvT0et62": {
  "a": "CEkRfDs...", "b": "JoLFRxFr...",
  "sameTeam": 6, "against": 6,
  "winsTogether": 1, "lossesTogether": 2, "cleanSheetsTogether": 4,
  "winsA": 4, "winsB": 2,
  "assists": 0, "assistsAToB": 0, "assistsBToA": 0 }
```

Across all 21 archived pairs the directional assists sum to 7 and the legacy `assists` to 8, against `totals.assists = 13`. So the pair counters account for barely half the season's assists, and the largest directional pair total is 2 — below `MIN_DUO_ASSISTS = 3` (`src/utils/seasonAwards.ts:137`), which is why `awards.deadlyDuo` is `null`. The archive *reader's* `topDuo` (`seasonArchive.ts:128-155`) has **no** floor, so the stats screen happily names a duo on 2 assists that the season refused to crown, with ties broken by Firestore map-iteration order. Ledger `archive-duo-reader-has-no-floor`, `assist-peers-account-for-half-the-season-assists`.

One more property of this document worth naming: **the players map has no order**. `Object.entries` over a Firestore map returns whatever the wire gave it, and two consecutive reads of this immutable document really did come back differently. `parseSeasonTable` now pins `players.sort((a,b) => a.uid.localeCompare(b.uid))` (`seasonArchive.ts:114`) — added in a commit that is **not** in either shipped 1.1.9 binary (ROUND4 §1).

`seasonCards/{groupId}__{seasonId}` — the list row

Same close, same numbers, one hundredth the size, written with `merge: true` so a resume is a no-op (`seasonRollover.ts:752-809`). The hall of fame reads only this; the archive is pulled lazily when a season is opened. Read rule mirrors `seasonSummary`'s member clause but has **no** participant clause — a departed player can open the archive and not the list.

```
{ "groupId": "HhzIwmjMl1i5HSOGHt3p", "seasonId": "s1", "no": 1,
  "startsAt": 1789547794676, "endsAt": 1789661526768,
  "completedRounds": 22,
  "totals": { "rounds": 37, "goals": 27, "assists": 13 },
  "players": 7,
  "winners": [ {"key":"topScorer","names":["הלן צברי"],"value":10},
               {"key":"mvp","names":["מתן לוי","Lioz Madar","Linoy Levi","איציק לוי",
                                     "Nofar Tzabari","Eliran Tzabari","הלן צברי"],"value":6},
               {"key":"mostLoyal","names":["מתן לוי"],"value":19}, ... ] }
```

players: 7 comes from `countSeasonParticipants`, which counts rows with `games > 0` and falls back to `rounds > 0`. `totals.rounds: 37` is **mini-games**, sitting one field away from `completedRounds: 22` which is **evenings**; the Hebrew copy renders the first as "37 משחקים" (games) and the second as "22 מחזורים" (evenings), forty points apart on the same card. `winners` carries names, not uids, and they are frozen — which is correct for a record and is also why a rename breaks the title-streak logic that joins on them.

The card does **not** carry `awardsDenominator`. It did for one day; because the field is `max(players.games)` and `mostLoyal` is the maximum of the same array, `value / denominator` was 1.0 identically and the loyalty medal drew platinum for every season of every club. The field is gone from both writer and card; see `awardsDenominatorOf`'s doc comment (`seasonRollover.ts:112–132`) and `ROUND4 §5`.

`users/{uid}/seasonTitles/{groupId}__{seasonId}__{titleKey}`

The public copy of a title, on the winner's profile. Deterministic id so a redelivery overwrites rather than duplicates; the duo title's joined key `a__b` is split and written to both profiles (`seasonRollover.ts:832–859`). Deleted exactly by `reopenSeason`, `seasonRollover.ts:1217–1245`. Rules: `allow read: if isSignedIn()`; `allow write: if false`; (`firestore.rules:2010`).

```
// users/1IdtNEjbEXfiRSqVlrJVn99NsfI2/seasonTitles/HhzIwmjMl1i5HSOGHt3p__s1__mostLoyal
{ "groupId": "HhzIwmjMl1i5HSOGHt3p",
  "groupName": "מועדון שכחת שושי", // frozen: survives a club rename or the player leaving
  "seasonId": "s1", "seasonNo": 1,
  "titleKey": "mostLoyal", "value": 19,
  "at": 1789661526768 }
```

`value` is the raw winning number and its unit varies by title: 19 evenings for `mostLoyal`, 10 goals for `topScorer`, 13 mini-game wins for `topWinner`, an average of 6.0 for `mvp`, a *rate* of 1.0 for `penaltyKing`. Nothing on the document says which. The type is `SeasonTitle` (`src/types/index.ts:1029–1038`).

`roundSummaries/{gameId}` **and** `eveningStandings/{gameId}__{uid}`

Neither is a seasons collection, but both are inputs to the seasons machinery.

`roundSummaries/{gameId}` is created once per sealed evening with `create()` (`index.ts:4876` , `:5161`). Its relevance here is twofold. First, **its existence is the sweep's liveness gate**: `clubIsQuiet` (`index.ts:15466` , the unsealed-game check at `:15533–15570`) blocks a close when one of the three newest finished games has no `roundSummaries` document and `didEveningHappen` says it happened. The collection only began on 2026-08-26 — the live club has 10 summaries against 23 finished games — and a legacy close is 'happened'. Ported and run over all 91 finished games in production, **24 of 30 clubs return** `{ok:false, blocker:'unsealedGame'}` **permanently**. Second, it carries an optional `seasonEvenings` field derived as `eveningsSealed + 1 - seasons.roundsAtStart` (`index.ts:5157`) — a *sixth* answer to "how many evenings", from the counter that started late. No production document carries it; all ten predate the field.

```
// roundSummaries/TZ7IWEJWVw2itvIKCd8
{ "gameId": "TZ7IWEJWVw2itvIKCd8", "groupId": "HhzIwmjMl1i5HSOGHt3p",
  "at": 1789655700000, "generatedAt": 1789655711641, "version": 1,
  "basis": { "since": 1787739600000, "eveningsCompared": 9 },
  "coverage": { "hasRoundHistory": false, "hasAssists": false },
  "stats": { "rounds": 0, "goals": 0, "assists": 0, "ties": 0, "shootouts": 0 },
  "events": [ { "type": "club_milestone", "metric": "evenings", "threshold": 10, "total": 10 } ],
  "leaders": { "topScorers": null, "topAssisters": null, "topWinners": null,
    "topCleanSheets": null, "topGoalInvolvement": null },
  "pairHighlight": null, "teamHighlights": { "best": [], "worst": [] } }
```

`eveningStandings/{gameId}__{uid}` is a per-player, owner-read-only row (`firestore.rules:1772`): the score, its delta, and the rank before and after. Seasons touch it only by resetting the table it ranks against; `at` is stamped `Date.now()` rather than the evening's `startsAt` , so a late confirmation dates the row by the confirmation (ROUND4 §30).

Six counters, one question

The reader will meet the phrase "how many מחזורים" in six different places. They are different numbers and they disagree in production **today**:

#	Where	Derivation	Value on the live club
1	<code>clubRecords.eveningsSealed</code>	<code>increment(1)</code> per seal, since 26.08.2026	10
2	<code>groups.seasons.playedRounds</code>	seeded at open, <code>increment(1)</code> per seal of the running season	0 (s2)
3	<code>completedRoundsFrom</code> fallback	<code>eveningsSealed - roundsAtStart</code>	3
4	client games scan, <code>inSeason + didEveningHappen</code>	22 for s1, 0 for s2	22 / 0
5	<code>seasonSummary.completedRounds</code>	whatever the closing caller passed	22 (hand-repaired)
6	<code>awardsDenominatorOf = max(players.games)</code>	the eligibility gate's denominator	19

#	Where	Derivation	Value on the live club
7	<code>seasonCards.totals.rounds</code>	mini-games, mislabelled in Hebrew as games	37

`functions/src/seasonCounters.ts` was extracted specifically to make 1–4 testable side by side (`tests/logic/seasonCounterReconciliation.test.ts`), and it is the right instinct. But 5, 6 and 7 live in `seasonRollover.ts` and are never reconciled with 1–4, and only 5 and 6 decide anything irreversible.

Absence semantics, collected

Field	Absent means	Distinguishable from 0?	Who relies on it
<code>games.seasonId</code>	season 1	n/a (string)	<code>inSeason</code> , <code>eveningInSeason</code> — both sides
<code>seasons.playedRounds</code>	fall back to the sealed-counter subtraction	yes, tested with <code>typeof</code>	<code>completedRoundsFrom:110</code>
<code>seasons.roundsAtStart</code>	0	no	the subtraction fallback
<code>seasons.cadence.targetRounds</code>	<code>{kind:'none'}</code> → never due	yes	<code>seasonFinishLine:148</code>
<code>seasons.targetHistory</code>	never moved	no (<code>[]</code> ≡ absent)	nothing reads it
<code>seasons.reopenedAt</code>	no grace period	no	<code>index.ts:15979</code> , <code>:16179</code>
<code>seasonSummary.zeroedAt</code>	resume a half-finished close	yes (truthiness of a timestamp)	<code>seasonRollover.ts:667</code>
<code>seasonSummary.endedEarly / partialData</code>	not early / not partial	yes (omitted, not <code>false</code>)	card badges
<code>players[uid].csRounds / asRounds</code>	coverage unknown → divide by rounds instead	yes, deliberately	<code>seasonArchive.ts:102</code> , efficiency tab
every other archive field	0	no	<code>num()</code> fallback, <code>seasonArchive.ts:9</code>
<code>seasonSummary.closedBy</code>	manual-close metadata	yes	nothing

The last two rows are the model's structural weakness. There is no schema version on any seasons document, so a reader cannot tell an archive written by an old build from one written today, and the only defence against silent degradation is two hand-maintained `typeof` checks on two fields.

Turning seasons on, off, and on again

Seasons are switched on by one callable, `enableClubSeasons`, and off by another, `disableClubSeasons`. Both live in `functions/src/index.ts` — enable at :16286–16611, disable at :16613–16626. Neither is a document write from the client: `firestore.rules:376–395` makes the `seasons` map server-owned and immutable from the app (`!affectedKeys().hasAny(['seasons']) || request.resource.data.seasons == resource.data.seasons`), and :334–341 forbids a club from being created holding one. Enabling can archive a club's entire history and hand out permanent titles in a single call, so it belongs to the server.

The decision the admin makes is modelled as a **plan** — a pure function, `planActivation`, in `src/utils/seasonActivation.ts:90–176`, with a byte-identical server mirror at `functions/src/seasonActivation.ts` (the only textual difference is the import path, `@/utils/seasonDates` → `./seasonDates`; verified by `diff`). The same function is called three times for one activation: once by the settings screen to decide whether the button is enabled, once implicitly by the confirmation sheet which renders the plan object it is handed, and once by the server, which recomputes it from the club's own data rather than trusting the request.

The inputs, and the branch tree

```
// src/utils/seasonActivation.ts:24–37
export interface ActivationInput {
  cadence: Cadence;           // 'date' | 'rounds'
  months?: number;           // date cadence: season length
  targetRounds?: number;     // rounds cadence: evenings per season
  choice: HistoryChoice;     // 'continue' | 'sealNow'
  playedHistory: number;     // evenings the club has already played
  today: CalendarDate;       // 'YYYY-MM-DD' in the club's timezone
  season1EndsOn?: CalendarDate; // chosen last day of season 1 (date+continue)
  hasHistory?: boolean;      // the club has run seasons before
}
```

`cadence` × `choice` gives four combinations, and `planActivation` treats each separately. **מחזור** (*machzor*, an evening) is the unit throughout — `targetRounds: 24` means twenty-four game-nights, not twenty-four mini-games. Nothing in this file knows that **משחקון** (*mischakon*, a mini-game) exists.

cadence	choice	plan fields returned	meaning
rounds	sealNow	<code>startsAtRounds: 0, roundsRemaining: target</code>	history sealed as season N, season N+1 opens empty
rounds	continue	<code>startsAtRounds: playedHistory, roundsRemaining: target - playedHistory</code>	season N opens already holding the club's history

cadence	choice	plan fields returned	meaning
date	sealNow	<code>startsOn: today , endsOn: seasonEndDate(today, months) , nextStartsOn</code>	history sealed, season N+1 runs <code>months</code> from today
date	continue, date given	<code>endsOn: season1EndsOn , nextStartsOn — no startsOn</code>	season N is transitional: it reaches back as far as the club does
date	continue, no date, <code>hasHistory</code>	<code>endsOn: seasonEndDate(today, months) , nextStartsOn</code>	the re-enable escape hatch, added 18.09
date	continue, no date, first time	error <code>season1EndRequired</code>	the admin must name an end date

The rounds+continue branch carries two refusals that are the heart of the feature's arithmetic (:112–121):

```
if (playedHistory > target!) return { ...base, error: 'historyExceedsTarget' };
if (playedHistory === target) return { ...base, error: 'historyFillsTarget' };
```

A season that opens already at or past its finish line is not a season — it would be archived on the hourly sweep's next pass. `MIN_SEASON_ROUNDS = 2 ( :80 )` is the floor, with no ceiling: "a club that wants a 200-evening season is describing a long season, not a mistake." For the date cadence the bounds are `MIN_SEASON_MONTHS = 1` and `MAX_SEASON_MONTHS = 24 ( src/utils/seasonDates.ts:156–166 )`.

Why the season-1 end date exists, and only there

Under a rounds cadence a transitional first season needs no dates: it starts at `playedHistory` and ends at `target`. Under a **date** cadence with **continue**, there is no honest start date — the history stretches back to the club's first game, months or years before the admin opened the settings screen. The code's answer is to make the first season open-ended at the front and let the admin name its *end*, with the chosen `months` applying only from the following season (`seasonActivation.ts:145–176` ; the server mirrors this at `index.ts:16459–16464` by writing `cadence.startsOn = null , cadence.endsOn = plan.endsOn , cadence.endsAt = null`).

The UI that picks that date is `SeasonsSettings.tsx:818–872` — four chips labelled with the actual dates (`SEASON1_MONTH_CHOICES = [1, 2, 3, 6]` , :67) plus a custom stepper. It is rendered inside `{firstTime ? ... }` (:795), and

```
// src/components/community/SeasonsSettings.tsx:309
const firstTime = (seasons?.count ?? 0) === 0 && !live;
```

Defect `live-club-cannot-use-date-cadence (P0)`. Until commit `ae2c042` (18.09 16:53) `planActivation` returned `season1EndRequired` whenever `season1EndsOn` was missing, with no `hasHistory` escape. Combined with the `firstTime` gate above, the date cadence was *unreachable for the rest of a club's life*: a club already running seasons, or any club with `count > 0` switching the feature back on, saw the enable button permanently disabled and a red line reading "1 בחרו תאריך סיום לעונה 1." ("Choose an end date for season 1.") —

naming a season it had archived, with no date control anywhere on the screen. The server half was patched first (`index.ts:16428–16432` derives the date itself when the client sends none, for the benefit of 1.1.7 clients in the store); the client half followed two commits later.

This fix is not in the shipped app. The store build is 1.1.9 (Android vc235, iOS 103) at commit `4de80d0` , 18.09 08:54 IDT. `git show 4de80d0:src/utils/seasonActivation.ts | grep -c hasHistory` returns `0`. Every user on the current release still has the dead date cadence; only the server-side derivation protects them, and only when the client lets them press the button, which it does not.

A related residue, ROUND4 finding 21, was fixed at HEAD: the `hasHistory` branch originally returned `ends0n` alone, so the confirmation sheet — which renders each date line only when its field is present (`SeasonConfirmSheet.tsx:180, :186, :196`) — showed an end date and no "מתחילה ב N+1 עונה". `seasonActivation.ts:161–170` now derives `nextStarts0n` too.

Note also that the **server never passes** `hasHistory` (`index.ts:16433–16454`). It reaches the same outcome by a different route, pre-computing `season1Ends0n` itself. Two code paths to one answer is a maintenance hazard in a file whose entire premise is "one plan, computed identically in three places".

The seeding: two counters, two eras

This is the part that produces wrong numbers on real clubs. When a season opens, two fields are stamped onto `groups/{id}.seasons` :

```
// src/utils/seasonSeed.ts:22–30 (and functions/src/seasonSeed.ts, identical)
export function seasonSeed(sealedEvenings: number, playedHistory = 0) {
  return { roundsAtStart: num(sealedEvenings), playedRounds: num(playedHistory) };
}
```

They come from **different sources covering different eras**, and the call site makes that explicit (`index.ts:16573–16576`):

```
...seasonSeed(
  await sealedEveningsOf(groupId),
  closedSoFar > 0 ? 0 : await playedEveningsFromGames(groupId),
),
```

field	source	what it counts	era it can see
roundsAtStart	<code>sealedEveningsOf</code> → <code>clubRecords/{groupId}.eveningsSealed</code> (<code>index.ts:15578–15583</code>)	every evening ever <i>sealed</i> , all-time, never resets, never decrements	from 26.08.2026 only — the counter did not exist before evening-sealing shipped
playedRounds	<code>playedEveningsFromGames</code> → scan of up to 200 terminal <code>games</code> , filtered by <code>didEveningHappen</code> (<code>index.ts:15655–15670</code>)	every evening the club actually held	the club's whole life, capped at 200 documents

`roundsAtStart` is a *zero mark* for the all-time counter, so that every future seal adds exactly one to a season-relative figure. `playedRounds` is the number the club is **shown** on its card and the number the rollover closes on. They are not the same quantity and must not be compared, but because both are "rounds" and both are seeded in the same expression, the codebase has confused them repeatedly.

On the one club that has ever run seasons, `HhzIwmjMl1i5HS0GHt3p` (מועדון שכחת שושי), the gap between the two eras is exactly measurable today:

```
clubRecords/HhzIwmjMl1i5HS0GHt3p.eveningsSealed = 10
games scan (23 terminal docs, 22 with didEveningHappen = 'happened') = 22
```

Twenty-three terminal games run 28.06.2026 → 18.09.2026. Nineteen of them predate the season stamp entirely and carry no `seasonId`; three carry `s1`; one (18.09, `DTNscolR...`) carries `s3` and is `playVerified: false`, so it does not count. The counter sees ten of those twenty-two because it only began on 26.08.

Defect (historic, fixed 18.09 in 0bb0812). `playedRounds` was originally seeded from `eveningsSealed` too. The club turned seasons on, asked for 24, and the card read "7 24 מתוך" ("7 of 24") — not its history (19 at the time), not a fresh start (0), but however far a two-week-old counter had got. The owner's report is quoted verbatim in the source: "שיחקנו 19 ולא 7" ("we played 19, not 7"). The fix routes `playedRounds` through the `games scan`, which is also what the club screen's "מפגשים שנערכו" has always counted.

The client's half of the same number comes from a different function again:

`gameService.getCommunityStats(groupId).totalFinished` (`SeasonsSettings.tsx:340–351`). That scan is also `limit(200)` over `status` in `['finished', 'cancelled']` and also asks `eveningPlayState`, so on the real club the two agree exactly at 22 — but it is a *coincidence of construction*, not a shared module, and `getCommunityStats` additionally applies a season filter (`inSeason`, `src/utils/seasonScope.ts:28–36`) when it is given a scope. The settings screen passes none, so the filter is inert there. Nothing enforces that.

Note the seeding is **skipped for a re-activation**: `closedSoFar > 0 ? 0 : ...`. A club that has closed a season before starts the next one at zero, because those evenings are already sealed inside an archive. Before that guard existed, a club with 22 evenings behind it could not open a 2-round season at all — the server answered `failed-precondition` and the app showed "משהו השתבש" ("something went wrong").

The `sealNow` branch

```
// functions/src/index.ts:16466–16512 (trimmed)
if (choice === 'sealNow') {
  const quiet = await clubIsQuiet(groupId);
  if (!quiet.ok) throw new HttpsError('failed-precondition', quiet.blocker ?? 'busy');
  const firstNo = closedSoFar + 1;
  await closeSeason({
    db, groupId, seasonId: `s${firstNo}`, seasonNo: firstNo,
    startsAt: 0, // display resolves the club's first game
    completedRounds: played, // ← see below
    roundsAtStart: 0,
    partialData: true, // assists only from 21.06, clean sheets from 17.08
    now,
  });
  await ref.set({ seasons: {
    enabled: true, currentNo: firstNo + 1, currentId: `s${firstNo + 1}`,
    startedAt: now, roundsAtStart: sealedAllTime, playedRounds: 0,
    reopenedAt: 0, cadence: rebaseCadence(cadence, now, now),
    targetHistory: [], count: firstNo,
  } }, { merge: true });
  await announceSeasonClosed({ ... });
  return { ok: true, closedSeasonNo: firstNo, currentNo: firstNo + 1 };
}
```

Defect `enable-sealnow-archives-counter-not-games (P1)`. `completedRounds: played` where `played` is computed at `:16328`:

```
const played = await completedRoundsOf(groupId, existing?.roundsAtStart);
```

Two arguments, not three — so `completedRoundsFrom` (`functions/src/seasonCounters.ts:105–118`) takes the fallback path and returns `eveningsSealed - roundsAtStart`. On a first activation `roundsAtStart` is absent, so this is simply `eveningsSealed`. Meanwhile the confirmation sheet the admin approved showed `plan.playedHistory`, which is the **games scan** (`SeasonConfirmSheet.tsx:159`, rendering `he.seasonsConfirmSealedNow(plan.playedHistory)` → "20 20", תיסגר כעת — קיימים מחזורים קיימים existing rounds — will be sealed now"). `closeSeason` writes that argument straight into the archive card (`functions/src/seasonRollover.ts:625–627`). Had שכחת שוישי taken this branch on 16.09, the sheet would have said **20** and the permanent archive would have stored **7**. The archive is created with `summaryRef.create()` — there is no second chance.

The club took `continue` instead, so the defect is unexercised in production; the sealed card `seasonCards/HhzIwmjMl1i5HSOGht3p__s1` today reads `completedRounds: 22`, written later by `endSeasonNow` after its own two-argument bug was repaired. Its `startsAt` is `1789547794676 = 16.09.2026`

11:36, the enable moment — which is why the hall of fame prints "2026 ספט – 2026 ספט" over a season containing three months of football.

Note also `roundsAtStart: sealedAllTime` here versus `seasonSeed(await sealedEveningsOf(...))` in the `continue` branch: the same figure, read twice from the same document in one invocation (`:16338` and `:16574`). Harmless, but it is a second Firestore read per activation.

The `continue` branch and the silent clamp

```
// functions/src/index.ts:16523–16531
if (cadence.type === 'rounds' && closedSoFar === 0) {
  cadence.targetRounds = Math.max(cadence.targetRounds ?? 0, played + 1);
}
```

Defect `rounds-clamp-uses-a-different-history (P2)`. The plan refused a target below `playedHistory` using the **games scan**; this clamp raises the target using `played`, i.e. `eveningsSealed`. Two histories, one decision. When the counter is *lower* than the scan — the normal case today, 10 against 22 — the clamp is a no-op and the plan's refusal governs. When the counter is *higher* — a club past the 200-document scan window, or one that has deleted games (deleting a game decrements nothing, which is precisely why `sealedEveningsOf` exists) — the clamp silently rewrites the admin's choice. The source comment names the failure: an admin choosing 24 gets 201. No `targetHistory` entry is written, no error is returned, and `SeasonConfirmSheet` has already promised 24. This is finding `approved-rounds-target-is-not-the-one-written` from the other side.

Switching off, and switching back on

`disableClubSeasons` is four lines of effect:

```
// functions/src/index.ts:16621–16624
const seasons = group.seasons as { enabled?: boolean } | undefined;
if (!seasons?.enabled) return { ok: true, alreadyOff: true };
await ref.set({ seasons: { enabled: false } }, { merge: true });
```

It deliberately does **not** close the running season — the dialog says so (`he.seasonsDisableBody`): "העונה שרצה" ("the season running now will not be closed and no titles will be awarded for it"). Everything else on the block survives, `count` included, so numbering continues: a club that ran seasons 1–3 re-opens at 4, never at 1.

The same dialog also promises: "ואם תפעילו עונות שוב הספירה תמשיך מהמקום שבו עצרה" — "**and if you turn seasons on again the count will continue from where it stopped.**"

Defect `disable-reenable-zeroes-running-season (P1)`. It does not. Re-enabling runs the `continue` branch with `closedSoFar > 0`, so `seasonSeed(eveningsSealed, 0)` writes `playedRounds: 0` and `startedAt: now`. Every evening the season had accumulated while the feature was on is erased from the card,

and the promise in the dialog is exactly inverted. The one in-between case is a club with `count === 0` that never closed anything: it is re-seeded from the games scan, which is *closer* to correct but still discards whatever the live season held.

QA Test Club (6zotsPlu5Wa18hIysQif) is sitting in that state right now. It is the only other club in the database of 195 groups with a `seasons` block:

```
enabled: false      count: 0      currentNo: 1      currentId: "s1"
playedRounds: 3     roundsAtStart: 0   reopenedAt: 1789509161010 (15.09)
cadence: { type: "rounds", targetRounds: 3, months: null, endsAt: null }
clubRecords/6zotsPlu5Wa18hIysQif: does not exist → sealedEveningsOf = 0
terminal games for this group: 0
```

Press "הפעל עונות" on it today and: `closedSoFar = 0` → `firstTime` on the client is true → `playedHistory = playedEveningsFromGames = 0` → `seasonSeed(0, 0)` → `playedRounds: 0`. Three evenings of progress replaced by zero, and `currentId` re-issued as `s1` — the same id the previous run used. Its `cadence` map also shows the merge hazard the code warns about: it has `months` and `endsAt` keys but no `startsOn` / `endsOn` at all, because it was written by an older build; `merge: true` into a nested map merges field by field, which is why the current enable path writes explicit `null`s on the unused half (`:16362–16368` , `:16386–16394`).

Defect `season-id-reissued-with-live-stamps (P1)`. `count` advances only on a *close*, so disable + enable re-issues the *same* `currentId` while games from the previous run still carry that stamp. The card, seeded to 0, and the season-scoped statistics screen, which counts stamped games via

`playedEveningsOfSeason / countSeasonEvenings` , then answer the same question differently — one says 0, the other says however many games bear the stamp. The live club already shows the general shape of stamp drift: `groups/HhzIwmjMl1i5HS0GHt3p.seasons.currentId` is `"s2"` , and game `DTNscolR...` played on 18.09 is stamped `"s3"` — a season the club document has never heard of.

Defect `reenable-inherits-stale-reopenedat (P2)`. The `sealNow` branch writes `reopenedAt: 0` (`:16502`); the `continue` branch does not write the field at all, and `merge: true` preserves it. `reopenedAt` buys a season 48 hours of immunity from closing (`REOPEN_GRACE_MS` , checked at `:15977–15979` on the seal path and `:16178–16182` in the hourly sweep). So a brand-new season opened by a re-enable within two days of an undo inherits the predecessor's grace and cannot be closed by either mechanism, however far past its target it is. שכיח carries `reopenedAt = 1789727847680 = 18.09.2026 10:37` today.

The same write also sets `targetHistory: []` unconditionally (`:16578`), discarding the audit trail of every finish-line move the club ever made — finding `reenable-wipes-targethistory` . On the live club that array currently holds one entry (a change from `targetRounds: 2` to `targetRounds: 24` by *Eliran Tzabari* at 18.09 10:37:54), and a disable/enable cycle would delete it.

Refusals, and whether the admin ever sees them

Every way `enableClubSeasons` can say no, and what the user gets:

thrown	code	client mapping (seasonService.ts)	Hebrew shown
sign-in required	unauthenticated	none	generic
groupId required	invalid-argument	null unless message contains targetRounds	generic
club not found	not-found	null (only no archive maps)	generic
admin only	permission-denied	notAdmin	"רק מנהל המועדון יכול לסיים עונה."
seasons already on	failed-precondition	seasonsAlreadyOn	"עונות כבר פעילות במועדון. רעננו את המסך..."
targetRounds required	invalid-argument	targetInvalid	"עונה חייבת להכיל לפחות 2 מחזורים."
season-plan: <error> ×6	failed-precondition	lengthInvalid / targetInvalid / historyExceedsTarget / historyFillsTarget / seasonEndRequired / seasonEndPast	the six written explanations
openGame / unsealedGame (sealNow only)	failed-precondition	openGame / unsealedGame	"יש מחזור פתוח במועדון..." / wait-for-seal

The mapping lives in `refusalOf` (`src/services/seasonService.ts:128–183`) and is turned into text by `seasonRefusalText` (`:77–126`), which takes a `SeasonRefusalContext` carrying the real season number — because every one of these messages was originally written about "עונה 1" while numbering continues across a disable/enable cycle.

Defect `season-plan-refusals-unmapped (P2)`. `git show 4de80d0:src/services/seasonService.ts | grep -c 'season-plan:'` returns 0. In the shipped 1.1.9 build all six plan refusals fall through to `he.seasonActionFailed` — "משהו השתבש. נסו שוב עוד רגע" ("Something went wrong. Try again in a moment.") — which is advice that can never work, printed beside a destructive button and an invitation to press it again. The six Hebrew explanations exist in `he.ts` (`:2333–2345`) and are reachable today only when the *client's own* `planActivation` reaches the same verdict first, which by construction it cannot when the disagreement is between client and server histories. The screen also renders the plan error inline (`SeasonsSettings.tsx:901–905`) and disables the button while `!plan.ok` (`:943–949`), so in practice a server-side plan refusal means the two sides disagreed — exactly the case where the generic line is least useful.

Two invariants this area is trying to hold, and their status:

- "The summary a person approves is the thing that actually happens." **Does not hold**. The clamp at `:16531` can raise an approved rounds target, and `sealNow` archives a different evening count from the one the sheet displayed.

- *"Numbering continues across the feature being switched off and on."* **Holds for** `count` **and** `currentNo` , **but not for the data.** The id is reused while old games keep the stamp, and the season's progress is reset to zero against a dialog that promised the opposite.
-

How a מחזור is counted — the three counters

A מחזור (*machzor*) is one game-night. This club plays one a week. A מישחקון (*mischakon*) is one mini-game inside that night; there are typically four to six of them, and they exist only in "advanced mode". The one club that has ever run a season is on the plain timer: across its ten sealed evenings it recorded `stats.rounds` of 4, 2, 0, 3, 2, 4, 0, 0, 0, 0 — and zero on every evening since the feature was switched on. Everything in this section counts **evenings**. Where a number in the codebase is called `rounds` and means mini-games, it is named as such.

The app holds three independent, separately-written answers to "how many evenings has this club played", plus four server derivations and two frozen archive copies. None of them is a view of another. This section is the map of all of them.

The three writers

#	Where it lives	Written by	Unit	Resets?	Client can read it?
1	<code>clubRecords/{groupId}.eveningsSealed</code>	server, <code>FieldValue.increment(1)</code>	evenings sealed since 26.08.2026	never	no — rules deny
2	<code>groups/{groupId}.seasons.playedRounds</code>	server, seeded + <code>increment(1)</code>	evenings in the running season	zeroed by every season-opening path	yes
3	nothing — derived on the phone	<code>gameService.getCommunityStats</code> scan	evenings in the 200-newest terminal games	n/a (recomputed each read)	it is the client

Counter 1 — `clubRecords.eveningsSealed`

The oldest of the three, and the only one that is *exact*. It is written in one place, `functions/src/index.ts:5194`, inside the batch at the bottom of `sealRoundSummary`:

```
// functions/src/index.ts:5188–5196
batch.set(
  db.collection('clubRecords').doc(groupId),
  {
    groupId,
    ...next,
    // INCREMENT, not `read + 1`.
    eveningsSealed: admin.firestore.FieldValue.increment(1),
```

The comment above it records the incident that forced the atomic form: three evenings sealed within a minute on the QA club left three summaries and a counter of two. It is safe to increment because `summaryRef.create()` at `:5162` throws if this evening was already sealed, so the batch runs at most once per

game.

Three properties matter, and all three are traps.

It began on 26.08.2026. `sealRoundSummary` shipped in commit `59cd9b6` (2026-08-25). Production proves the date exactly: `clubRecords/HhzIwmjMl1i5HS0Ght3p` carries `since: 1787739600000 = 2026-08-26T10:20:00Z`, which is the `startsAt` of `games/T7qRTLRO...`, the club's first sealed evening. Every evening before it — twelve of them, from 28.06 to 24.08 — has **no** `roundSummaries` **document at all** and is invisible to this counter. I enumerated the club's 23 terminal games and probed `roundSummaries/{gameId}` for each:

Evening (UTC)	roundSummary	basis.eveningsCompared	mini-games
28.06 – 24.08 (12 evenings)	absent	—	—
26.08 10:20	present	0	4
31.08 13:35	present	1	2
07.09 05:25	present	2	0
08.09 06:35	present	3	3
11.09 05:35	present	4	2
14.09 09:40	present	5	4
15.09 13:00	present	6	0
16.09 06:20	present	7	0
17.09 14:30	present	8	0
17.09 14:35	present	9	0
17.09 21:19	absent (auto-closed, <code>playVerified:false</code>)	—	—

`eveningsCompared` is `eveningsSealed` as read at the top of that seal (`index.ts:5153`), so the ladder 0→9 is the counter's own history. It stands at **10** today. The club has played **22**.

Nothing decrements it. The deletion handler at `index.ts:5357–5391` writes a `gameDeletions` audit row and sends pushes; it does not touch `clubRecords` or `seasons`. Delete a sealed evening and the counter keeps it. This is deliberate — `src/utils/seasonLifecycle.ts:34–37` states it as the reason to prefer the counter over a query — but it means counter 1 and counter 3 drift permanently apart on any deletion, in the opposite direction from the 26.08 blindness.

The client cannot see it. `firestore.rules:1813` is `allow read, write: if false`. This is why counter 2 exists at all.

Counter 2 — `seasons.playedRounds`

The mirror. It is the number the club card shows, the number the hourly sweep closes on, and the number `endSeasonNow` archives. `src/components/community/SeasonsCard.tsx:151` :

```
<Text style={styles.big}>{seasons.playedRounds ?? 0}</Text>
<Text style={styles.big0f}>{he.seasonsCardOfTarget(cadence!.targetRounds as number)}</Text>
```

`he.seasonsCardOfTarget` (`src/i18n/he.ts:2126`) renders מתוך {target} מחזורים ("out of {target} evenings"), so the card reads e.g. "מתוך 24 מחזורים 0".

It is **seeded** at activation and **incremented** at every qualifying seal (`index.ts:5216–5236`):

```
if (seasonRoundsAtStart !== null) {
  batch.set(
    db.collection('groups').doc(groupId),
    { seasons: { playedRounds: admin.firestore.FieldValue.increment(1) } },
    { merge: true },
  );
}
```

`seasonRoundsAtStart` is non-null only when the evening belongs to the *running* season (`index.ts:5092–5106`):

```
if (sea?.enabled) {
  tableZeroedAt = num(sea.roundsAtStart);
  const stamp = typeof args.seasonId === 'string' ? args.seasonId : '';
  const mine = stamp ? stamp === sea.currentId : sea.currentNo === 1;
  if (mine) seasonRoundsAtStart = tableZeroedAt;
  else { console.log('[season] seal is not for the running season – progress not credited', ...); }
}
```

That `mine` expression is the stamp rule (below), and it is why an admin confirming a forgotten evening from an archived season no longer pushes the current season one evening closer to its target.

The **seed** comes from `src/utils/seasonSeed.ts` (mirrored at `functions/src/seasonSeed.ts`), called at `index.ts:16572–16575` :

```
...seasonSeed(
  await sealedEveningsOf(groupId), // → roundsAtStart
  closedSoFar > 0 ? 0 : await playedEveningsFromGames(groupId), // → playedRounds
),
```

so season 1 of a first-time club is seeded with the **games count**, and every later season with **0**. Season 1 owns the club's history; season 5 does not.

Every path that opens a season zeroes it: `index.ts:15905` (the rollover's lifecycle write), `:16501` (the `sealNow` activation branch), `:17057` (`endSeasonNow`). The `continue` activation branch seeds it via `seasonSeed`. The reopen recomputes it from the games (`:16925–16934`).

Because the field is zeroed by three paths and written by `merge:true set()`s, `disable → re-enable` on a club with a running season re-seeds `playedRounds: 0` even though the disable dialog promises `הספירה תמשיך` ("the count will continue from where it stopped") — LEDGER `disable-reenable-zeroes-running-season`. This is not hypothetical on the live club: `seasons.startedAt` is `2026-09-17T21:05:41Z`, five hours *after* season 1 closed at 16:12Z, which is the signature of exactly that re-open.

What happens when it is absent. `completedRoundsFrom` (below) falls through to counter 1 minus the offset. That fallback is the single most expensive line in the feature's history.

Counter 3 — the client scan

`src/services/gameService.ts:949`, `getCommunityStats(groupId, season?)`. One query, one pass, two tallies:

```
// src/services/gameService.ts:1112-1118
const q = query(col.games(), where('groupId','==',groupId),
  where('status','in',['finished','cancelled']),
  orderBy('startsAt','desc'), limit(200));
```

```
// src/services/gameService.ts:1150-1186 (trimmed)
for (const doc of snap.docs) {
  const g = doc.data();
  {
    // — LIFETIME, before the season gate
    const st = eveningPlayState(g as PlayableEvening);
    if (g.status === 'cancelled' || st === 'notHappened') lifeCancelled += 1;
    else if (st !== 'unverified') { lifeFinished += 1; /* ...attendance... */ }
  }
  if (!inSeason(g as { seasonId?: string }, season)) continue;
  const state = eveningPlayState(g as PlayableEvening);
  if (g.status === 'cancelled') { totalCancelled += 1; continue; }
  if (state === 'unverified') continue; // counts as NEITHER half of the rate
  if (state === 'notHappened') { totalCancelled += 1; continue; }
  totalFinished += 1;
  ...
}
```

Two exported rules do all the deciding — `inSeason` (`src/utils/seasonScope.ts:28`) and `eveningPlayState`. Neither is re-implemented here, which is new: until 18.09 the season rule was a closure inside this function, and the only test of it was a copy of the rule re-declared inside `tests/logic/seasonScopedStats.test.ts` (LEDGER `inseason-tested-as-a-private-copy`).

`totalFinished` is the season-scoped answer; `lifetime.totalFinished` is the all-time one. The screen picks between them at `src/screens/communities/CommunityStatsScreen.tsx:769`:

```
value={scopedCard ? scopedCard.completedRounds
  : scope.k === 'all' ? (stats?.lifetime?.totalFinished ?? 0)
  : (stats?.totalFinished ?? 0)}
label={he.communityStatsEvenings} // 'מחזורים'
```

so one tile can render three different numbers for the same club depending on which chip is selected — by design, but see the matrix.

roundsAtStart — the offset, and why its source must match the counter's

roundsAtStart exists only to make counter 1 usable. Counter 1 never resets, so a season must remember its own zero; completedRoundsFrom subtracts:

```
// functions/src/seasonCounters.ts:105–118
export function completedRoundsFrom(
  sealedEvenings: number, roundsAtStart?: number, playedRounds?: number,
): number {
  if (typeof playedRounds === 'number' && playedRounds >= 0) return playedRounds;
  const all = typeof sealedEvenings === 'number' && sealedEvenings > 0 ? sealedEvenings : 0;
  const base = typeof roundsAtStart === 'number' && roundsAtStart > 0 ? roundsAtStart : 0;
  return Math.max(0, all - base);
}
```

The invariant the subtraction needs is: roundsAtStart **must be a reading of the same counter, taken at the season's start**. It is — seasonSeed's first argument and index.ts:16500, :17056, :15885 are all sealedEveningsOf(groupId), i.e. clubRecords.eveningsSealed.

The invariant that does **not** hold is the one the fallback silently assumes: that eveningsSealed – roundsAtStart equals the season's evenings. It only equals the season's evenings *since the counter started*. Season 1 of this club was seeded {roundsAtStart: 7, playedRounds: 19} on 2026-09-16T08:36:34Z — 7 from the counter, 19 from the games. Three seals later the two had moved in lockstep (10 and 22), but the difference was 3 and the truth was 22. Two shipped code paths reached for that difference:

- endSeasonNow called completedRoundsOf(groupId, seasons.roundsAtStart) with two arguments, so playedRounds was undefined and the fallback fired. Pressing "סיים עונה עכשיו" would have sealed 3 into a write-once archive for a season the card said was 22. (LEDGER end-season-now-ignores-playedrounds; fixed, the fix is the comment at index.ts:17022–17034.)
- reopenSeason derived playedRounds the same way, then — after that was fixed — from stamped games only, which gave 3 again from the other side (LEDGER reopen-season1-drops-the-19-unstamped-evenings).

Both fixes are in functions/src/. Note ROUND 4 finding 17: HEAD's committed functions/lib/ is the compiled output of an older commit and seasonCounters.js is untracked, so what git records as "built" is not what runs.

The four server derivations

All four live in `functions/src/index.ts` and delegate their *rules* to `functions/src/seasonCounters.ts`, which exists precisely because they used to be four inline loops no test could reach.

Function	Line	Source	Bound	Returns on failure
<code>sealedEveningsOf</code>	15578	<code>clubRecords.eveningsSealed</code>	1 doc	0
<code>playedEveningsOfSeason</code>	15599	games query + <code>countSeasonEvenings</code>	<code>limit(300)</code>	-1 (sentinel)
<code>playedEveningsFromGames</code>	15655	games query + <code>countPlayedEvenings</code>	<code>limit(200)</code>	0
<code>completedRoundsOf</code>	15695	<code>completedRoundsFrom(...)</code>	0-1 docs	—

```
// functions/src/index.ts:15695-15708
async function completedRoundsOf(
  groupId: string, roundsAtStart?: number, playedRounds?: number,
): Promise<number> {
  if (typeof playedRounds === 'number' && playedRounds >= 0) {
    return completedRoundsFrom(0, roundsAtStart, playedRounds);
  }
  return completedRoundsFrom(await sealedEveningsOf(groupId), roundsAtStart);
}
```

Three observations a reviewer should hold on to.

1. **The three game-scans use three different windows.** The client reads 200 docs, `playedEveningsFromGames` reads 200, `playedEveningsOfSeason` reads 300. For a club past 200 terminal games — none exist yet; the largest in the database is this one at 23 — season 1's unstamped evenings truncate differently on each surface. The doc-comment above `playedEveningsOfSeason` still claims the 200 bound (ROUND 4 24).
2. **Only `playedEveningsOfSeason` distinguishes "zero" from "I could not count".** Its -1 is honoured by exactly one caller, the reopen at `index.ts:16925-16934`. `playedEveningsFromGames` returns 0 on a failed query, and 0 is indistinguishable from a brand-new club — so a transient failure during activation seeds season 1 at zero and the club's whole history is silently dropped from its first season.
3. `completedRoundsFrom(0, roundsAtStart, playedRounds)` deliberately passes 0 as the sealed count so that the fallback branch is unreachable when the mirror is present; the `clubRecords` read is skipped entirely. Good, but it means the two branches of one function are fed by different arities at eight call sites, which is how the two-argument `endSeasonNow` bug survived review.

The frozen fourth and fifth answers

When a season closes, its evening count is copied twice and never recomputed:

- `seasonSummary/{groupId}__{seasonId}.completedRounds` — the archive, written with `create()` (`functions/src/seasonRollover.ts:584`), read by the server.
- `seasonCards/{groupId}__{seasonId}.completedRounds` — the list row, read by `src/services/seasonHistoryService.ts:82` and rendered by `SeasonHistoryScreen` , `SeasonPoster.tsx:201` and the hero tile above.

For a **running** season the same field is synthesised from counter 2 — `src/services/seasonSummaryService.ts:512` , `completedRounds: num(seasons.playedRounds)` . So `SeasonSummaryScreen` prints `he.seasonClubRounds(model.completedRounds)` (" {n} מחזורים שוחקו בעונה הזאת " — "{n} evenings were played in the club this season") from counter 2 while the season runs and from the frozen archive afterwards. Those are different numbers with the same name, and nothing recomputes the frozen one if the counter it was copied from was wrong.

The matrix — מועדון שכחת שושי , today

Group `HhzIwmjMl1i5HSOGht3p` .Live seasons block: `{enabled:true, currentId:'s2', currentNo:2, count:1, roundsAtStart:10, playedRounds:0, startedAt:1789679141952 (17.09 21:05Z), reopenedAt:1789727847680 (18.09 10:37Z), cadence:{type:'rounds', targetRounds:24}}` . `clubRecords.eveningsSealed:10` . 23 terminal games, all finished , of which by `eveningPlayState: 22` happened , `1` notHappened (`DTNscolR...` , auto-closed then `playVerified:false`). Stamps: **19 carry none**, **3 carry s1** , **1 carries s3** — and `s3` does not exist.

Question asked of	Expression	Answer today	Why
Counter 1	<code>clubRecords.eveningsSealed</code>	10	blind to the 12 evenings before 26.08
Counter 2 (the card)	<code>seasons.playedRounds</code>	0	zeroed when s2 re-opened; no seal since
Counter 3, season scope	<code>getCommunityStats(g, {s2,2}).totalFinished</code>	0	no game carries an s2 stamp, and currentNo is 2 so unstamped games are not admitted
Counter 3, lifetime	<code>... lifetime.totalFinished</code>	22	22 happened in the 200-doc window
Counter 3, org. rate (lifetime)	<code>lifeFinished/(lifeFinished+lifeCancelled)</code>	22/23 = 95.7%	the notHappened evening is the denominator's other half
<code>sealedEveningsOf</code>	—	10	
<code>playedEveningsOfSeason(g, 's2', 2)</code>	<code>countSeasonEvenings</code>	0	
<code>playedEveningsOfSeason(g, 's1', 1)</code>	<code>countSeasonEvenings</code>	22	19 unstamped + 3 stamped s1 , all happened

Question asked of	Expression	Answer today	Why
<code>playedEveningsFrom Games(g)</code>	<code>countPlayedEvenings</code>	22	
<code>completedRoundsOf(g, 10, 0)</code> (3-arg, every close)	mirror	0	
<code>completedRoundsOf(g, 10)</code> (2-arg fallback)	<code>10 - 10</code>	0	agrees <i>by coincidence</i> : no seal since s2 opened
<code>seasonCards... __s1.completedRounds</code>	frozen	22	
<code>seasonSummary... __s1.completedRounds</code>	frozen	22, with <code>roundsAtStart0 fSeason: 7</code>	
<code>seasonSummary... __s1.players[*].games</code>	rollup	max 19	three evenings never credited attendance
<code>seasonSummary... __s1.totals.rounds</code>	mini-games	37	the other unit, same screen family

Read that matrix as a reviewer: the club card says **0** **מחזורים 24 מתוך**, the stats screen's "מחזורים" tile on the *current* chip says **0**, on the *all-time* chip says **22**, on the *1 עונה* chip says **22**, and the archive of season 1 simultaneously says 22 evenings, 19 games-attended for its best attendee, and 37 משחקונים. Every one of those is produced by a different expression and only two of them share a source.

The 2-argument fallback reading 0 today is worth dwelling on: it is *right by accident*. It was 3 on 17.09 (`10 - 7`) and will be wrong again the moment `eveningsSealed` moves while `roundsAtStart` does not — i.e. on the next evening this club plays.

One more live consequence: `roundsAtStart (10) === eveningsSealed (10)` means the next seal computes `seasonEvenings = eveningsSealed + 1 - tableZeroedAt = 1 (index.ts:5157)`, which is the "first evening of a season" case (`functions/src/roundSummary.ts:698`). No production `roundSummaries` document carries `seasonEvenings` at all — the field post-dates the three seals of 16–17.09 — so that suppression has never actually fired in production (ROUND 4 **29**, and LEDGER's rank-movement findings).

`src/utils/eveningPlayed.ts` — the one thing allowed to say "it happened"

248 lines, mirrored byte-for-byte into `functions/src/eveningPlayed.ts` below a 15-line header; `tests/logic/eveningPlayedMirror.test.ts` fails if they drift (though see LEDGER `mirror-each-ignores-its-own-params`: the `describe.each` declares five module pairs and the body hard-codes this one, so the other four mirrors are not actually compared).

Four states (`src/utils/eveningPlayed.ts:70–71`):

State	Meaning	Counts?
happened	it took place	yes, everywhere
notHappened	cancelled, or an admin said so	no — counts as a cancellation in the organisation rate
unverified	the sweep closed it, it left no trace, nobody has said	neither — excluded from both halves of the rate
pending	not over yet	nothing to decide

Three fields decide it: `status`, `endedBy` (`'admin'` | `'auto'`), and `playVerified` (`boolean`, set only by an admin resolving an unverified evening). Everything else feeds `playEvidence`.

```
// src/utils/eveningPlayed.ts:186–226 (trimmed)
export function eveningPlayStateWithReason(game) {
  if (!game) return { state: 'pending', reason: null };
  if (game.status === 'cancelled') return { state: 'notHappened', reason: null };
  if (game.status !== 'finished') return { state: 'pending', reason: null };
  if (game.playVerified === true) return { state: 'happened', reason: 'adminVerified' };
  if (game.playVerified === false) return { state: 'notHappened', reason: null };
  if (game.endedBy === 'admin') return { state: 'happened', reason: 'manualCompletion' };
  const evidence = playEvidence(game);
  if (evidence) return { state: 'happened', reason: evidence };
  if (game.endedBy === 'auto') return { state: 'unverified', reason: null };
  if (isLegacyClose(game)) return { state: 'happened', reason: 'legacy' };
  return { state: 'unverified', reason: null };
}
```

`playEvidence` (:120–172) is an ordered list of signals, most direct first: the kickoff stamp or a live `phase`; closed timer windows (`activeIntervals`); any timer press (`timerEvents`, `timerAccumulatedMs`, `timerLastStartedAt`); a server-aggregated mini-game (`committedRoundCount`); uncommitted scoreboard goals or a positive score; a committed `rotation`. There is deliberately no minimum duration — "a night that ran four minutes is still a night". A timer-only club can only ever produce the `timer` signals, and that is not a branch, it is what the list evaluates to when the other fields cannot exist.

Why the absence of `endedBy` is load-bearing. `isLegacyClose` (:181–183) is `game.endedBy !== 'admin' && game.endedBy !== 'auto'`. Every evening closed before this module shipped carries no `endedBy`, so it falls through to `happened` with reason `'legacy'` and counts exactly as it always did. That is the entire backward-compatibility story: no migration, no backfill, and not one historical number moves. On this club it is 19 of the 22 evenings — including the three with no timer, no rotation and no goals (06.07, 16.07, 04.08), which have *no evidence whatsoever* and are `happened` purely because nobody recorded how they ended. Flip the default and the club loses three evenings and its season-1 archive becomes uncomputable.

The corollary is the sharp edge: `endedBy: 'auto'` with no evidence is the *only* route to `unverified`. Once the sweep started stamping `endedBy`, the system began declining to guess — and `DTNscolR...` is the first such evening in the database.

Why nothing may re-derive it. Before this module there were two answers: the server credited attendance on `liveMatch.startedAt` alone, the client asked only `status === 'finished'`. A night the server refused to count appeared in players' totals and in the club's organisation rate. Today the callers are `gameService.getCommunityStats ( :1156 , :1179 )`, the game list (`:327 , :1703`), the attendance scan (`:867 , :2107`), `countSeasonEvenings` and `countPlayedEvenings` (`functions/src/seasonCounters.ts:78 , :86`), and the seal trigger's transition gate (`index.ts:5868–5872 , !wasHappened && isHappened`). The rule that matters for a reviewer: **if any surface re-derives "did this evening happen" from a timer field, a goal array or a status string, the three counters can no longer be reconciled**, because the arbiter would no longer be single.

The `seasonId` stamp

Who writes it: exactly one place, `functions/src/index.ts:5802–5850`, inside `onGameRosterChanged`.

```
const stampStatusChanged = before?.status !== after.status;
const stampRetryPending = (after as {seasonStampRetry?: boolean}).seasonStampRetry === true;
if (after.groupId && (stampStatusChanged || stampRetryPending) &&
    !(after as {seasonId?: string}).seasonId &&
    (after.status === 'active' || after.status === 'finished')) {
  const gSnap = await groupOnce();
  const seasons = gSnap.data()?.seasons;
  if (seasons?.enabled && seasons.currentId) {
    await event.data!.after.ref.update({ seasonId: seasons.currentId, ...clearRetry });
    stampedSeasonId = seasons.currentId;
  }
}
```

When: on the transition into `active` (where it belongs) or into `finished` (the backstop), and on any later write while the `seasonStampRetry` marker is set. **Write-once from the server:** the `!after.seasonId` guard means the stamp is never corrected once present. **Not write-once from the client:** `firestore.rules` never mentions `seasonId` on `/games` — grep it, there are zero hits outside the `seasonCards / seasonSummary` document-id comments — so a member may create a game pre-stamped with any season string and the server will not overwrite it (LEDGER `games-seasonid-unpinned`, emulator-proven with four successful writes).

What an unstamped game means: season 1. This is one rule with two copies:

```
// src/utils/seasonScope.ts:28–34
export function inSeason(game: { seasonId?: string }, season?: SeasonScope): boolean {
  if (!season) return true; // a club with no seasons: everything is in scope
  const stamp = typeof game.seasonId === 'string' ? game.seasonId : '';
  return stamp ? stamp === season.currentId : season.currentNo === 1;
}
```

```
// functions/src/seasonCounters.ts:53-60
export function eveningInSeason(game: StampedEvening, seasonId: string, seasonNo?: number): boolean {
  const stamp = typeof game.seasonId === 'string' ? game.seasonId : '';
  return stamp ? stamp === seasonId : seasonNo === 1;
}
```

They disagreed by nineteen evenings until 18.09: the client admitted unstamped games into season 1, the server did not (LEDGER `unstamped-season-1-rule-disagrees-with-the-server-by-19`).

What an orphaned stamp means: nothing, and it is unrepairable. `games/DTNscolRojYDf0ZmT0sF` carries `seasonId: 's3'` on a club whose `seasons` block knows only `s2` and whose archive holds only `s1` . It got there because season 3 briefly existed and a reopen at 18.09 10:37:27Z rolled the club back to `s2` , deleting the `s3` archive and leaving the stamp. Today that evening belongs to no scope: it is excluded from `s2` by the stamp and from `s1` by having one. It cannot be repaired — the stamp trigger only writes when the field is absent, finished games are client-read-only, and no admin tool exists. And when `s2` eventually closes, `s3` is re-issued with the same id, at which point a night played on 17.09.2026 will be inside the brand-new season 3 from its first hour (LEDGER `seasonid-stamp-orphan-unrepairable` , `orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id`). It is currently harmless to the counters only because it is also `notHappened` .

Two further consequences of the write-once + transition-gated design. For the 191 clubs with seasons off, the stamp is never written, so `!after.seasonId` stays true forever and, before the transition gate was added, the group document was re-read on every write to an active or finished game (LEDGER `group-doc-read-three-times-per-sealed-evening`). And with the gate added, a game that loses its `groupOnce()` read on *both* transitions has no further transitions to retry on — hence the `seasonStampRetry` marker, which is itself defeated by the memoised-rejection bug in `groupOnce()` (ROUND 4 **22, 23**).

What is and is not reconciled

`tests/logic/seasonCounterReconciliation.test.ts` (228 lines, added 18.09) is the assertion that was missing — LEDGER `no-test-reconciles-the-three-counters` , which is now **partially** discharged. It runs `inSeason` , `eveningInSeason` , `countSeasonEvenings` , `countPlayedEvenings` and `completedRoundsFrom` over `tests/fixtures/realClub.ts` — a fixture built from this club's production shape (22 evenings that happened, 19 unstamped, one `unverified` , one `cancelled` , `eveningsSealed: 10` against `roundsAtStart: 7`) — and asserts all five return 22. It passes.

What it does **not** cover, and what the reviewer should treat as still unreconciled:

- The three *writers* are untested. The `increment(1)` at `index.ts:5194` and `:5228` are checked only by `tests/logic/sealedCounterAtomicity.test.ts` , which is a regular expression run over `index.ts` as a string.
- `gameService.getCommunityStats` itself cannot be loaded by a test — it pulls React Native in through the auth layer — so the *scan* is exercised only through its two extracted rules, never end to end.
- The 200/300-document bounds, the `-1` sentinel, and the failure paths of all three game queries are untested.

- `closeSeason` , `reopenSeason` , `runSeasonRollovers` , `performSeasonClose` and `closeSeasonIfRoundsTargetMet` have no functional test at all; their names appear in the suite only inside comments (LEDGER `nothing-tests-the-sweep`).
- No test asserts that the *frozen* copies (`seasonCards.completedRounds` , `seasonSummary.completedRounds`) equal what the live counters said at the moment of the close — which is the one comparison that would have caught both the 3-vs-22 archive bugs.

Finally, a naming hazard the reviewer will hit within an hour: `playedRounds` means **evenings** on `groups.seasons` and **mini-games** in `src/utls/eveningStats.ts:84,164` , where it is "how many mini-games the player actually took the field for". Same identifier, two units, one repository.

Cadence — how a season knows it is over

All line numbers are against HEAD, commit `e35394a`. The only club in production that has ever run a season is `HhzIwmjMl1i5HSOGHt3p` (מועדון שכחת שושי, "Shoshi's Forgetfulness FC"). A scan of all 195 `groups` documents on 19.09.2026 found **two** clubs with a `seasons` block at all, and **both use the rounds cadence**. No club has ever run a date season. Everything in the date half of this section is therefore verified by reading and by arithmetic, not by observing it work.

The finish line is one function, asked by two callers

Every decision about whether a season is over passes through `seasonFinishLine` and `isSeasonDue` in `functions/src/seasonCounters.ts:136` and `:175`. This centralisation is recent; before it the hourly sweep and the seal-time close each classified a cadence for themselves, which is how one path came to close a season at a number the other had never computed.

```
// functions/src/seasonCounters.ts:136
export function seasonFinishLine(cadence: {
  type?: string; endsOn?: unknown; endsAt?: number | null; targetRounds?: number | null;
}): SeasonFinishLine {
  if (cadence?.type === 'rounds') {
    const target = cadence.targetRounds;
    return typeof target === 'number' && target > 0
      ? { kind: 'rounds', target }
      : { kind: 'none', cadence: 'rounds' };
  }
  if (isCalendarDate(cadence?.endsOn)) return { kind: 'date', endsOn: cadence.endsOn };
  if (typeof cadence?.endsAt === 'number' && cadence.endsAt > 0)
    return { kind: 'epoch', endsAt: cadence.endsAt };
  return { kind: 'none', cadence: cadence?.type ?? '(none)' };
}
```

Four kinds, not two: `rounds`, `date` (a calendar day), `epoch` (the legacy millisecond deadline), and `none`. `isSeasonDue` then answers each — `played >= target`, `isSeasonOver(endsOn, today)`, `now >= endsAt`, and `false`.

ROUNDS cadence

`cadence.targetRounds` is the season's **total** number of `מהזורים` (evenings/game-nights), not a remainder. It is measured against `seasons.playedRounds`, a mirror on the club document that is seeded to 0 when a season opens and incremented by one every time an evening is sealed. The read path is `completedRoundsOf` (`functions/src/index.ts:15695`), which delegates to `completedRoundsFrom` (`seasonCounters.ts:105`):

```
export function completedRoundsFrom(sealedEvenings, roundsAtStart?, playedRounds?) {
  if (typeof playedRounds === 'number' && playedRounds >= 0) return playedRounds;
  const all = typeof sealedEvenings === 'number' && sealedEvenings > 0 ? sealedEvenings : 0;
  const base = typeof roundsAtStart === 'number' && roundsAtStart > 0 ? roundsAtStart : 0;
  return Math.max(0, all - base);
}
```

The fallback — `clubRecords/{groupId}.eveningsSealed - seasons.roundsAtStart` — is a subtraction of two counters from different eras. `eveningsSealed` only began accumulating on 26.08.2026. On the real club it currently reads **10**, while the club has played **22** evenings. Any caller that forgets to pass `playedRounds` therefore measures the season against a number twelve short of reality. The code comments record this having happened three separate times; `endSeasonNow` (`index.ts:17030`) carries a comment saying it would have sealed a season of three that the club had watched reach twenty-two.

Two paths notice that the target is met.

1. **On the seal.** `onGameRosterChanged` seals an evening, increments `playedRounds`, and then calls `closeSeasonIfRoundsTargetMet` (`index.ts:15943`) from `index.ts:5287`. It computes `playedNow = liveSeasons.playedRounds + 1` (the in-hand snapshot predates the batch it just committed), asks `seasonFinishLine`, returns immediately unless `line.kind === 'rounds'`, and closes if `played >= target` — subject to a 48-hour `REOPEN_GRACE_MS` immunity and to `clubIsQuiet`.
2. **The hourly sweep.** `runSeasonRollovers` (`index.ts:16055`), run last inside `cronEvery60Min` (`index.ts:13585`, `schedule: 'every 60 minutes'`). It pages 200 clubs at a time over `where('seasons.enabled', '==', true)`, asks the same two functions, and closes at most `MAX_CLOSES_PER_SWEEP = 5` seasons per run.

A date season can only ever be closed by path 2. A rounds season is closed by whichever gets there first; `performSeasonClose` re-reads the club document and abandons the close if `currentId` has moved (`index.ts:15823-15834`).

The clamp at activation. When seasons are first switched on and the admin chooses to carry season 1 (which holds the club's entire history) forward under a rounds cadence, the server raises the target so that "continue" cannot mean "close immediately":

```
// functions/src/index.ts:16530
if (cadence.type === 'rounds' && closedSoFar === 0) {
  cadence.targetRounds = Math.max(cadence.targetRounds ?? 0, played + 1);
}
```

`played` here comes from `completedRoundsOf(groupId, existing?.roundsAtStart)` at `index.ts:16328` — **two** arguments, so it takes the `eveningsSealed - roundsAtStart` fallback. But `planActivation`, which validated the same activation ten lines earlier and produced the sheet the admin approved, was given `playedEveningsFromGames(groupId)` — a scan of the games themselves. These are different numbers: 10 and 22 on the one real club (defect `rounds-clamp-uses-a-different-history`). The clamp can only raise, and the plan's own rule (`historyExceedsTarget`, `seasonActivation.ts:113`) already requires `target >`

gamesCount , so today the clamp is a silent no-op — unless eveningsSealed exceeds the games count, which happens whenever a finished game is deleted, because deletion decrements no counter. Then an admin who confirmed 24 gets eveningsSealed + 1 and is never told.

playedEveningsFromGames (index.ts:15655) has two further teeth: .limit(200) , so a club with more than 200 finished-or-cancelled games undercounts its own history, and a catch that returns 0. Zero is a claim, not an absence: it tells planActivation the club has never played, which disables every history refusal.

Production worked example. Season 1 of HhzIwmjMl1i5HSOGht3p was archived at seasonSummary/HhzIwmjMl1i5HSOGht3p__s1 with originalTarget.targetRounds = 22 and completedRounds = 22 — target met exactly. totals.rounds = 37 : that is משחקונים (mini-games), a different unit, and it is the number several client screens display beside the word מחזורים. The club is now in season 2 with roundsAtStart: 10 , playedRounds: 0 , cadence: {type:'rounds', targetRounds:24, months:null, endsAt:null, endsOn:null, startsOn:null} . completedRoundsFrom(_, 10, 0) returns 0, so the season is 0/24 and the fallback would agree (10 – 10 = 0). This is the only configuration in production where the two derivations happen to coincide.

DATE cadence: four fields for one deadline

Field	Type	Written by	Read by	Absent ⇒
mont hs	1–24 int	enableClubSeasons :16396, updateSeasonTarget :16763, rebaseCadence :15441	rebaseCadence (to compute the next season's length); SeasonsSettings seeds its chips from it	rebaseCadence guesses from endsAt – startedAt , then defaults to 6
ends On	'YYYY– MM–DD'	same three	seasonFinishLine first; SeasonsCard line + countdown	falls through to endsAt
ends At	epoch ms	same three, except the carry-on branch which nulls it (:16463)	seasonFinishLine only if endsOn is absent; SeasonsCard legacy branch	falls through to none
star tsOn	'YYYY– MM–DD'	same	nothing — one grep hit per write, zero reads	—

startsOn is write-only. endsAt is retained so that pre-1.1.7 clients keep rendering something. endsOn is the authority.

They do not agree. endsOn is computed as seasonEndDate(todayIn(), months) — the day *before* the anniversary. endsAt is computed as addMonthsClampedServer(now, months) — the anniversary instant itself. So endsAt lands on the **next season's first day** (endson–endsat–one–day–apart). Verified by executing both algorithms with TZ=UTC , as Cloud Functions run:

now	todayIn()	endsOn (3 months)	endsAt	endsAt as an Israel date
2026-09-19T07:00Z	2026-09-19	2026-12-18	2026-12-19T07:00Z	2026-12-19
2026-09-19T22:00Z	2026-09-20	2026-12-19	2026-12-19T22:00Z	2026-12-20

now	todayIn()	endsOn (3 months)	endsAt	endsAt as an Israel date
2026-08-31T07:00Z	2026-08-31	2026-11-29	2026-11-30T07:00Z	2026-11-30

The gap is one day in every case, but *which* day each field names depends on the hour the admin pressed the button, because `todayIn` uses Asia/Jerusalem and `addMonthsClampedServer` uses `new Date().getDate()`, which in Cloud Functions is UTC. An old client and a new client looking at the same season will disagree about its last day.

`src/utils/seasonDates.ts`, in full

166 lines, pure except one function. `functions/src/seasonDates.ts` is a byte-identical copy with a seven-line header (`diff` confirms: the header is the only difference), so every server line number in this file is the client's plus 7.

- `CLUB_TZ = 'Asia/Jerusalem' ( :21 )` — the club's calendar. Hard-coded; there is no per-club timezone anywhere in the schema.
- `CalendarDate` — a `'YYYY-MM-DD'` string. `isCalendarDate ( :28 )` is a regex test only: `'2026-13-45'` passes. The shape is chosen because lexicographic order *is* chronological order for it, which is what makes `compareDates ( :83 )` a two-character comparison.
- `daysInMonth(y, m) ( :43 )` — `new Date(Date.UTC(y, m, 0)).getUTCDate()`. Month is 1-indexed here and 0-indexed in `Date.UTC`, so passing `m` unchanged asks for "day 0 of month `m`", i.e. the last day of month `m-1` in 0-index = month `m` in 1-index. Correct, and leap years fall out for free.
- `addMonths(d, months) ( :55 )` — converts to an absolute month number, adds, converts back, and **clamps the day to the last valid day of the target month**. `31.08 + 6 = 28.02`, not `03.03`.
- `previousDay ( :64 ) / nextDay ( :73 )` — cross month and year boundaries with no special cases.
- `seasonEndDate(start, months) ( :94 )` — `previousDay(addMonths(start, months))`. The *inclusive last valid day*. This minus-one is why two consecutive seasons cannot both claim the anniversary.
- `nextSeasonStart(end) ( :102 )` — `nextDay(end)`.
- `todayIn(tz = CLUB_TZ, now = Date.now()) ( :113 )` — the only timezone-aware function. Uses `toLocaleDateString('en-CA')` because that locale formats as `YYYY-MM-DD`, with an `Intl.DateTimeFormat` fallback for runtimes that ignore the locale's ordering.
- `isSeasonOver(end, today) ( :138 )` — `compareDates(today, end) > 0`. Strictly greater: the season is valid **through** its last day.
- `monthsBetween ( :143 )` — whole months, used only to describe a legacy season's length.
- `MIN_SEASON_MONTHS = 1, MAX_SEASON_MONTHS = 24 ( :156-157 )`, enforced by `isValidSeasonMonths`.

Worked example: a 31st-of-the-month start, and the anniversary ratchet. Six-month seasons from 31.08.2026, chaining `seasonEndDate` → `nextSeasonStart`:

Season	Runs	Boundary day-of-month
1	2026-08-31 → 2027-02-27	31
2	2027-02-28 → 2027-08-27	28

Season	Runs	Boundary day-of-month
3	2027-08-28 → 2028-02-27	28
4	2028-02-28 → 2028-08-27	28
5	2028-08-28 → 2029-02-27	28

`addMonths` clamps *downwards only*, and `nextSeasonStart` then feeds the clamped day back in as the next season's start. The day-of-month is therefore **monotonically non-increasing over the club's life**: once a season boundary passes through February it can never return to the 31st. A club on one-month seasons from 31.01.2027 walks 31 → 28 in a single step and stays there: `2027-01-31-02-27` , `02-28-03-27` , `03-28-04-27` , ... The club permanently loses three days of the calendar month it chose. This is a consequence of the design, not a bug in it, but nothing in the UI says it will happen and no season summary records the drift.

Three different month arithmetics

Function	Where	Operates on	Timezone	Live?
<code>addMonths</code>	<code>seasonDates.ts:55</code>	calendar dates	none	yes
<code>addMonthsClampedServer</code>	<code>index.ts:15733</code>	epoch ms	the process's local zone = UTC	yes, writes <code>endsAt</code>
<code>addMonthsClamped</code>	<code>seasonLifecycle.ts:169</code>	epoch ms	the device's local zone	dead

`src/utils/seasonLifecycle.ts` — 177 lines containing a second `isSeasonDue` , `canCloseNow` , `roundsRemaining` , `validateTargetChange` , `continueSeasonTarget` and `addMonthsClamped` — is imported by exactly one file in the repository: `tests/logic/seasonLifecycle.test.ts` , which runs 22 tests against it. No screen, no service and no Cloud Function imports it. Its header claims it exists so that "the client (which greys out a button and explains why) and the server (which is the one that actually decides)" cannot disagree; in fact the client's copy of `isSeasonDue` never consults `endsOn` at all, so if it *were* wired up it would disagree with the server on every date season opened since calendar boundaries shipped. Similarly, `functions/src/index.ts:15729` declares `const MONTH_CHOICES = [1, 3, 6, 12]` under a nine-line comment about the harm it once did; `grep` finds no use of it.

`rebaseCadence` — the finish line moves at every rollover

`functions/src/index.ts:15405` . A **rounds** target survives a rollover untouched: "24" means "another 24 evenings" for every season. A **date** target cannot, because an inherited end date is already in the past and would make the new season due the instant it opened — one empty archived season per hour, for ever.

```
function rebaseCadence(cadence, seasonStartedAt, now) {
  if (!cadence || cadence.type !== 'date') {
    return (cadence ?? { type: 'date', months: 6 }) as { ... };
  }
  let months = isValidSeasonMonths(Number(cadence.months)) ? Number(cadence.months) : 0;
  if (!months && seasonStartedAt > 0 && typeof cadence.endsAt === 'number') {
    const ranMonths = Math.round((cadence.endsAt - seasonStartedAt) / (30*24*60*60*1000));
    months = isValidSeasonMonths(ranMonths) ? ranMonths : 0;
  }
  if (!months) months = 6;
  const startsOn = todayIn(undefined, now);
  return { type: 'date', months, endsAt: addMonthsClampedServer(now, months),
    startsOn, endsOn: seasonEndDate(startsOn, months), targetRounds: null };
}
```

Caller	Line	What it is rebasing
performSeasonClose	15886	the successor of a season the sweep or the seal just closed
enableClubSeasons , seal-now branch	16505	season $N+1$, measured from the activation moment
reopenLastSeason	16957	the season being brought back
endSeasonNow	17061	the successor of a manually-ended season

Two problems live here.

A missing cadence produces a season that can never end. The first branch returns `{type: 'date', months: 6}` with **no** `endsOn` **and no** `endsAt`. Fed to `seasonFinishLine`, that is `{kind: 'none', cadence: 'date'}` — and `isSeasonDue` returns `false` for `none`, for ever. Any close of a season whose club document lacks a `cadence` map opens a successor that no path will ever close. This is `cadence-with-no-finish-line-never-closes`. The sweep at least now *says so* (`index.ts:16141-16152`, `noFinishLine += 1` plus a `console.warn`) instead of the bare `continue` it used to be — but it is a log line, not an error row, and the production `errors` collection has never held a seasons entry.

Reopen gives a date season a whole new life. `reopenLastSeason` is the undo button for an accidental close. It rebases from `archive.originalTarget` — so a six-month season that was ended three days early comes back running **six months from today**, not three days (`reopen-extends-a-date-season-by-a-full-length`). `rebaseCadence` is the function for opening the *next* season; the reopen needs the original `endsOn` restored, and there is no code that does that.

The four end-date options

On a **first** activation with a date cadence and "carry season 1 on", the admin must name season 1's last day, because season 1 contains the club's entire history and has no computable start. `SeasonsSettings.tsx:67` defines the shortcuts and `:831-845` renders them as dates:

```
const SEASON1_MONTH_CHOICES = [1, 2, 3, 6] as const;
...
{SEASON1_MONTH_CHOICES.map((m) => {
  const d = seasonEndDate(today, m);
  return <Chip key={m} label={formatCalendarDate(d)} active={!season1Custom && season1EndsOn === (
    onPress={() => { setSeason1Custom(false); setSeason1Months(m); setSeason1EndsOn(d); }}
  )}}
  )}}
```

They are deliberately shorter than the season-length chips (`MONTH_CHOICES = [3, 6, 12]` , `:62`) because this season already holds everything played so far. A fifth chip, "מותאם אישית" ("custom"), opens a 1–24 stepper that also writes a date rather than a number — the comment at `:867` records that it previously moved a number while the date the admin was actually choosing never changed.

Verified against `today = 2026-09-19` : the chips render 18.10.2026, 18.11.2026, 18.12.2026 and 18.03.2027, and `nextSeasonStart` of each is the 19th. Correct, and consistent with `planActivation`'s `nextStartsOn` . `planActivation` (`seasonActivation.ts:174`) then refuses `end <= today` — so the same-day option, if a custom stepper could reach it, would be rejected with `season1EndNotFuture` → "חייב להיות N תאריך הסיום של עונה" ("season N's end date must be in the future").

Three ranges are in play for one concept and they do not match: the client offers 3/6/12, the client's custom stepper allows 1–24, the server accepts 1–24 (`isValidSeasonMonths`), and the dead server constant says 1/3/6/12. The server's date branch also still silently rewrites anything outside 1–24 — including a missing `months` field — to 6, directly under a comment declaring that "rewriting silently is not [honest], and 6 was neither what they picked nor what they confirmed". The class of input was narrowed, not eliminated.

updateSeasonTarget — moving the line mid-season

`functions/src/index.ts:16627` . The client calls it from `SeasonsSettings.saveTarget` (`:513`) with `{groupId, cadenceType, months?, targetRounds?}` and nothing else.

What an admin can change: the target number, and the cadence *type* itself — rounds↔date, mid-season, with no extra guard. A club twelve evenings into a 24-evening season can switch to a three-month date cadence; `playedRounds` stays at 12 but stops meaning anything.

What is refused:

Condition	Code	Message the admin sees
not an admin	<code>permission-denied</code>	"הפעולה מיועדת למנהלי המועדון"
<code>seasons.enabled !== true</code>	<code>failed-precondition</code> <code>seasons are off</code>	mapped
<code>rounds</code> , non-finite or ≤ 0	<code>invalid-argument</code> <code>targetRounds required</code>	mapped → <code>targetInvalid</code>

Condition	Code	Message the admin sees
rounds < MIN_SEASON_ROUND S (2), :16723	invalid-argument target N is below the 2-round floor	unmapped — refusalOf (seasonService.ts:145) only matches invalid-argument messages containing the literal targetRounds , so this reaches the admin as the generic "משהו השתבש"
rounds <= played , :16730	failed-precondition target N is not above the M rounds already played	mapped → targetBehind , and playedFromError recovers M
rounds <= played and an evening in play :16686	failed-precondition openGame / unsealedGame	mapped

Nothing is refused on the date side. months is at least 1 by the time it is used, so addMonthsClampedServer(now, months) is always in the future: **a date re-target can only ever extend a running season**, never shorten it below one month from today, and never close it.

What targetHistory records. One arrayUnion entry per accepted change (:16802): at , by , byName (frozen), from (the whole previous cadence map, or null), to (the whole new one), playedAtMove (sealed because played is a moving number), and endsTheSeason .

What it does not record. No seasonId — which is exactly what let a reopen reattach one season's history to another (target-history-entry-carries-no-season-identity). Not the initial target: the first entry's from is the only trace of it. Not the rewrites rebaseCadence performs at every rollover and at every reopen, which move a date club's finish line with no entry at all. And nothing shows it: targetHistory is typed (src/types/index.ts:1004), deserialized (src/firebase/firestore.ts:691) and unit-tested (tests/logic/groupSeasonsReader.test.ts:50), and rendered by no screen — there is no Hebrew string for it (target-history-never-shown-to-anyone). The client's SeasonTargetChange type also describes only {type, endsAt, targetRounds} for from/to, so months , endsOn , startsOn , playedAtMove and endsTheSeason are invisible to TypeScript even though they are in the document.

target-history-erased-at-close . performSeasonClose (:15911) and endSeasonNow (:17062) both write targetHistory: [] for the successor season. The array is now sealed into the archive first (:15838 , :17042), but the one archive that exists in production — seasonSummary/HhzIwmjMl1i5HS0GHt3p__s1 , whose full key set is awards, closedAt, completedRounds, endsAt, groupId, groupName, no, originalTarget, pairs, players, roundsAtStartOfSeason, seasonId, startsAt, totals, zeroedAt — **has no targetHistory key**. It was closed before the sealing existed. That club's finish line moved from 22 to 2 to 24 and only the last move survives anywhere.

target-change-can-close-a-season-with-no-confirmation is now half fixed, and the fix made a second mechanism unreachable. The asked <= played throw at :16730 means the endsTheSeason computation twenty lines later —

```
const endsTheSeason = next.type === 'rounds'
  && typeof next.targetRounds === 'number' && next.targetRounds <= played;
```

— **can never be true**, because that call has already thrown. Consequently `seasons.targetMovedToClose` is written by nothing (grep: one gated write at `:16816`, three unconditional deletes, one read), and the `endedEarly / closedBy` branch in `performSeasonClose` at `:15845–15864` that exists to read it back is dead. The residual hole is `played + 1`: it is accepted, it is not flagged, and the very next seal closes the season — from a plain one-tap button, because `SeasonsSettings` routes a live club straight to the server (`:953`: `onPress={live ? saveTarget : () => setConfirmOpen(true)}`) while the functionally identical "סיים עונה" ("end the season now") gets an `appAlert` confirmation. The code comment at `:16787` argues `played + 1` is a season "reaching its finish line", which is defensible for the flag and not for the missing confirmation.

`date-retarget-restarts-the-clock`. The date branch writes `startsOn: todayIn(now)` and recomputes `endsOn` from today, while `seasons.startedAt` keeps the season's real start. The same season then has two start dates: `SeasonsCard`'s progress bar uses `seasons.startedAt` (`SeasonsCard.tsx:123`) and the settings preview labels today "תחילת העונה" ("start of the season", `he.ts:2310`). `months` means "length" when a season is enabled and "time remaining from now" when it is re-targeted, with no word anywhere marking the difference.

`date-season-cannot-be-renewed-at-the-same-length`. The save button is gated on `targetChanged` (`SeasonsSettings.tsx:401`):

```
if (cadence === 'rounds') return c?.type !== 'rounds' || c.targetRounds !== rounds;
return c?.type !== 'date' || c.months !== months;
```

For a date club the comparison is on `months` alone — the very field the server does *not* treat as an identity. An admin who wants "another three months from today", which is precisely the move the 48-hour reopen grace exists to allow, finds the button disabled and the line "זה היעד שמוגדר כרגע במועדון" ("this is the club's current target"). The one correct action is the one the UI calls a no-op.

Finally, `SeasonsSettings` seeds `history` for a live club from `seasons.playedRounds` (`:336`), which is the same figure the server measures against — so `settings-screen-validates-a-live-target-against-zero` is fixed at HEAD. Per ROUND4 finding 1, neither that fix nor any other client fix from 18.09 is in the shipped store binaries, which were built from `4de80d0`.

Closing a season

Everything in this section is quoted from HEAD = e35394a (2026-09-18 21:40 +0300). Note before anything else: the deployed Cloud Functions were built from the source as it stood at 16:04Z on 2026-09-18, and three commits to these files landed after that (bc6a562 17:03, 5531a21 19:02, 8235851 21:14, all +0300). Where HEAD and production differ I say so; a reader diffing the running system against this document will otherwise be misled.

Three doors into one room

A season can be closed by exactly three pieces of code. All three end in the same function, closeSeason in functions/src/seasonRollover.ts:307 , and two of the three first pass through performSeasonClose (functions/src/index.ts:15790), which wraps the close in a pre-flight check and a lifecycle write.

	closeSeasonIfRoundsTargetMet	runSeasonRollovers	endSeasonNow
where	index.ts:15943	index.ts:16055	index.ts:16971
fires on	the seal of an evening, from onGameRosterChanged (call site index.ts:5274–5292)	cronEvery60Min , last in the job (index.ts:13604)	an admin tapping סיים עונה עכשיו ("end season now")
cadences it can finish	rounds only (line.kind !== 'rounds' → return, :15967)	all of them	ignores the cadence entirely
quiet check	clubIsQuiet(groupId, {mode: 'afterSeal', exceptGameId})	clubIsQuiet(doc.id) (sweep mode)	clubIsQuiet(groupId) (sweep mode)
on "busy"	logs, returns; the sweep is the backstop	stamps seasons.dueBlockedSince , retries next hour	throws failed-precondition to the client
goes via performSeasonClose	yes	yes	no — it calls closeSeason directly and writes its own lifecycle block
extra archive fields	endedEarly / closedBy only if targetMovedToClose is set	same	always endedEarly:true , closedBy , closedByName
completedRounds	known.played = mirror +1	completedRoundsOf(id , roundsAtStart, playedRounds)	completedRoundsOf(id, roundsAtStart, playedRounds)

The last row is the one that used to differ, and it is worth being exact about, because whatever it produces is written into a document created with create() and never recomputed.

`completedRounds` : the argument the three paths disagreed about

There are four numbers in this system that all answer "how many `מחזורים` (evenings) has this season held", and they are not equal:

```
// functions/src/seasonCounters.ts:105
export function completedRoundsFrom(
  sealedEvenings: number,
  roundsAtStart?: number,
  playedRounds?: number,
): number {
  if (typeof playedRounds === 'number' && playedRounds >= 0) return playedRounds;
  const all = typeof sealedEvenings === 'number' && sealedEvenings > 0 ? sealedEvenings : 0;
  const base = typeof roundsAtStart === 'number' && roundsAtStart > 0 ? roundsAtStart : 0;
  return Math.max(0, all - base);
}
```

`playedRounds` is the **mirror**: seeded when the season opens (`functions/src/seasonSeed.ts:26`) from `playedEveningsFromGames` — a `didEveningHappen` count over the club's last 200 terminal games — and incremented by one on every seal. `roundsAtStart` is `clubRecords.eveningsSealed` at the moment the season opened. The fallback branch, `eveningsSealed - roundsAtStart` , subtracts two counters from different eras: `eveningsSealed` only began being written on 26.08.2026, and the club's games go back to June.

On `HhzIwmjMl1i5HS0GHt3p` (מועדון שכחת שושי, the only club that has ever run a season) at the moment season 1 closed — 2026-09-17 16:12:06Z — the live values were `roundsAtStart: 7` , `clubRecords.eveningsSealed: 10` , `seasons.playedRounds: 21` (the seal that triggered the close had just made it 22). So:

path	expression	value
seal	<code>liveSeasons.playedRounds + 1 (index.ts:5282-5285)</code>	22
sweep	<code>completedRoundsFrom(_, 7, 22)</code>	22
<code>endSeasonNow</code> , as deployed before 18.09	<code>completedRoundsFrom(10, 7, undefined)</code>	3

That third row is finding **endseasonnow-archives-wrong-length** (P0, confirmed twice). The callable simply omitted the third argument; `seasons.playedRounds` was not even in its local type literal. Had the admin pressed the button instead of playing the twenty-second evening, the club would hold a permanent archive saying season 1 lasted three evenings. It is fixed at `index.ts:17030-17034` and the comment there records the counterfactual. The fix is deployed.

The sealed archive proves the surviving path was correct: `seasonSummary/HhzIwmjMl1i5HS0GHt3p__s1` carries `completedRounds: 22` .

closeSeason , in order

WHAT IT READS

```
// seasonRollover.ts:310
const [psSnap, csSnap, pairSnap] = await Promise.all([
  db.collection('communityPlayerStats').where('groupId', '=', groupId).get(),
  db.collection('communityStats').doc(groupId).get(),
  db.collection('communityPairStats').where('groupId', '=', groupId).get(),
]);
```

Three reads, then a fourth: one `getAll` per 300 members over `/users`, because `communityPlayerStats` carries no name and the archive must still render after an account is deleted (:336–348). The pair query is the expensive one — see the guest-pair discussion below.

THE PLAYERS MAP

```
// seasonRollover.ts:366
if (num(x.rounds) === 0 && num(x.games) === 0) continue;
```

A row with no mini-games **and** no evenings is not a participant. This matters more than it looks: nothing ever deletes a `communityPlayerStats` row, and the close winds rows back to zero rather than deleting them, so after one close every member of the club is sitting at zero. Without this test every subsequent season's archive would carry every ex-member forever. It also means a close run over a table that a previous close already emptied archives `players: {}` — which is exactly what happened on this club four hours later; see the worked example.

Two fields are deliberately allowed to be **absent** rather than zero (:377–379): `csRounds` and `asRounds`, the coverage denominators for clean-sheet and assist rates. They arrived later than the metrics they divide, and writing `0` would say "measured over zero rounds", which readers cannot tell from a real zero.

The field list is `PLAYER_SEASON_FIELDS` (:184–211) — 19 counters including `eveningScoreSum` and `eveningScoreCount`, which must reset together or the MVP average stops being an average. Everything not on that list survives the close: `bestEvening` (a personal high-water mark), `lastEveningScore`, and `kingGoalsSum / kingGoalsCount` (the benchmark the evening score is graded against). The header comment at :18–24 explains why a blanket overwrite here would silently inflate every future evening score by about a quarter of the scale.

THE TOTALS

`CLUB_SEASON_FIELDS` (:234–242) is copied off `communityStats`; `assists` and `cleanSheets` have no club counter at all, so they are summed from the member rows (:392–399) — otherwise they become underivable the moment the rows are zeroed.

THE PAIRS, AND THE CAP

Three filters, in order (:417–479):

```

if (!isReal(a) || !isReal(b)) { droppedGuestPairs += 1; continue; } // :436
const playedTogether = num(x.sameTeam) + num(x.against) + num(x.assists)
  + num(x.assistsAToB) + num(x.assistsBToA);
if (playedTogether === 0) { droppedEmptyPairs += 1; continue; } // :450
if (Object.keys(pairs).length >= MAX_ARCHIVED_PAIRS) { droppedOverflowPairs += 1; continue; }

```

`isReal` is `!id.startsWith('guest:')` (:31). `MAX_ARCHIVED_PAIRS` is 1,200 (:181), sized so the archive cannot exceed Firestore's 1 MB document limit and make the club *permanently unable to close a season* — the failure mode is unrecoverable, because a season summary is written with `create()`.

Guests are the reason the cap exists. A guest identity is minted fresh for every game, so `communityPairStats` gains a permanent new document for every stranger who ever turns out. Production, counted by paging the whole collection today: **314 pair documents for a seven-player club, 293 of them guest pairs (93%), 21 real**. The collection holds 717 documents across the entire database, so this one club is 44% of it. Finding **guest-pairs-make-every-close-read-fifteen-times-what-it-archives** (P2) is exactly this: the close reads 314 documents to archive at most 21, the sweep runs last inside `cronEvery60Min`'s shared 540-second budget, and at roughly 45 pair documents per player a sixty-player club is about 2,700 reads per close. HEAD mitigates the *growth* by deleting guest pair rows during the wipe (:944–949) — but that code is not yet deployed, and the 293 documents are still there.

The surviving pair is stored under a sorted key, with `a / b` and the four directional counters flipped to match (`seasonPairKey` , :104–110). One key, derived one way, used by both the wipe and the restore: if they disagreed a reopen would give a pair chemistry it never had.

THE ARCHIVE WRITE, AND THE `CREATE()` LATCH

```

// seasonRollover.ts:580
const summaryRef = db.collection('seasonSummary').doc(`${groupId}__${seasonId}`);
try {
  await summaryRef.create({ /* ... totals, players, pairs, awards ... */ });
} catch (err) {
  const code = (err as { code?: number | string }).code;
  if (code !== 6 && code !== 'already-exists') throw err;
  const existing = await summaryRef.get();
  if (existing.get('zeroedAt')) { // :667
    console.log('[season] already closed – skip', groupId, seasonId);
    return { archived: false, players: 0, pairs: 0 };
  }
  resuming = true; // :676
  resumedAwards = existing.get('awards');
  resumedPlayers = existing.get('players');
  resumedTotals = existing.get('totals');
  resumedEvenings = existing.get('completedRounds');
}

```

That single `create()` is the whole idempotency story, and the ordering — build in memory, archive, *then* wipe — is what stops a retry from sealing a season of zeros over the real one.

`zeroedAt` (written last, `:1024`) separates the two meanings of `ALREADY_EXISTS`. Present: the first pass finished, the live rows now belong to the next season, stop. Absent: the first pass died between the archive landing and the wipe, and the club is half-closed — the season sealed, the titles never written, the table never reset. On the resume path every derived value is read back off the archive, not recomputed, because the live rows may be half-wiped.

[defect: resume-subtracts-recomputed-row, P2] — with one exception, and it is the whole of the wind-back. `resumedPlayers` is used for the card (`:744`) and `resumedAwards` for the card and the titles (`:717`, `:832`), but the subtraction at `:879` and `:903` uses `players` — the map recomputed from the live rows at the top of *this* invocation:

```
// seasonRollover.ts:878
for (const d of psSnap.docs) {
  const archivedRow = players[(d.data() as { userId?: string }).userId ?? ''];
  if (!archivedRow) continue;
  ...
  Object.assign(patch, windBackRow(data, archivedRow, PLAYER_SEASON_FIELDS));
}
```

On a resume, an evening played between the crash and the retry is inside `players` but not inside the sealed archive. Subtracting the recomputed row therefore removes that evening from the live table while the archive never recorded it: it exists nowhere. This is the precise failure the module header at `:46–61` says subtraction exists to prevent, on the one path where the two rows differ.

THE CARD

`seasonCards/{groupId}__{seasonId}` (`:755–809`) is a compact projection for the hall of fame — date, three totals, a participant count and the resolved winner names — because the archive is large and the list screen would otherwise pull every season in full. Two details:

- `countSeasonParticipants` (`functions/src/seasonParticipants.ts:23`) counts `games > 0 || rounds > 0`. It shipped counting `rounds` alone, which is **משחקונים** (mini-games), which only exist in advanced mode — so every season a timer-only club closed reported "0 שחקנים" ("0 players") no matter how many people turned up.
- `sealedCardEvenings` (`:155–165`) prefers the archived `completedRounds`, then the caller's, then the awards denominator. The card prints this as "מחזורים N". It is written with `merge:true`, so a resume converges rather than duplicating.

The field `awardsDenominator` lived on both the archive and the card for one day and was removed in `8235851`. `awardsDenominatorOf` (`:133–139`) is `max(players.games)`, and `mostLoyal` is the maximum of the same array, so `winner.value / awardsDenominator` is 1.0 by construction — the medal tier it fed was platinum for every season of every club, forever. Production proves the identity: s1's archived `games` are `{19,18,18,18,18,18,17}` and `awards.mostLoyal.value` is 19. The number survives only as the awards *eligibility gate's* denominator (`:556–559`), never as a published share.

THE TITLES

:823–860 writes one document per winner per title at `users/{uid}/seasonTitles/{groupId}__{seasonId}__{titleKey}`, with `groupName` frozen, batched 400 at a time, outside the `create()` latch on purpose. A pair title's key is split on `__` so both halves get a copy; `guest:` ids are skipped. `if (!award) continue` — a null award is a result, not a gap. Production holds 14 such documents for s1: eight titles, of which `mvp` has seven winners and `deadlyDuo` is null.

THE WIND-BACK

The invariant, stated at :46–61 and implemented in `windBackRow` (:62–74):

```
out[f] = archived ? Math.max(0, rowNum(current?.[f]) - rowNum(archived[f])) : 0;
```

Subtract what was archived; do not write zeroes. The rows were read at the top and are wiped at the bottom; anything committed in between is not in the archive, and an absolute zero would erase it from the live table too, leaving it nowhere. `archived === null` means "this row belongs to no season" — a pair past the cap — and is zeroed. Subtraction is not idempotent, so each row gets `seasonWoundBack: seasonId` and `seasonReopened` is deleted so a later reopen can stamp its own restore. Player rows go one transaction each (:881); pair rows go in 400-op batches (:980).

The skip guards read `if (resuming && row.seasonWoundBack === seasonId)` (:886 for players, :973 for pairs). The `resuming && half` was added in 8235851 and is **not deployed**. Without it, a stamp naming the season being closed was treated as proof that season had already been wound back — but a *reopen* leaves that stamp on every row the close touched and the restore did not reach. Finding **stale-pair-woundback-stamp-skips-next-wipe** (P1) is live right now: all 314 pair documents on this club carry `seasonWoundBack: "s2"` while **s2 is the running season**. Against the deployed binary, closing s2 archives those 21 real pairs' chemistry and leaves it standing live for s3 to inherit, double-counted, with the archive write-once. Against HEAD it is correct. The reopen now also sweeps stale stamps (:1174–1198), but it ran five and a half hours before that code deployed.

[defect: club-totals-zeroed-not-subtracted, P1] The club document breaks the rule the module is built on:

```
// seasonRollover.ts:1004
const zeroClub: Record<string, number> = {};
for (const f of CLUB_SEASON_FIELDS) zeroClub[f] = 0;
await db.collection('communityStats').doc(groupId).set({ ...zeroClub, ... }, { merge: true });
```

Absolute zero, not `windBackRow`. Player rows and pair rows subtract; the seven club counters (`rounds`, `goals`, `guestGoals`, `ownGoals`, `tiedRounds`, `shootoutRounds`, `scorelessRounds`) do not. An evening committed between the read at :310 and this write is preserved on every player row and destroyed on the club total, so the club's goals stop equalling the sum of its members' goals with nothing anywhere recording the discrepancy. `chemistrySince` is re-stamped to `now` in the same write, which is correct — otherwise the new season's chemistry card would date itself from the old one.

Season 1 of מועדון שכת שושי, reconciled against the games

seasonSummary/HhzIwmjMl1i5HS0GHt3p__s1 , read live:

field	value	reconciles to
startsAt	1789547794676 = 2026-09-16 08:36:34Z	the moment seasons were switched on, not the club's first game (June). The hall of fame therefore prints "2026 ספט' – 2026 ספט'" above "22 מחזורים" — finding season1-startsat-is-the-enable-moment
endsAt = closedAt	1789661526768 = 2026-09-17 16:12:06Z	the seal of TZ7IWEJWVww2itvIKCd8
zeroedAt	1789661533425	6.7 s later — the whole close took under seven seconds for 7 players and 314 pair rows
complete dRounds	22	19 unstamped legacy evenings + 3 stamped s1 (yA48XsyC , nRHEgxVt , TZ7IWEJW); a 23rd finished game, DTNsco1R , is stamped s3 and carries playVerified:false
roundsAt StartOfSeason	7	clubRecords.eveningsSealed on 16.09. Ten roundSummaries documents exist for this club; seven were already written when seasons were switched on, and three (yA48XsyC , nRHEgxVt , TZ7IWEJW) were sealed during s1
original Target	{type: 'rounds', targetRounds: 22}	the cadence at close time; the club's current cadence is 24, moved on 18.09
totals. rounds	37	משחקונים, mini-games — not evenings. The same document says 22 in completedRounds . Both numbers are correct and they measure different things
totals. goals / guestGoals	27 / 4	
totals. assists	13	= 1+5+1+2+3+0+1, summed from the seven member rows at :392
totals. cleanSheets	75	= 12+11+13+8+10+11+10, summed at :396
players	7 rows	
pairs	21	exactly C(7,2) — every member pair played together at least once; all 293 guest pairs dropped
awards	8 of 9 decided	deadlyDuo: null

The seven archived games values are {19, 18, 18, 18, 18, 18, 17} . Counted directly from the 22 game documents' players arrays, actual attendance is {22, 20, 20, 20, 20, 20, 19} . The gap is finding archive-games-undercounts-every-player (P1) and it resolves exactly: three evenings — 2SMr1CGH (06.07),

MMtE8J6H (16.07), FCa5UtSd (04.08) — never left `liveMatch.phase: "organizing"`, have `rotation: null`, and credited nobody a `games` increment. `didEveningHappen` counts them (they are legacy closes with no `endedBy`, so `isLegacyClose` (`functions/src/eveningPlayed.ts:196`, `endedBy` neither `'admin'` nor `'auto'`) calls them 'happened'), the per-player counter does not. לוי מתן attended all three ($22 - 3 = 19$); the other five regulars attended two each ($20 - 2 = 18$); אִיִצִיק לוי attended two ($19 - 2 = 17$). **The archive's own season length and its own per-player attendance are counted by two different rules that disagree by three on this club** — which is precisely why `awardsDenominatorOf` exists, and why the eligibility gate is set from `max(games) = 19` rather than from `completedRounds = 22`.

The pair archive reconciles too: summed over the 21 entries, `sameTeam` 94, `against` 140, `winsTogether` 46, `lossesTogether` 35, `cleanSheetsTogether` 50, `winsA` 62, `winsB` 52, directional assists $4 + 3 = 7$ against a legacy undirected `assists` of 8. `deadlyDuo` is null because the best pair score is 2 and the floor is `MIN_DUO_ASSISTS - 1 = 2` with strict `>` (`seasonAwards.ts:152`, :159–166).

`mvp` is awarded to **all seven members at exactly 6.0**. Every player's `eveningScoreSum` / `eveningScoreCount` is 18/3 or 12/2. 6.0 is the bottom of the evening-score scale and also what the score returns for a player who took the field for no mini-games. `MVP_SCALE_FLOOR = 6` with a strict `>` now blocks this (`seasonAwards.ts:156`, :236) — but s1's archive is write-once and seven junk שחקן העונה ("player of the season") title documents sit on seven profiles permanently. Finding **mvp-title-crowns-the-entire-club** (P1).

AND THE CLOSE THAT ARCHIVED NOTHING

Four hours after s1 closed, at 21:12:08Z, a second close ran on the same club. Its archive has since been deleted by a reopen, but the state it left behind is unambiguous and is still on disk:

- All seven `communityPlayerStats` rows read `{games:0, rounds:0, goals:0, ...}` and still carry `seasonWoundBack: "s1"` — **not** `"s2"`. They were already empty, so the `:366` filter excluded every one of them from `players`, so `players` was `{}`, so the wind-back loop's `if (!archivedRow) continue` (`:880`) skipped every row and wrote no stamp.
- All 314 pair rows read zero and carry `seasonWoundBack: "s2"` — the wipe reached them (guest deletion was not yet deployed, so they were zeroed rather than removed). The production log the audit captured for this close reads kept 0, dropped 293 guest, 21 empty.
- `communityStats` is all zeros with `seasonReopened: "s2"`.

So the close archived an empty `players` map, an empty `pairs` map, zero totals, and — because `endSeasonNow` still passed two arguments, giving `completedRounds = 10 - 10 = 0`, and `computeSeasonAwards` returns all-null for `completedRounds <= 0` (`seasonAwards.ts:199–205`) — nine null awards. It then wrote a `seasonCards` document reading "שחקנים", pushed nobody (`announceSeasonClosed` at `index.ts:15342–15351` iterates the empty `players` map), and advanced the club to season 3. `games/DTNscolRojYDf0ZmT0sF`, kicked off seven minutes later, still carries the orphan stamp `seasonId: "s3"` on a club whose `seasons` block now knows only s2.

This is finding **perform-close-writes-next-season-even-when-nothing-was-archived** (P1) in its observable form: *nothing in either close path is conditional on the archive containing anything.*

`clubIsQuiet`

`index.ts:15466-15571`. The one thing a close must not do is split an evening: each mini-game commits separately, so mini-games 1-3 would land in the old season and 4-6 in the new, while the player's career total — written in the same batch — keeps the whole. Two checks.

Check one: is a game in play. A `games` query filtered by status and bounded on both sides:

```
// index.ts:15495
.where('status', 'in', afterSeal ? ['active'] : ['scheduled','open','locked','active'])
.where('startsAt', '<=', now + STARTING_SOON_MS) // +3h
.where('startsAt', '>=', now - TONIGHT_MS) // -12h
.limit(2)
```

The upper bound is `now + 3h` because a game can be started before its scheduled kickoff. The lower bound exists because a game left open days ago is stale and must not hold a club's season hostage forever. The status list narrows to `['active']` on the `afterSeal` path: the evening the close would split just ended, so only a genuinely-in-play second game can block, not next week's recurring clone — and *nearly every real club runs a recurring fixture*, which is why the unbounded earlier version meant no such club could ever close a season on any path.

Check two: is an evening finished but not yet sealed. The three newest `finished` games; for each, if it has no `roundSummaries/{gameId}` document, block. Closing in that gap makes `sealRoundSummary` compare tonight against a table with no history, so every stat reads as a brand-new club record — and the summary is written once, so the wrong story is permanent.

Three floors let a game out of check two (`:15542` , `:15557` , `:15564`): the evening did not happen per `didEveningHappen` ; it is a **legacy close** (`endedBy` is neither `'admin'` nor `'auto'`); or it started more than `STALE_SEAL_MS` (24 h) ago.

[defect: quiet-legacy-evening-blocks-club-forever, P1-really-P0] The legacy floor at `:15557` is the fix; before it, that clause did not exist. `roundSummaries` began on 26.08.2026, so every evening finished before that date carries no `endedBy` , is called 'happened' by `isLegacyClose` , and will never be sealed by anything. "Finished but not yet sealed" was therefore *permanently* true for them. An auditor ported both functions and ran them over all 91 finished games in production: **24 of the 30 clubs that have ever finished a game returned `{ok:false, blocker:'unsealedGame'}` forever.** Those clubs could not enable seasons with "seal now", could not end a season by button or by sweep, and could not undo a close — while the Hebrew told the admin to wait for something that was never going to happen. The window only advances when the club plays a *new* evening, and those games are terminal. This is why exactly one club in a 195-club database has ever run a season. The fix is deployed; the historical distortion of every adoption number is not undone by it.

One more note on scope: `clubIsQuiet` is a per-club gate on *closing*, not a lock. Nothing prevents a mini-game from committing during the seven seconds the close takes; that is what the subtraction semantics exist for, and what the club-totals zero throws away.

`performSeasonClose` : the rewrite of the `seasons` block

Two things happen around `closeSeason` here.

Pre-flight (`index.ts:15823–15834`). Both callers arrive holding a snapshot that can be minutes old, so the group document is re-read:

```
const preSnap = await groupRef.get();
const pre = preSnap.data()?.seasons;
if (pre?.enabled !== true || pre?.currentId !== seasonId) {
  console.log('[season] close abandoned – the club moved under us', ...);
  return false;
}
```

[defect: sweep-closes-a-club-that-turned-seasons-off, P1] This check is the fix for that finding. The sweep takes one paged snapshot at the top of a run that walks every enabled club; a club that disabled seasons in between would previously have had its table wiped, its titles awarded, everyone pushed, and been left with seasons off and an archive no screen can reach — because the lifecycle write below does not carry `enabled`, so nothing turns them back on. Note the check is a plain read, not part of a transaction with the close: the window is narrowed from minutes to milliseconds, not eliminated.

The same read harvests two fields that the lifecycle write is about to destroy: `targetHistory` (reset to `[]` for the next season, so the archive is the only place it can survive — this club went `22 → 2 → 24` and only the last survives anywhere) and `targetMovedToClose`, which, if present, marks the archive `endedEarly` with the admin who moved the finish line onto the club's current position.

The lifecycle write (`:15887–15924`), in a transaction re-checking `currentId`:

```
tx.set(groupRef, { seasons: {
  currentNo: nextNo, currentId: `s${nextNo}`,
  startedAt: now,
  roundsAtStart: nextRoundsAtStart,      // await sealedEveningsOf(groupId) – the ABSOLUTE count
  playedRounds: 0,
  reopenedAt: 0,
  cadence: nextCadence,                  // rebaseCadence(cadence, args.startedAt, now)
  targetHistory: [],
  targetMovedToClose: FieldValue.delete(),
  dueBlockedSince: FieldValue.delete(),
  count: closedSoFar + 1,
} }, { merge: true });
```

`rebaseCadence` (`:15405–15447`) is a no-op for a `rounds` cadence and, for a `date` cadence, recomputes `months`, `endsAt`, `startsOn` and `endsOn` from `now`. All four are returned explicitly because the block is written with `merge:true` into a nested map: an omitted field keeps whatever the previous cadence left there. `endsOn` was the field this function once failed to return, and since the sweep asks `endsOn` first, the new season inherited the date the old one died on, was due the hour it opened, and closed again — every hour, forever, archiving an empty season each time.

`count` is read from the document inside the transaction rather than from the caller's snapshot, because `count` is what the undo button uses to decide which season to reopen (`reopenLastSeason` , `index.ts:16882-16883` : `lastId = 's' + count`). `closeSeason`'s `archived:false` latch guards the archive and nothing else, so before this transaction existed a seal-close and a sweep racing on one club both fell through and the loser rewrote the lifecycle of the season the winner had just *opened*: `startedAt` moved to now, `playedRounds` back to 0, `targetHistory` erased, `count` incremented past anything ever played. The transaction closes the race; `return advanced` then tells the caller whether it won.

`endSeasonNow` does not use any of this. It writes the equivalent block itself at `index.ts:17050-17070` — same fields, same deletes, same `rebaseCadence` — outside any transaction and with no pre-flight identity check, on the reasoning that an admin pressing a button is synchronous. It is the only close path that can run while the season has not met its target, and the only one that always stamps `endedEarly` .

Finally, `announceSeasonClosed` is called only `if (result.archived)` on both `performSeasonClose` (`:15928`) and `endSeasonNow` (`:17071`) — so a resumed close, which returns `archived: true` , will re-announce, while a redelivery, which returns `false` , will not.

[defect: partialdata-never-set-on-a-rollover-close, P2] `partialData` marks an archive whose metrics predate the counters that feed them (assists collected only from 21.06, clean sheets from 17.08). It is passed by exactly one caller — `enableClubSeasons` ' "seal now" branch, `index.ts:16483` . Neither `performSeasonClose` nor `endSeasonNow` passes it, so a season 1 that is *carried on* and then rolls over is sealed with no flag at all, and the hall of fame presents its partial numbers as complete. s1 of this club is precisely that case: its archive has no `partialData` field, and its clean-sheet total of 75 covers only the portion of the season after 17.08.

Undoing a close

The shape of the operation

Two functions do this. `reopenLastSeason` (`functions/src/index.ts:16848–16968`) is the callable an admin presses; it owns the *lifecycle* — which season the club is on, its numbering, its finish line. `reopenSeason` (`functions/src/seasonRollover.ts:1067–1253`) is the pure-ish half that owns the *numbers* — it puts the stats back and deletes the archive. The division is stated in the doc-comment at `seasonRollover.ts:1063–1065` :

The caller is responsible for restoring the club's `seasons` block — this function owns the numbers, not the lifecycle.

That split is also the source of one of the defects below, because the two halves are not atomic with respect to each other.

The client entry point is `SeasonsSettings.tsx:525–589` (`reopenLast`), rendered as an outline button at `:971–990` , gated on `live && (seasons?.count ?? 0) > 0` . The service wrapper is `src/services/seasonService.ts:264–267` , a bare callable invocation returning `{ reopenedNo }` .

What the button is for

It is not a general time machine and the code says so twice. `index.ts:16836–16845` :

Deliberately only the LAST one, and only while it is still the club's most recent: reopening season 2 of four would leave seasons 3 and 4 sitting on numbers that were counted from a table that no longer starts where they think it does. This is a correction for "I pressed it a week early", not a time machine.

Three preconditions are enforced, in this order (`index.ts:16873–16890`):

Check	Code	Failure
Caller is a club admin	<code>requireClubAdmin(data.groupId, uid)</code>	<code>permission-denied</code>
At least one closed season	<code>seasons.count >= 1</code>	<code>failed-precondition 'no closed season'</code>
Club is quiet	<code>clubIsQuiet(groupId)</code> (no open game, nothing unsealed)	<code>failed-precondition 'openGame' \ 'unsealedGame'</code>
Archive still exists	<code>seasonSummary/{groupId}__s{count}</code>	<code>not-found 'no archive for the last season'</code>

The quiet rule exists for the same reason it exists on the close: restoring a table while an evening is mid-play folds that evening's mini-games (משחקונים) into the season being reopened, and each mini-game commits separately, so the split is per-round rather than per-evening.

Note the season the button targets is computed purely from the counter: `lastNo = seasons.count; lastId = \ s${lastNo}` (index.ts:16882-16883)`. Nothing reads `currentId` to work out what the *successor* was. The successor season is simply overwritten — it is never archived, never announced, and nothing records that it existed.

The two lists, side by side

`closeSeason` writes to seven places. `reopenSeason` reads five of them back.

#	What the close does	Where	What the reopen does	Net
1	<code>create()</code> s <code>seasonSummary/{g}__{sid}</code> — every player row, every real pair's ten counters, the totals, the nine decided titles	<code>seasonRollover.ts:584–643</code>	<code>summaryRef.delete()</code> , last	destroyed
2	<code>set()</code> s the compact <code>seasonCards/{g}__{sid}</code> for the hall of fame	<code>:752–818</code>	<code>.delete()</code>	symmetric
3	<code>set()</code> s <code>users/{uid}/seasonTitles/{g}__{sid}__{key}</code> for every winner (duo key split on __, guests skipped)	<code>:823–860</code>	<code>batch.delete()</code> on the same deterministic ids, same split, same guest skip	symmetric
4	Per player row: subtracts the 18 <code>PLAYER_SEASON_FIELDS</code> , stamps <code>seasonWoundBack</code> , deletes <code>seasonReopened</code>	<code>:878–906</code>	adds the archived row back, stamps <code>seasonReopened</code> , deletes <code>seasonWoundBack</code>	symmetric <i>for archived rows only</i>
5	Per real pair: subtracts the 10 <code>PAIR_SEASON_FIELDS</code> , stamps <code>seasonWoundBack</code> , deletes <code>seasonReopened</code> . Pairs with no archive row are zeroed . Guest pairs are deleted outright	<code>:924–1002</code>	adds back only pairs present in <code>archive.pairs</code> ; then a sweep clears <code>seasonWoundBack == seasonId</code> from every other pair row	partial
6	Club doc <code>communityStats/{g}</code> : zeroes the 7 <code>CLUB_SEASON_FIELDS</code> , deletes <code>seasonReopened</code> , re-stamps <code>chemistrySince = now</code>	<code>:1004–1022</code>	adds the archived totals back, stamps <code>seasonReopened</code> . <code>chemistrySince</code> untouched	partial
7	Lifecycle (<code>performSeasonClose</code> , <code>index.ts:15884–15925</code>): <code>currentNo+1</code> , new <code>currentId</code> , <code>startedAt=now</code> , <code>roundsAtStart=eveningsSealed</code> , <code>playedRounds=0</code> , <code>reopenedAt=0</code> , re-based cadence, <code>targetHistory=[]</code> , deletes <code>targetMovedToClose / dueBlockedSince</code> , <code>count+1</code>		<code>index.ts:16896–16966</code> restores all of it from the archive	mostly symmetric — see <code>playedRounds</code> and <code>cadence</code> below
8	<code>announceSeasonClosed</code> (<code>index.ts:15324–15372</code>) creates one <code>notifications</code> doc per participant, deep-linked to the sealed season		nothing	left dangling

#	What the close does	Where	What the reopen does	Net
9	Nothing — but every game played in the discarded successor keeps its <code>seasonId</code> stamp	<code>onGameRosterChanged</code>	nothing	left dangling

Everything in the first column that has no entry in the third is permanent, because step 1 runs last and the archive is the only copy.

The permanent losses, enumerated

Guest pair chemistry (`reopen-never-restores-guest-pair-chemistry`). A guest id is minted per game, so `communityPairStats` accumulates one document per stranger-pairing for ever. The close deliberately keeps them out of the archive (`isReal()` at `seasonRollover.ts:31`, drop at `:436–439`) because copying them in would grow one document without bound until the club could never close a season again. Then `:943–949` deletes the guest pair documents outright. The restore loop at `:1129–1156` iterates `archive.pairs` only. So the data is gone from both sides at once. On the one club that has run seasons this is **293 of 314 pair documents, 93% of the collection** — I paged the whole collection live to confirm the counts. The module's own comment at `:1035–1041` states the rule and its consequence: *"a counter missing from this list is cleared by the close and never restored by the reopen, and the archive that held it is deleted at the end of the reopen. There is no way back from a gap here."*

Pairs past the archive cap. `MAX_ARCHIVED_PAIRS = 1200` (`:181`). Pair 1,201 onwards is zeroed by the close (`archivedPair` is `null`, so `windBackRow` writes absolute zeros — `:65–73`) and is not in the archive, so the reopen cannot restore it. For a 50-player club this is unreachable; for a large one it is silent.

`chemistrySince` (`chemistry-since-not-restored`). The close sets it to the close instant (`:1017`) so the new season's chemistry card does not date itself from the old one. The reopen never touches it. Live on the club right now: `communityStats.chemistrySince = 1789679528192 = 2026-09-17T21:12:08Z`, which is the moment `s2` was *closed*; `s2` has been running again since 10:37Z the next morning and its own `startedAt` is 21:05:41Z. The card therefore dates a running season's chemistry from six and a half minutes after it started.

Coverage denominators. `csRounds` and `asRounds` are deliberately *omitted* from the archive when the live row has no number for them (`:377–379`) — "measured across zero rounds" and "never measured" are different facts. But `windBackRow` has no matching branch: `out[f] = Math.max(0, rowNum(undefined) - 0) = 0`. So the close converts absent into `0` on exactly those rows, and the restore adds the archive's absent-as-zero back, i.e. nothing. The distinction is destroyed by the close and cannot be recovered by the undo. It affects the longest-standing members — the ones who predate the fields — and it blanks two efficiency columns for them permanently.

targetHistory of the discarded successor. The close seals the *closing* season's history into the archive (`index.ts:15838–15840`) and resets the block to `[]`. The reopen restores the archive's copy, or `[]` (`index.ts:16945–16947`). Anything the successor recorded — every finish-line move made during its life — is destroyed with no archive at all, because the successor is never archived. On this club the discarded `s3`'s `targetHistory` is gone; what the block holds today is one entry written **27 seconds after the reopen** (at `1789727874911 = 10:37:54.911Z` vs `reopenedAt: 1789727847680 = 10:37:27.680Z`) recording a rounds target moved `2 → 24`.

`seasonId` stamps on games played in the discarded season. Nothing rewrites them. Live: one finished game, `DTNscolRojYDf0ZmT0sF` at 2026-09-17T21:19:42Z, carries `seasonId: 's3'` while the club is on `s2` and no season 3 exists. Its *statistics* were folded into the live table the restore added to; its *evening* belongs to a season id that has been withdrawn.

The notification documents (`undo-leaves-dangling-season-summary-pushes`).

`createNotificationOnce` writes a durable doc per recipient; the reopen deletes none of them. Live, I fetched all seven by their deterministic ids (`seasonSummary:{uid}:group:HhzIwmjMl1i5HS0GHt3p:season-s1__b2959`): all seven exist, all seven have `read: false`, all created 2026-09-17T16:12:14Z. Pressing undo once more — which the button allows today — deletes `seasonSummary/...__s1` and leaves those seven live, unread, deep-linking into a season that no longer has an archive. And the client does not say so:

`seasonSummaryService.load` (`src/services/seasonSummaryService.ts:429-456`) tries the archive first, and when `archived.exists()` is false it falls straight through to the branch that builds the **running** season's model. The reader taps "עונה 1 הסתיימה" and is shown the current season under a heading for it.

The two latches

Neither half of this operation is idempotent — the close subtracts, the reopen adds — so each row carries a stamp naming the season it was last moved for.

```
// close, seasonRollover.ts:886-900 (player rows)
if (resuming && data.seasonWoundBack === seasonId) return;
const patch = {
  seasonWoundBack: seasonId,
  seasonReopened: admin.firestore.FieldValue.delete(),
  ...
};

// reopen, seasonRollover.ts:1101-1107 (player rows)
if (cur.seasonReopened === seasonId) return;
const patch = {
  seasonReopened: seasonId,
  seasonWoundBack: admin.firestore.FieldValue.delete(),
  ...
};
```

`seasonWoundBack` protects against a *close* being applied twice; `seasonReopened` protects against a *reopen* being applied twice. They are mutually exclusive by construction: whichever one is written deletes the other. Each is cleared by the opposite operation precisely so a `close→reopen→close→reopen` chain can keep working.

The `resuming &&` guard on the close side is load-bearing and recent. A `seasonWoundBack` stamp is only evidence of "already wound back" when *this invocation* wrote it — i.e. on the resume path, after the archive already exists and the wipe died halfway. On a first pass the only thing that can have left a stamp naming the

season about to be closed is a *reopen* of that same season, which handed the numbers back live while the stamp still says they were taken away. Treating it as evidence there means archiving chemistry and then skipping the wipe for it — sealing it into the write-once archive and leaving it standing live for the next season to inherit.

That is pair-wipe-skipped-on-next-close-after-reopen , and it is armed in production right now. All 314 pair documents on `HhzIwmjMl1i5HS0Ght3p` carry `seasonWoundBack: "s2"` , with `updatedAt: 1789679528192` , while `s2` is the running season. None carries `seasonReopened` . The stale-stamp sweep that was written to fix this (`seasonRollover.ts:1157-1198`) runs on *reopen* only:

```
const stamped = await db
  .collection('communityPairStats')
  .where('groupId', '=', groupId)
  .where('seasonWoundBack', '=', seasonId)
  .get();
```

The reopen that armed the state ran at `2026-09-18T10:37:27Z`; the sweep deployed at `2026-09-18T18:16:55Z` (`gcloud functions describe reopenLastSeason`). Nothing re-runs it over existing data. The 293 guest rows are no longer the exposure — the close now deletes them before it reaches the stamp check (`:943-949`) — but the 21 real pair rows will be archived and then skipped when `s2` closes.

The other asymmetry worth naming: the *player* rows only started deleting `seasonReopened` inside the close in the current HEAD (`:900`). The pair and club writes always did. Before that, a `close→reopen→play→close→reopen` cycle left every player's `seasonReopened` stamp from the *first* reopen in place, so the second reopen's per-row guard returned early for every player: club totals and pair chemistry came back, every player's season stayed at zero, and the archive was deleted on the way out. The LEDGER records this one as verified fixed in the deployed bundle.

There is no latch at all on `communityStats` for the close — it writes absolute zeros (`zeroClub` , `:1004-1022`) rather than subtracting, so it is naturally idempotent. The reopen does add, so it carries `seasonReopened` (`:1201-1216`). Live, `communityStats` holds `seasonReopened: "s2"` and nothing else from the pair collection does — which is itself the tell that the `s2` restore reached the club document and found no archived pairs to restore.

`playedRounds` , wrong in two directions

`seasons.playedRounds` is the mirror the club card prints as "N מחזורים M מתוך" ("N out of M game-nights") and the number the sweep closes on. It has had three implementations inside the undo path in two days.

```
// index.ts:16925-16936, as it stands today
playedRounds: await (async () => {
  const counted = await playedEveningsOfSeason(groupId, lastId, lastNo);
  if (counted >= 0) return counted;
  return typeof archive.get('completedRounds') === 'number'
    ? archNum(archive.get('completedRounds'))
    : Math.max(0, (await sealedEveningsOf(groupId)) - reopenedRoundsAtStart);
})();
```

1. `eveningsSealed - roundsAtStart`. A subtraction of two counters that drift for different reasons — `clubRecords.eveningsSealed` only exists from 26.08.2026. A season closed at 23 evenings came back holding 1.
2. `archive.completedRounds` (commit 4be7f1c, "A reopened season counts its own games, not a number it inherited"). Correct in principle — the archive is the record — and it inherits any error the archive holds. A junk season that a seeding bug opened and closed inside six and a half minutes had archived `completedRounds: 22`, the club's entire lifetime, and reopening it restored 22 onto a season that had held nothing. The club card then read "22 24 מתוך" while the statistics screen, which counts stamped games, read 0.
3. **Counted from the games** (today). `playedEveningsOfSeason` (index.ts:15599-15637) queries the club's terminal games ordered by `startsAt desc`, `limit 300` and hands them to `countSeasonEvenings` (functions/src/seasonCounters.ts:70-79), which asks `eveningInSeason` for scope and `didEveningHappen` for reality.

The third fix shipped in two halves, and the first half was itself wrong (`reopen-playedrounds-ignores-unstamped-games`). 4be7f1c filtered on `seasonId == lastId` with no notion of season 1. The stamp is only written from the day the feature shipped, so a club's pre-seasons history carries nothing — and **that history is season 1**. Nineteen of this club's twenty-two evenings are unstamped, so undoing season 1 wrote `playedRounds: 3`, which is the same defect as implementation 1 arriving from the other side. ae2c042 added the third parameter:

```
// seasonCounters.ts:53-60
export function eveningInSeason(game, seasonId, seasonNo?): boolean {
  const stamp = typeof game.seasonId === 'string' ? game.seasonId : '';
  return stamp ? stamp === seasonId : seasonNo === 1;
}
```

Run over the live games today (23 terminal games, `didEveningHappen` evaluated field by field): 3 stamped s1 happened, 19 unstamped happened, 1 stamped s3 did not (`playVerified: false`). So undoing today would write `playedRounds: 22` for s1 — matching the archive's `completedRounds: 22`, the card's 22, and the club screen's "מפגשים שנערכו". The reopen that actually ran yesterday wrote `playedRounds: 0` for s2, which is also correct: no evening was ever stamped s2.

Two caveats remain. The count is bounded at 300 documents and is *not* a query on `seasonId`, so a club with more than 300 terminal games silently under-counts an old season. And the evenings played inside the **discarded successor** are counted for nobody: their statistics were just added into the reopened season's live rows, while their games still carry the successor's stamp, so `playedEveningsOfSeason` skips them. Reopen a season after the successor has held five evenings and the club's table contains those five evenings' goals while its progress bar says they never happened.

Numbering, and the re-issued id

The reopen writes `count: closedSoFar - 1` and `currentNo: lastNo ( index.ts:16962 , :16901 )`. Since `currentNo` is always `count + 1`, the club's arithmetic stays consistent — but the *identifier* is not unique over time. The season the reopen discards had an id; nothing remembers that it was used. When the reopened season is closed again the lifecycle write allocates `s${seasonNo + 1} ( index.ts:15884 )`, which is the same id.

Live worked example. The club closed s1, opened s2, closed s2 six and a half minutes later, opened s3, then undid — so today: `count: 1, currentNo: 2, currentId: 's2'`, one finished game stamped `s3`, and `seasonSummary/...__s2` and `seasonCards/...__s2` both 404. When s2 closes again the next season will be s3 again, and that 17.09 game will be inside the new season 3 on its first day (`orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id`). The same re-issue also means the write-once protections reset: `seasonSummary/{g}__s3` can be `create()` d again because the old one never existed, and `users/{uid}/seasonTitles/{g}__s3__*` will be overwritten rather than duplicated.

`undo-chains-backwards-with-no-limit`. There is no time bound and no already-undone guard. The only precondition is `count >= 1` and an archive at `s${count}`; both hold again immediately after a successful reopen, because `count` was decremented and the *previous* season's archive is intact. Two presses walk back two seasons; five walk back five, each one deleting an archive, stripping its titles off profiles, and leaving its push notifications pointing at nothing. The Hebrew confirmation (`he.ts:2234-2238`) does name the season number, and does say the archive is deleted — "ויסיכום העונה והארכיון שלה יימחקו — אי אפשר לשחזר אותם אחר כך" ("the season summary and its archive will be deleted — they cannot be recovered afterwards") — but nothing says which press you are on, and nothing refuses a third.

The 48-hour grace

`REOPEN_GRACE_MS = 48 * 60 * 60 * 1000 ( index.ts:15455 )`. The reopen stamps `seasons.reopenedAt = Date.now() ( :16905 )` and two closing paths honour it:

```
// hourly sweep, index.ts:16178-16182
const reopenedAt = archNum(seasons.reopenedAt);
if (reopenedAt > 0 && now - reopenedAt < REOPEN_GRACE_MS) {
  console.log('[season] due but just reopened - holding', doc.id);
  continue;
}
// close-on-seal, index.ts:15977-15979 - same two lines
```

The reason is that a season is usually reopened *because it met its target*, so it is due the instant it comes back; without the grace the sweep would re-close it within the hour and the button would be inert. The grace is a window for the admin to move the finish line. It is cleared by the next close (`reopenedAt: 0` , `:15906`) and by enabling seasons on a club that had them off (`:16502`) — but *not* by `disableClubSeasons` , which writes only `{ seasons: { enabled: false } }` (`:16622`). A club that reopens, disables and re-enables within 48 hours hands its brand-new season the tail of somebody else's grace (`reopenedAt-survives-disable-enable`).

The grace protects against re-closing. It does **not** protect the *cadence*.

`reopen-extends-a-date-season-by-a-full-length` . `index.ts:16957-16961` re-bases the restored cadence through `rebaseCadence` (`:15405-15446`) — the function written for opening the *next* season:

```
cadence: rebaseCadence(
  (archive.get('originalTarget') ?? seasons?.cadence) as never,
  archNum(archive.get('startsAt')),
  Date.now(),
),
```

For `type: 'rounds'` this is a pass-through and is right. For `type: 'date'` it computes `months` , then sets `endsAt = addMonthsClampedServer(now, months)` , `startsOn = todayIn(now)` , `endsOn = seasonEndDate(startsOn, months)` . A six-month season closed on its end date and undone the next morning is given six more months *from today* — twelve months in total — and its `startsOn` is moved to today while `startedAt` is restored to the original start from the archive. The season then reports a start six months ago and a calendar window that begins today. The comment at `:16955-16956` says the re-base exists "so it is not immediately due again on a date cadence", which is true and is also a much larger intervention than the 48-hour grace it duplicates.

The window where the club is double-counted

`club-seasons-block-written-only-after-the-restore-succeeds` . The order is: run `reopenSeason` to completion — including `summaryRef.delete()` as its final act (`seasonRollover.ts:1252`) — and only then `ref.set({ seasons: {...} })` (`index.ts:16896`). There is no transaction spanning the two, and no compensating write.

If the invocation dies in between (the callable's ceiling is `timeoutSeconds: 300` , and the work is one transaction per player plus one per pair plus a batched title delete plus a collection scan), the club is left in a state that cannot be repaired by the same button:

- every player row and the club totals hold the reopened season's numbers **added on top of** the successor's, and are stamped `seasonReopened: s{n}` ;
- `seasonSummary/...__s{n}` and `seasonCards/...__s{n}` are gone, so nothing can say what was added;
- the club block still says `currentId: s{n+1}` , `count: n` ;
- a retry throws `not-found 'no archive for the last season'` at `index.ts:16889` , because the archive it needs to read is the one just deleted;
- even if the archive were restored by hand, every row's `seasonReopened` guard would return early.

The failure is silent, permanent, and doubles the club's whole table. Reversing the two — lifecycle first, archive delete last — would make the operation resumable, which is the property the close's own `create()` -then-wipe ordering was built around (`seasonRollover.ts:1-16`).

One more thing the re-close does not do

`reclose-after-undo-notifies-nobody` . `announceSeasonClosed` routes through `createNotificationOnce` , whose doc id is `dedupeKey + '__b' + floor(now / cooldown)` and whose cooldown for `seasonSummary` is **seven days** (`functions/src/notificationDedup.ts:95`). The write is a `create()` , and `AlreadyExists` is swallowed. The undo deletes the archive but not the notification documents, so closing the same season again inside the same seven-day bucket collides on every id and reaches nobody. The toast still says "הסתיימה ונשמרה בארכיון N עונה" ("season N has ended and was saved to the archive"), which is true, and the admin has no way to learn that the one push the whole feature builds towards was suppressed.

The nine titles

Where the code lives

The whole award decision is one pure function in one file, duplicated byte-for-byte on the server:

file	role	lines
<code>src/utils/seasonAwards.ts</code>	the canonical copy, bundled into the app	1–247
<code>functions/src/seasonAwards.ts</code>	the deployed copy, a 15-line header then an exact transcript of the client file	1–15 header, 16–262 copy
<code>tests/logic/seasonAwardsMirror.test.ts</code>	fails the build if the two ever drift	—
<code>functions/src/seasonRollover.ts</code>	the only caller: builds the input rows, calls it, seals the result	503–560
<code>src/utils/seasonMedalTier.ts</code>	grades a won title into bronze/silver/gold/platinum	1–177

Because of the 15-line header, every line number in the server mirror is the client line number + **15**. All line numbers below refer to the client file unless stated.

Nothing on the client ever calls `computeSeasonAwards`. The titles are decided exactly once, inside the close, and written into three places: `seasonSummary/{groupId}__{seasonId}.awards` (the archive), `seasonCards/{groupId}__{seasonId}.winners` (the denormalised card the hall of fame reads, `seasonRollover.ts:715–732`), and one document per winner at `users/{uid}/seasonTitles/{groupId}__{seasonId}__{titleKey}` (`seasonRollover.ts:831–859`). The client file is compiled and shipped, but on the app side it is used only for its `SeasonTitleKey` type and `SEASON_TITLE_KEYS` ordering.

The input rows, and what actually writes each field

`computeSeasonAwards` takes three arguments and reads nothing else:

```
// src/utils/seasonAwards.ts:171
export function computeSeasonAwards(
  players: readonly SeasonPlayerLine[],
  pairs: readonly SeasonPairLine[],
  completedRounds: number,
): SeasonAwards
```

`SeasonPlayerLine` (lines 26–52) is assembled at `seasonRollover.ts:503–522` from the live `communityPlayerStats/{groupId}__{uid}` rollup rows, which are read at `seasonRollover.ts:310–313` before the close wipes them. The provenance of each field is the single most important fact in this section,

because it decides which titles a club can win at all:

field	unit	written by	present for a timer-only club?
games	evenings (מחזורים)	onGameRosterChanged , functions/src/index.ts:5901 , +1 per attendee per sealed evening, no-shows excluded	yes
rounds	mini-games (משחקונים)	commitRoundStats , index.ts:14330–14338	no
goals , assists , wins , cleanSheets	mini-games	commitRoundStats	no
penTaken/penSc ored/penFaced/p enSaved	kicks	commitRoundStats §1c	no
mvpAvg	evening score 6–10	derived at seasonRollover.ts:516–519 from eveningScoreSum / eveningScoreCount , both incremented at index.ts:6378–6379	yes — but always exactly 6.0 (below)

commitRoundStats is the advanced-mode mini-game commit. A club that runs the plain live timer never calls it. So for the common club, **eight of the nine titles are decided on fields that are permanently zero**, and only mostLoyal can ever be awarded. The Hebrew settings copy at src/i18n/he.ts:2103–2107 acknowledges this in a code comment but gets the number wrong — it says such a club "can never award more than three" (לא יכול לחלק יותר משלושה); the correct figure is one.

SeasonPairLine (lines 55–62) is built at seasonRollover.ts:525–537 from communityPairStats :

```
score: p.assistsAToB + p.assistsBToA,    // directional assists, both ways
together: p.sameTeam,                    // mini-games on the same side
```

leaders() — the single ranking primitive

Every one of the nine goes through the same eight-line function (lines 144–161):

```
function leaders<T>(rows, value, id, floor = 0): SeasonAward | null {
  let best = -Infinity;
  for (const r of rows) { const v = value(r); if (v > best) best = v; }
  if (!Number.isFinite(best) || best <= floor) return null;
  const winners = rows.filter((r) => value(r) === best).map(id);
  return winners.length ? { winners, value: best } : null;
}
```

Three properties follow, and they are the whole ranking policy:

1. **Ties are shared, always.** Every row on the maximum is returned. There is no uid tie-break anywhere in this file — deliberately, per the header comment at lines 18–20. This is the opposite of the club-screen leaders (see below), and it is why a single sealed season in production carries a title with seven holders.
2. **The floor is strict (`best <= floor → null`).** With the default `floor = 0` , "everyone on zero" yields *no title*, not a winner on zero. `null` is written into the archive and rendered as ' לא זכה ' ("not awarded", `he.ts:2496`) in a fixed nine-slot cabinet.
3. **Equality is exact float equality.** The comment at lines 157–158 states this is intentional so 8.3746 and 8.3751 are two numbers. For integer titles this is harmless. For `mvp` , which is a quotient of floats, it means a shared MVP is essentially unreachable for real data ($21.6/3 = 7.199999999999999$), while the *degenerate* sentinel case ties perfectly — see below.

The nine, one by one

`SEASON_TITLE_KEYS` (lines 83–93) fixes both the type union and the render order of the medal cabinet; `SeasonHistoryScreen.tsx:143–146` chunks it three-per-shelf, so a title nobody won is drawn as an empty socket rather than closing the gap.

key	Hebrew (<code>he.ts:34–43</code>)	measures	field read	unit	floor	eligible set
<code>topScorer</code>	כתר השערים	goals scored	<code>p.goals</code>	mini-game goals	<code>> 0</code>	eligible players
<code>topAssister</code>	כתר הבישולים	assists	<code>p.assists</code>	mini-game assists	<code>> 0</code>	eligible players
<code>mvp</code>	כתר העונה	mean evening score	<code>p.mvpAvg</code>	6–10 score	<code>> 6</code> (<code>MVP_SCALE_FLOOR</code> , line 141)	eligible players
<code>topWinner</code>	כתר הניצחונות	mini-games won	<code>p.wins</code>	mini-games	<code>> 0</code>	eligible players
<code>mostLoyal</code>	כתר ההתמדה	evenings attended	<code>p.games</code>	evenings	<code>> 0</code>	eligible players
<code>cleanSheetKing</code>	כתר השער הנקי	mini-games whose side conceded nothing	<code>p.cleanSheets</code>	mini-games	<code>> 0</code>	eligible players
<code>penaltyKing</code>	כתר הפנדלים	conversion rate	<code>penScored/penTaken</code>	ratio 0–1	<code>> 0</code>	eligible and <code>penTaken >= minAttempts</code>
<code>penaltyKeeper</code>	כתר העצירות	save rate	<code>penSaved/penFaced</code>	ratio 0–1	<code>> 0</code>	eligible and <code>penFaced >= minAttempts</code>

key	Hebrew (he.ts:34–43)	measures	field read	unit	floor	eligible set
deadlyDuo	הצמד הקטלני	assists exchanged between two players	assistsAToB + assistsBToA	assists	> 2 (MIN_DUO_ASSISTS - 1 , line 244)	pairs where both halves are eligible

Every one of them shares ties; there is no per-title tie-break to document, because there are none.

Two units are worth flagging for the reader, because the product's own vocabulary collides here. `mostLoyal` counts **evenings** and is rendered 'מחזוריים' (he.ts:2443–2444). `cleanSheetKing` , `topWinner` , `topScorer` , `topAssister` count **mini-games** or events inside them. The same season card therefore prints "מחזוריים 19" beside "שערים נקיים 13" where the 19 counts evenings and the 13 counts mini-games out of 37 — two denominators, neither shown.

`deadlyDuo` is the only title whose id is not a uid: `leaders` is given ``${p.a}__${p.b}`` (line 243), and the title-write loop at `seasonRollover.ts:837` splits that key back apart to place one document on each of the two profiles, skipping any half whose id starts with `guest:` .

Defect — `deadlyduo-together-is-collected-and-never-read` . `SeasonPairLine.together` (line 61, "Rounds the two were on the same side") is populated at `seasonRollover.ts:536` from `p.sameTeam` and is never read by `computeSeasonAwards` — `grep -n together src/utils/seasonAwards.ts` returns exactly one hit, the field declaration. The partnership-volume gate the field exists for was never written. A pair that played together twice and exchanged three assists outranks a pair that played together fifteen times and exchanged three, and the code has the number to distinguish them sitting unused in the same object.

The eligibility gate

```
// lines 102–117
export function eligibilityThreshold(completedRounds: number): number {
  return Math.ceil(Math.max(0, completedRounds) / 2);
}
export function isEligible(line: Pick<SeasonPlayerLine, 'games'>, completedRounds: number): boolean {
  return line.games >= eligibilityThreshold(completedRounds);
}
```

Applied once at line 190 (`const eligible = players.filter(...)`) and inherited by all nine titles. Pairs inherit it transitively at lines 195–198: both halves must clear it on their own, so a regular and a one-night guest cannot take the duo between them.

A zero-length season short-circuits before any of this (lines 184–189) and returns nine `null` s, because `eligibilityThreshold(0) === 0` would otherwise open every gate.

The user-facing promise is at `he.ts:2416` :

'...לא חולקו תארים בעונה הזאת. תואר ניתן רק למי שהגיע לפחות לחצי מערבי המשחק של העונה' "No titles were awarded this season. A title goes only to whoever attended at least half of the season's game evenings."

Now the subtlety that makes that sentence false. The `completedRounds` argument is *not* the season's length. The caller computes it at `seasonRollover.ts:556–559`:

```
const seasonEvenings = awardsDenominatorOf(
  awardLines.map((l) => l.games),
  args.completedRounds,
);
const awards = computeSeasonAwards(awardLines, awardPairs, seasonEvenings);
```

and `awardsDenominatorOf (seasonRollover.ts:133–139)` is:

```
export function awardsDenominatorOf(attendances: readonly number[], fallback: number): number {
  if (attendances.length === 0) return Math.max(0, rowNum(fallback));
  return Math.max(0, ...attendances.map((g) => rowNum(g)));
}
```

The denominator is **the maximum attendance of any single player**, and the true season length (`args.completedRounds`, ultimately `clubRecords.eveningsSealed` minus the season's starting offset, via `completedRoundsOf` at `index.ts:15695–15708`) is used only as a fallback when the club has no player rows at all. The stated rationale, written out at `seasonRollover.ts:114–121`, is a units/era argument: per-player `games` has been counted since 22.06 while the sealed-evening counter was only born on 25.08, so comparing a numerator from one era against a denominator from the other admitted far too many players. `max(games)` is at least in the same unit and the same era as the numerators it is compared against.

The consequence is algebraic and unconditional. Let `D = max_i(games_i)` and let `p*` be a player attaining it.

- `isEligible(p*, D)` is `games_{p*} >= ceil(D/2)`, i.e. `D >= ceil(D/2)`, which is true for every `D >= 0`. **The top attendee can never fail the gate.**
- `mostLoyal = leaders(eligible, p => p.games)` returns `best = max` over the eligible set. Since `p*` is always in that set, `best = D`. `awards.mostLoyal.value === D` **identically, for every attendance distribution.**

So the gate's denominator is, by construction, the value of one of the titles it gates — and it is a *different number* from the season length that the same close writes to `seasonSummary.completedRounds` (line 625) and `seasonCards.completedRounds` (line 767).

On the real archive. `seasonSummary/HhzIwmjM1i5HS0GHt3p__s1`, read live:

```
players[*].games = { 1IdtNEjb: 19, alsobLSA: 18, K5rSGB4J: 18,
                    B5KpY04I: 18, CEkRfDs2: 18, YIZlKWbV: 18, JoLFRxFr: 17 }
completedRounds = 22
awards.mostLoyal = { winners: ['1IdtNEjb...'], value: 19 }
```

`D = 19 ; mostLoyal.value = 19 ;` the season was 22 evenings long. The gate that the copy calls "half the season" was `games >= ceil(19/2) = 10` , i.e. **10 of 22 = 45%**. All seven members cleared it either way here, so on this club the gate's membership did not change — but a title did, which is the point below.

Defect — eligibility-denominator-is-max-attendance . Two numbers describing "how long was this season" live in one archive document (19 and 22) and neither is labelled as the other's alternative; `awardsDenominator` is no longer published anywhere (see below), so a reader of the sealed archive cannot tell which denominator the titles were decided on without re-deriving `max(games)` by hand.

Defect — loyalty-crown-is-structurally-platinum (fixed at HEAD, and worth the reader's attention as a worked example of the identity). For about a day, `seasonRollover.ts` wrote `awardsDenominator: seasonEvenings` onto the season card and `SeasonHistoryScreen.tsx` graded the loyalty medal as `medalTier('mostLoyal', w.value, season.awardsDenominator ?? season.completedRounds)` . Because `w.value === awardsDenominator` identically, the ratio was **1.0** for every season of every club, and the attendance scale's platinum stop (`>= 1`) fired unconditionally: silver and gold became unreachable code. Both the field and the read are gone at HEAD — `SeasonHistoryScreen.tsx:172` is now `medalTier(key, w.value, season.completedRounds)` and `seasonRollover.ts:786–800` carries a long comment explaining why the field must not come back. The production `seasonCards/HhzIwmjMl1i5HS0GHt3p__s1` predates the field and does not carry it. **Nothing recomputes a sealed season**, so this is history rather than live damage — but the *gate* still uses `max(games)` , which is the same identity in the place it was not removed from.

The volume gates

```
// lines 126–128
export function minPenaltyAttempts(completedRounds: number): number {
  return Math.min(5, Math.max(2, Math.ceil(Math.max(0, completedRounds) / 10)));
}
```

Two attempts minimum, one more per ten "rounds", capped at five. Because the argument is `max(games)` rather than the season length, this gate moves with the club's best attendee. **On production s1 this flipped a real title.**

denominator	minPenaltyAttempts	qualifying kickers	rates	penaltyKing
<code>max(games) = 19</code> (what ran)	2	alsobLSA 2 taken, K5rSGB4J 3 taken	2/2 = 1.000 , 2/3 = 0.667	הלן צברי , value 1.0
<code>completedRounds = 22</code>	3	K5rSGB4J only	2/3 = 0.667	Nofar Tzabari, value 0.667

The sealed archive holds `penaltyKing: { winners: ['alsobLSA...'], value: 1 }` — reproducing the stored result requires 19, not 22, which is independent confirmation that the denominator in use is the maximum attendance. (`penaltyKeeper` is unaffected: at both 2 and 3 the qualifying keepers are the same three, and alsobLSA's 3 saves from 3 faced wins either way.)

Defect — `penalty-titles-crowned-on-two-kicks`. A season crown at a perfect 100% off **two kicks** is what the archive actually contains, and `medalTier('penaltyKing', 1.0, ...)` grades it platinum (`RATE_STEPS = [0.7, 0.85, 1]`, `seasonMedalTier.ts:70`). The poster renders `'100% הצלחה בפנדלים'` (`he.ts:2461`).

Two methods, one Hebrew name. The club screen crowns `כתר הפנדלים / מלך הפנדלים` from a completely different algorithm. `src/utils/penaltyStats.ts:132–162` ranks by a **Wilson score lower bound** (`wilsonLowerBound`, lines 120–128, $z = 1.96$):

```
// penaltyStats.ts, pickLeader: rank by Wilson desc, then count, then attempts, then uid asc
score: wilsonLowerBound(count, attempts)
```

Its doc comment states the design intent explicitly: *"8/10 (0.49) outranks 1/1 (0.21), so a one-shot 100% no longer beats a proven high-volume scorer."* The season title does the exact opposite — a raw rate behind a hard floor, where 2/2 beats 2/3 and beats 8/10. On the same seven players, the club card and the season medal can name different people for the same crown, and since `he.ts:1370` points `communityStatsPenaltyKing` at the same `TITLE.penaltyKing` string, the two answers are labelled identically. There is also a second inconsistency in the same pair: `penaltyStats.pickLeader` breaks ties deterministically (`cand.userId < best.userId`, line 155) and returns exactly one leader, while `leaders()` shares. One crown, two rankers, two tie policies.

The same split exists for clean sheets. **Defect** — `cleansheetking-ranks-on-count-while-the-club-table-ranks-on-rate`. The season title is `leaders(eligible, p => p.cleanSheets)` — a raw count. The club's efficiency tab shows a sortable `cleanSheetPct` column (`CommunityStatsTable.tsx:251–252`) computed as `cleanSheets / csRounds` (`efficiencyStats.ts:80–83`), and the club's leaders card uses yet a third function, `leaderBy` (`CommunityStatsScreen.tsx:118–132, 463`), which is a strict-`>` scan that silently keeps the **first** row on the maximum — no shared ties, no floor beyond zero. On s1 the three disagree:

player	cleanSheets	csRounds	rate
alsobLSA (הלן צברי)	13	17	76.5%
1IdtNEjb (מתן לוי)	12	15	80.0%
YIZlKBv (Eliran Tzabari)	11	13	84.6%

The season crowned `הלן צברי` on the count; the club's own rate column puts Eliran Tzabari top. Both are labelled `TITLE.cleanSheetKing = 'כתר השער הנקי'`.

The MVP

`mvp` is the mean of a player's *evening* scores: `mvpAvg = eveningScoreSum / eveningScoreCount` (`seasonRollover.ts:516–519`), with `0` substituted when the count is zero. The per-evening score comes from `eveningScoreServer` (`functions/src/eveningScoreCore.ts:34–65`), whose last line is:

```
if (gamesPlayed <= 0) return 6.0; // line 43 – the sentinel
...
const score = 6 + (weighted / 10) * 4;
return Math.round(Math.min(10, Math.max(6, score)) * 10) / 10; // line 64 – clamp to [6,10]
```

Three facts follow, and together they are the worst defect in this section:

1. The scale's **floor is 6.0, not 0** — a genuinely bad evening cannot score below 6.
2. **6.0 is also the sentinel** returned when `gamesPlayed <= 0`. `gamesPlayed` here is the player's *mini-game* count for the evening (`index.ts:6288`, `e.rounds`), which is zero for every evening a club runs on the plain timer. So the value that means "we recorded nothing about this player tonight" is numerically identical to the bottom of the scale.
3. **The sample size is never checked.** The eligibility gate tests `games` (evenings attended); `eveningScoreCount` is a different counter that only started incrementing when the accumulator shipped. A player with 19 attended evenings and one scored evening is compared on a mean of one.

Defect — `mvp-title-held-by-the-entire-club`. With the original `floor = 0`, the sentinel cleared the floor and every eligible player tied at exactly 6.0. Production `s1` is exactly that:

```
eveningScoreSum / eveningScoreCount = 18/3, 18/3, 18/3, 18/3, 18/3, 12/2, 12/2 → 6.0 × 7
awards.mvp = { value: 6, winners: [ ...all seven uids... ] }
```

Seven of the club's seven members hold `כתר העונה` at the value that means "nothing was recorded". A collection-group query over `seasonTitles` across the entire database returns **14 documents, of which 7 are mvp at value 6** — half of every season title ever awarded in this product is the no-data sentinel, shared by an entire club. Note also that `eveningScoreCount` is 2 or 3 against `games` of 17–19: even setting the sentinel aside, the "season average" was a mean over the last two or three evenings of a 22-evening season.

At HEAD this specific outcome is closed, by one argument added at line 221:

```
const MVP_SCALE_FLOOR = 6; // line 141
mvp: leaders(eligible, (p) => p.mvpAvg, (p) => p.uid, MVP_SCALE_FLOOR), // line 221
```

`best <= 6` now returns `null`, so a club whose evenings are all sentinels awards no MVP at all. Two residual problems remain, and neither is addressed:

- The sealed archive is not recomputed. `seasonSummary...__s1.awards.mvp`, the card's `winners`, and the seven `users/*/seasonTitles/*__mvp` documents all still exist in production and still render.
- A player with a *single* real mini-game evening (score, say, 6.3) and eighteen sentinel evenings averages $(6.3 + 18 \times 6.0) / 19 = 6.016$ — above the floor — and takes the title over teammates who are all flat 6.0. The floor removes the degenerate all-tie; it does not make the metric a season average of anything.

The medal tier

`src/utils/seasonMedalTier.ts` grades a *won* title's value into one of four metals. It is a pure presentation layer: nothing in the award computation reads it, and the tier is not stored — `SeasonHistoryScreen.tsx:172` recomputes it on every render from the card's `value` and `completedRounds`.

`SCALE` (lines 57–67) is a **total** `Record<SeasonTitleKey, Scale>`, deliberately, so a tenth title is a compile error rather than a silent bronze. Four kinds:

```

topScorer:      { kind: 'count', steps: [8, 16, 28] }
topAssister:    { kind: 'count', steps: [6, 12, 22] }
topWinner:      { kind: 'count', steps: [10, 20, 34] }
cleanSheetKing: { kind: 'count', steps: [5, 11, 20] }
deadlyDuo:      { kind: 'count', steps: [5, 11, 20] }
penaltyKing / penaltyKeeper: { kind: 'rate' }          // RATE_STEPS = [0.7, 0.85, 1]
mostLoyal:      { kind: 'attendance' }                  // ATTENDANCE_STEPS = [0.6, 0.8, 1], value /
mvp:            { kind: 'eveningScore' }                // MVP_STEPS = [6.6, 7.6, 8.8]

```

`step()` (lines 92–97) is `>=` at every stop, so `1.0` on a rate is platinum and `value === completedRounds` on attendance is platinum. `medalTier` guards `completedRounds <= 0` → 'bronze' for the attendance scale only (line 120).

The count steps are absolute constants tuned against nothing documented, and they are graded against **mini-game** totals. On a club that plays ~37 mini-games a season, `topWinner`'s platinum at 34 wins and `topScorer`'s at 28 goals are not reachable; the season's entire club goal total was 27.

`MVP_STEPS` carries an honest comment (lines 72–86) admitting the original `[6.5, 7.5, 8.5]` was written against an assumed 1–10 scale that the clamp makes impossible, and that a "correction" to `[7, 8, 9]` made the title *harder* to lift off the floor. The current `[6.6, 7.6, 8.8]` are quarters of the real `[6, 10]` range; the club's own evening-standings leader sits at 6.93, which is silver. The ledger's `mvp-medal-can-never-leave-bronze` finding referred to the earlier constants.

`titleStreak` (lines 141–177) counts consecutive seasons with the same holder set, walking towards older seasons and stopping at a gap in `season.no`. It matches **by display name**, joined on `U+0000`, because a sealed `SeasonWinner` is `{key, names, value}` and carries no uid — a player who renames themselves silently ends their own streak, and the code says so in its comment rather than pretending otherwise. No production club has closed two seasons, so this path has never run against real data.

The one sealed season, end to end

Everything above, applied to `HhzIwmjMl1i5HSOGHt3p__s1` (מועדון שכחת שושי), season 1, 22 evenings, 37 mini-games, 7 members:

title	winner(s)	value	medal at HEAD	note
topScorer	הלן צברי	10 goals	silver (8/16/28)	37% of the club's 27 goals
topAssister	Nofar Tzabari	5 assists	bronze (6/12/22)	club total 13
mvp	all seven members	6.0	bronze	the sentinel; would be <code>null</code> at HEAD
topWinner	Lioz Madar	13 wins	silver (10/20/34)	mini-games, not evenings
mostLoyal	מתן לוי	19 evenings	gold (19/22 = 86%)	was platinum for one day; <code>D</code> = 19
cleanSheetKing	הלן צברי	13	gold (5/11/20)	rate ranking would name Eliran Tzabari

title	winner(s)	value	medal at HEAD	note
penaltyKing	הלן צברי	1.000	platinum	two kicks ; flips to Nofar at D = 22
penaltyKeeper	הלן צברי	1.000	platinum	three faced, three saved
deadlyDuo	—	—	empty socket	best pair exchanged 2 directional assists, floor is > 2

Eight of nine awarded; one player holds four of the eight; one title is held by the entire club. `seasonCards/..._s1.winners` carries all eight with names frozen at close time, and fourteen `seasonTitles` documents were written across seven profiles.

Season scope versus lifetime — how the statistics are divided

Every number on a club screen comes from one of four places: a **live scan** over the club's finished game documents (`gameService.getCommunityStats`), a **live rollup** of per-player counters (`communityPlayerStats` , read through `gameService.getCommunityChampionship`), a **sealed archive** of one closed season (`seasonSummary/{groupId}__{seasonId}` , plus its small twin `seasonCards`), or a **merge** of the live rollup with every archive (`src/utils/allTimeTable.ts`). Closing a season zeroes the rollups and leaves the scan untouched, so the divide between "this season" and "all time" is not one mechanism — it is two different mechanisms that have to be kept in agreement by hand, on every screen, for every row. This section is about the scan, which is the half that was retro-fitted with a season filter, and about the five places that call it.

All line numbers are HEAD (`e35394a` , 2026-09-18 21:40 +0300, version 1.1.10). Where a defect was fixed after the shipped store binaries were cut, that is stated: the 1.1.9 artifacts on both stores were built from `4de80d0` , which predates most of what follows (ROUND4 #1).

`gameService.getCommunityStats` — one query, two tallies

`src/services/gameService.ts:949–1262` . The signature is one required club id and one optional season scope:

```
async getCommunityStats(  
  groupId: GroupId,  
  season?: { currentId: string; currentNo: number },    // :965  
): Promise<{ /* ~14 fields, listed below */ }>
```

The query (`:1112–1117`) is fixed and bounded:

```
const q = query(  
  col.games(),  
  where('groupId', '==', groupId),  
  where('status', 'in', ['finished', 'cancelled']),  
  orderBy('startsAt', 'desc'),  
  limit(200),  
);
```

So the universe is **the 200 most recent terminal game documents of this club, newest first**. Nothing paginates past that. A permission-denied read returns the all-zero `empty` object (`:1008–1033`); any other error is logged to the `errors` inbox and re-thrown, which every caller turns into `null` with a `.catch` .

The loop (`:1150–1217`) runs **once** and maintains **two independent tallies**. The lifetime tally is computed first, in a bare block *before* the season gate:

```

for (const doc of snap.docs) {
  const g = doc.data();
  {
    // :1155 – lifetime, unscoped
    const st = eveningPlayState(g as PlayableEvening);
    if (g.status === 'cancelled' || st === 'notHappened') lifeCancelled += 1;
    else if (st !== 'unverified') {
      lifeFinished += 1;
      /* attendance set, 30d / 365d windows, lifeNights.push(...) */
    }
  }
  if (!inSeason(g as { seasonId?: string }, season)) continue; // :1171 – the gate
  const state = eveningPlayState(g as PlayableEvening); // :1179
  if (g.status === 'cancelled') { totalCancelled += 1; continue; }
  if (state === 'unverified') continue; // :1188 – counts for neither
  if (state === 'notHappened') { totalCancelled += 1; continue; } // :1191
  totalFinished += 1; // :1195
  /* thisMonthFinished, attendedTally, activeMonth, activeYear, nights.push(...) */
}

```

Two things are worth naming here. First, *whether an evening happened* is never re-derived: both tallies ask `eveningPlayState (src/utils/eveningPlayed.ts)`, which returns `'happened'`, `'notHappened'`, `'unverified'` or `'pending'`. An `'unverified'` night — auto-closed by the sweep with no timer, no goals and no rotation, and no admin ruling — is excluded from **both** halves of the organisation-rate fraction rather than counted as a failure. Second, the lifetime block has no `thisMonthFinished`, no `attendedByUser`, no `avgAttendance` and no `topPlayers`: those four exist only in the season-scoped half, which is why a caller that wants an unscoped attendance count must pass no season at all rather than read `lifetime`.

The return (:1230–1260) is:

Field	Scope	Meaning	Absent →
<code>totalFinished</code>	season	evenings <code>'happened'</code> in scope	0
<code>totalCancelled</code>	season	<code>cancelled</code> + <code>notHappened</code> in scope	0
<code>organizationRate</code>	season	<code>finished / (finished + cancelled)</code> , 0 when both are 0	0
<code>avgAttendance</code>	season	member arrivals ÷ finished nights	0
<code>thisMonthFinished</code>	season	finished within 30 days	0
<code>activeThisMonth</code>	season	distinct attendees, 30-day window $\cap$ season	0
<code>activeThisYear</code>	season	distinct attendees, 365-day window $\cap$ season	0
<code>topPlayers</code>	season	top 5 {uid, attended}, desc	[]

Field	Scope	Meaning	Absent →
<code>attendedByUser</code>	season	uid → nights attended (the authoritative "הופעות")	<code>{}</code>
<code>longestStreak</code> / <code>longestStreakUid</code>	season	longest consecutive-night run	0 / <code>null</code>
<code>currentStreakByUser</code>	season	uid → current run	<code>{}</code>
<code>lifetime.totalFinished</code> / <code>.totalCancelled</code> / <code>.organizationRate</code>	lifetime	as above, unfiltered	0
<code>lifetime.activeThisMonth</code> / <code>.activeThisYear</code>	lifetime	as above, unfiltered	0
<code>lifetime.longestStreak</code> / <code>.longestStreakUid</code>	lifetime	the club's permanent record	0 / <code>null</code>

The streak is computed by one shared helper, `lifetimeStreak` (:550–575), called twice — once on `nights` (:1218) and once on `lifeNights` (:1239) — so the season record and the lifetime record can never be computed by two different rules. It walks nights oldest→newest, resets a player's run on a missed night, and keeps the first uid to reach each new maximum (`if (run[uid] > longestStreak)` is strict). On a club where several regulars never miss, the named holder is therefore decided by `g.players` array order in the earliest night of the window — ROUND4 #28.

`limit-200-window-is-a-lifetime-label`. The doc-comment at :934–947 is honest that this is a window, not all time, but the field is named `lifetime` and the screen renders it under the word "אי פעם" ("ever"). For a club past 200 terminal documents — roughly four years of weekly play, or one year for a club that also cancels — `lifetime.totalFinished` silently becomes "the last 200 attempts". Worse under scoping: the `limit(200)` is applied by Firestore **before** `inSeason` runs in JavaScript, so a season that sits entirely outside the newest 200 documents scores zero with no error. For the one production club (23 terminal docs) neither bound bites.

`avgAttendance` **counts members only** (ROUND4 #16). The inner loop iterates `g.players` (:1207); guests live in a separate `g.guests` array whose ids never appear there. Recomputed over the real club's 22 held evenings: **141 member arrivals against 98 guest slots**, so the screen renders $141/22 = 6.4$ for nights that averaged 10.9 on the pitch — one night was 7 members and 11 guests. The label is "ממוצע הגעות למחזור" ("average arrivals per game-night"), and the club's own sealed archive records `guestGoals: 4`, so guests are participants elsewhere.

The season predicate

`src/utls/seasonScope.ts` holds the whole rule:

```
export function inSeason(game: { seasonId?: string }, season?: SeasonScope): boolean {
  if (!season) return true;
  const stamp = typeof game.seasonId === 'string' ? game.seasonId : '';
  return stamp ? stamp === season.currentId : season.currentNo === 1;
}
```

Three cases. **No scope** → everything passes, and the function degenerates into the pre-seasons lifetime scan. **A stamp** → exact string equality with the running season id. **No stamp** → the evening belongs to season 1, because the `seasonId` stamp is written by the server only from the feature's ship date onward, and "your history becomes season 1" is what enabling seasons promises.

`unstamped-season-1-rule-disagrees-with-the-server-by-19` — **now fixed, and the fix has a sharp edge**. The server's copy is `eveningInSeason` in `functions/src/seasonCounters.ts:53–60`, character for character the same rule, and `functions/src/index.ts:15599–15637` (`playedEveningsOfSeason`) now passes `seasonNo` through to it. Before that fix the server counted only stamped games and answered **3** where the client answered **22**; a reopen of season 1 would have written 3.

`tests/logic/seasonCounterReconciliation.test.ts` runs both copies over one fixture.

The edge is what the rule does for a club **past** season 1. Production, `HhzIwmjMl1i5HSOGHt3p` (מועדון שכחת שושי), read live:

```
groups/HhzIwmjMl1i5HSOGHt3p.seasons =
  { enabled: true, currentId: 's2', currentNo: 2, count: 1,
    playedRounds: 0, roundsAtStart: 10, startedAt: 1789679141952,
    cadence: { type: 'rounds', targetRounds: 24 } }
```

23 terminal game documents. Their stamps: **19 carry none, 3 carry 's1', 1 carries 's3'** (an orphan — the club's seasons block has never known an `s3`). With `season = {currentId:'s2', currentNo:2}`, `inSeason` returns false for all 23: the unstamped ones fail `currentNo === 1`, the `s1` and `s3` ones fail the string compare. **Not one game in the club's history is in scope for the season it is currently playing**. Every season-scoped field above is therefore `0 / {} / null`.

A structural consequence nobody has written down: the stamp is written by `onGameRosterChanged` only on a transition into `active` or `finished` (`functions/src/index.ts:5802–5810`). A game cancelled from `open` never reaches either, so it is never stamped, so for any club in `season ≥ 2` a genuine cancellation can never enter `totalCancelled`. The season-scoped `organizationRate` is therefore **either exactly 0 (no evenings in scope) or exactly 1.0** — the only way to lose points is the `notHappened` route, an evening that kicked off and was later declared not to have happened. It is not a rate; it is a two-valued flag wearing a percentage sign.

The five callers

Caller	Passes a season?	Scope it gets	What it renders
<code>CommunityStatsScreen.tsx:199</code>	yes (:193–197)	season, with <code>lifetime</code> beside it	hero evening tile, <code>mostLoyal</code> , org donut, streak row, "פעילים השנה", <code>attendedByUser</code> → league table, badges, club level
<code>playerCompareService.ts:159</code>	yes (:146–152)	season	the <code>games</code> / "מהזורים" row of the compare card, and the denominator of every per-game rate on it
<code>CommunityDetailsScreen.tsx:189</code>	no	lifetime	"מפגשים שנערכו" headline, and the four-cell <code>CommunityStatsSection</code>
<code>assistantInsightsService.ts:170</code>	no	lifetime	the coach's attendance streak and "נ' ערבים על הדשא" lines
<code>SeasonsSettings.tsx:341</code>	no	lifetime	<code>history</code> , the club's pre-seasons evening count shown in the activation confirm sheet

Two of those five are deliberate and correct, and three are deliberate and unscoped. The two season-passing callers both read the club document first for the sole purpose of learning the scope, and both carry a comment saying so — one extra document read per screen.

`CommunityStatsScreen` (:182–200) resolves the scope from the group:

```
const g = await groupService.get(groupId).catch(() => null);
const scope = g?.seasons?.enabled && g.seasons.currentId
  ? { currentId: g.seasons.currentId, currentNo: g.seasons.currentNo ?? 1 }
  : undefined;
```

(The local `scope` shadows the screen's `scope` state for the length of the effect; harmless, but it is the same word meaning two different things eight lines apart.) A club with seasons switched off passes `undefined` and gets its lifetime numbers in both halves, which is why the same function serves both kinds of club.

`CommunityDetailsScreen` is the screen the owner's original complaint was about. It calls with no season (:189) and renders `communityStats.totalFinished` as `matchesHeld` (:614) plus a four-cell block (:1284–1310) reading `totalFinished`, `thisMonthFinished`, `organizationRate` and `avgAttendance`. This is right for what it is — a club screen, not a season screen — but nothing on it says so.

`evenings-22-on-one-screen-0-on-the-next` (ROUND4 #2) is the collision of those two rows in the table. Hand-recomputed from the 23 raw documents, exactly as the code computes it:

```
lifetime: finished 22, cancelled 1, organizationRate 0.9565 (96%),
  activeThisMonth 7, activeThisYear 7,
  longestStreak 22 held by 1IdtNEjbEXfiRSqvLrJVn99NsfI2,
  attendance { 22, 20, 20, 20, 20, 20, 19 }
season s2: finished 0, cancelled 0, organizationRate 0, activeThisYear 0,
  longestStreak 0, attendedByUser {}
```

So `CommunityDetailsScreen` prints "22 מפגשים שנערכו" and "96% אחוז הצלחה בארגון", and one tap away `CommunityStatsScreen` prints "0 מחזורים". Both are correct answers to different questions, neither screen states which question it asked, and the shipped 1.1.9 binary makes it worse: at 4de80d0 the organisation donut read the *season* rate (`stats?.organizationRate` , :876 of that revision) and rendered **0% in green** — "no evening was ever cancelled" and "every evening was cancelled" render identically.

`playerCompareService` — `compare-card-divides-season-goals-by-lifetime-nights` is **fixed at HEAD and unshipped**. The card's goals/assists/wins/rounds come from `getCommunityChampionship` (the rollup a close zeroes) while `games` is overridden from the scan (:191–192 : `games: attended[uidA] ?? baseA.games`), and `toPlayer` divides one by the other (:103 : `goalsPerGame = row.goals / row.games`). With an unscoped scan that is *this season's goals over the club's lifetime evenings*. On this club it produced goals 0-0, assists 0-0, 0.0-0.0 מחזור למחזור, ממוצע גולים 0%, אחוז ניצחון 0%-0% and then 22-20 מחזורים, a single non-zero row, which set `verdict.leader` and printed "אתה מוביל 🏆 ב-1 מתוך 6 קטגוריות" ("you lead in 1 of 6 categories") over five zeros — on a card designed to be captured to PNG and shared. HEAD passes the season, which makes both columns read 0 and makes the new `if (!baseA || !baseB) return null` guard (:178) the thing that stops the card rendering at all. The club league table solved the same problem the other way:

`CommunityChampionship.tsx:179` suppresses the override outright (`attendedByUser={seasonNo || seasonId || rows ? undefined : attendedByUser}`) rather than re-scoping it.

`assistantInsightsService` deliberately stays lifetime. Its two consumers are `rules.ts:425–438` : `attendanceStreak >= 3` → "הגעות רצופות N אתה מחזיק רצף של" and `attendedNights >= 10` → "נערכים על הדשא N". Both are statements about a person's history, not about a season, so the unscoped scan is the right source — but the *same object* also carries table-derived places and crowns from the season-zeroed rollup, so one coach card mixes a 22-night lifetime streak with a season standing of nothing. ROUND4 #6 described `getClubInsight` returning `null` before it could say either; HEAD splits the two cases (:163 `if (!mine && rows.length > 0) return null`) so the attendance half still speaks when the table is empty. The 200-document scan is still paid for, every 6 h per user, on a branch named `perf/firestore-read-costs` .

`SeasonsSettings` (:339–351) uses the scan for something else entirely: `st.totalFinished` is the club's *pre-seasons history*, the number the activation sheet promises will become season 1. Unscoped is correct — the club has no season yet — and the call is made only on the `firstTime` branch. For a live club the same state is filled from `seasons.playedRounds` instead (:336), which for this club is 0 while the scan would say 22.

The three scopes on the statistics screen

`scope` is a union (:60): `{k:'current'}` , `{k:'season', id}` , `{k:'all'}` . `slice` (:356) is `null` for the running season, the parsed archive for a closed one, and the merge for all-time; the whole screen derives from `viewChamp / slice` rather than from `champ` directly, so a past season renders through exactly the same

code path as the running one.

Row on screen	running season	closed season	all time
tiles / גולים / בישולים / משחקונים	live rollup (champ)	archive totals	mergeAllTime totals
tile (: 769) מהזורים	season scan totalFinished	card scopedCard.completedRounds	lifetime.totalFinished
top scorer hero, leaders card	live rollup rows	archive players rows	merged rows
mostLoyal / כתר ההתמדה (: 479–490)	scan topPlayers[0]	archive games column	merged games column
הצמד הקטלני	live communityPairStats	archive pairs	not offered (pair data is per-season)
organisation donut (: 907–914)	lifetime	hidden	hidden
longest streak row (: 943–953)	lifetime	hidden	hidden
פעילים השנה (: 975–985)	lifetime activeThisYear	replaced by "N בעונה שיחקו" from row count	same replacement
section כימיה	live pairs	hidden	hidden
league table (own fetch)	rollup with keepAll=true	archive	merged rows
club badges + level (: 494–517)	lifetime + archived goals	hidden	hidden

Three of those rows are lifetime *inside* the season scope, and the tooltip under the picker says so:

ה. communityStatsScopeClosedInfo — "הכימיה בין השחקנים ותארי המועדון נמדדים על המועדון" — "the organisation rate, the streaks, the chemistry between players and the club titles are measured over the club's whole life, not over a single season — which is why they are shown only in the running season"). The reason is that a record and a badge are permanent things the club did: season-scoping them meant a club un-earned its gold "שערי המועדון" badge and dropped a club level the morning after every close, and its 22-night attendance record vanished from the app entirely.

closed-season-tooltip-promises-lifetime is **true of HEAD and false of every shipped binary**. At 4de80d0 the donut read stats?.organizationRate, the streak row read stats.longestStreakUid / stats.longestStreak, and "פעילים השנה" read stats?.activeThisYear — all three season-scoped, all three zero, under a tooltip stating they were lifetime. Commit 5531a21 switched exactly those three to stats.lifetime.* and added the fields to make it possible. It also created ROUND4 #13: the **tile מהזורים** above them stayed **season-scoped**, so the next build reads "0 מהזורים", then "96% יצאו לפועל", then "הגיע 22 מהזורים ברצף", then "שחקנים היו פעילים השנה 7" — four rows, two scopes, one card, nothing marking the boundary. The old version was consistently wrong; this one is inconsistently right.

`activeThisYear-is-season-intersected` is why that switch was needed. The 365-day window is applied *inside* the post- `inSeason` branch (`:1213`), so the season figure is the intersection of a year and a season. A season three days old makes "השנה" ("this year") mean three days, and this club — whose entire roster of seven played this month — read "שחקנים היו פעילים השנה 0". The invariant the two tallies exist to hold is `season ≤ lifetime` for every counter; it holds by construction, because the season branch is nested inside the lifetime one. The mock block at `:1034–1105` used to violate it (`activeThisYear: 28` over `lifetime.activeThisYear: 18`), which is ROUND4 #25, and has been rewritten to derive both from one monotone tally.

`appearances-rollup-undercounts-the-scan` . `mostLoyal` deliberately reads the scan for the running season and the archive's `games` column for a closed one — and the two disagree. Production, side by side:

<code>scan (attendedByUser)</code>	<code>archive seasonSummary/..._s1 (players[].games)</code>
1IdtNEjb... 22	1IdtNEjb... 19 ← <code>mostLoyal</code> winner, value 19
YIZlKWbv... 20	YIZlKWbv... 18
B5KpY04I... 20	B5KpY04I... 18
alsobLSA... 20	alsobLSA... 18
CEkRfDs2... 20	CEkRfDs2... 18
K5rSGB4J... 20	K5rSGB4J... 18
JoLFRxFr... 19	JoLFRxFr... 17

The gap is exactly three evenings — `2SMrlCGHyjnCmFRVthlS` (06.07), `MMtE8J6HCLHpjhHL99A` (16.07) and `FCa5UtSdIb7m3X9rs9og` (04.08) — and I confirmed the cause directly: those three games have an **empty** `finishCredited` **subcollection**, while every other night has one document in it. That marker is the idempotency latch created in the same batch as the `games: increment(1)` writes (`functions/src/index.ts:5895–5915`); no marker means the increment never ran. So the scan is right and the rollup is short. The card for season 1 prints `completedRounds: 22` while the title it awarded, כתר ההתמדה, was decided on 19 — and the eligibility denominator for all nine titles was derived from the same 19 (`seasonRollover.ts:541–544`).

`archived-goals-silently-zero` . Lifetime club goals are assembled on the client as `champ.totalGoals + archivedGoals` (`:508`), where `archivedGoals` is summed from `seasonHistoryService.list` (`:217–219`). `list` returns the string 'error' on failure and the handler is `if (!alive || list === 'error')` `return;` — so a failed read leaves `archivedGoals` at its initial 0 and leaves `pastSeasons` empty, which removes the "כל הזמנים" chip and every past-season chip from the picker. One failed `seasonCards` query silently collapses club goals to this season's (zero, after a close), and removes the only route to the club's history. On this club the damage is currently invisible: 27 archived goals against a 100-goal bronze threshold, and 303 vs 276 club-level points both landing in level 3. On a club with three seasons behind it the same failure is a visible demotion with no error shown.

What a close does to the live tables

`performSeasonClose` writes the archive, then winds the live counters back. `PLAYER_SEASON_FIELDS` (`functions/src/seasonRollover.ts:184–210`) names 21 columns — goals, assists, rounds, wins, losses, ties, games, cleanSheets, ownGoals, the six penalty counters, `csRounds` , `asRounds` , `eveningScoreSum` ,

eveningScoreCount — and every one is subtracted from each communityPlayerStats row. PAIR_SEASON_FIELDS (:220–231) does the same for all ten counters on every communityPairStats row. CLUB_SEASON_FIELDS (:234–242) — rounds , goals , guestGoals , ownGoals , tiedRounds , shootoutRounds , scorelessRounds — is **set to absolute 0** on communityStats/{groupId} (:1004–1010) rather than subtracted, which the module's own comment forbids for the other two.

The result, read live today, three weeks after the 17.09 close:

```
communityStats/HhzIwmjMl1i5HSOGht3p = { goals: 0, rounds: 0, guestGoals: 0,
                                           ownGoals: 0, tiedRounds: 0, ... }
all 7 communityPlayerStats rows       = { goals:0, assists:0, wins:0, games:0, rounds:0,
                                           cleanSheets:0, penTaken:0, ... }
```

Everything downstream of those rows is therefore zero at once. rankChampionshipRows (src/utils/championship.ts:139–147) keeps a row only if keepAll || goals>0 || assists>0 || games>0 || wins>0 , and getCommunityStats's sibling getCommunityChampionship(groupId) is called with no memberIds from the stats screen (gameService.ts:1365 , keepAll=false) — so champ.players comes back **empty**, hasScoring is false, and the hero card, the leaders card and six of the eight fun facts disappear in one paint. The league table at the bottom of the same screen makes its own call **with** memberIds (CommunityChampionship.tsx:90 , keepAll=true) and therefore renders seven rows of zeros: one screen, one function, two different answers to "does this club have players". The banner he.communityStatsSeasonFresh exists precisely to stop a ten-year member reading that as "— מחקו לי הכל" מחקו לי הכל, אז הטבלה עוד ריקה. שום דבר לא נמחק N עונה ("season N has only just begun, so the table is still empty. Nothing was deleted").

How long does it last? Until the club's **next evening finishes with evidence of play**. That single transition (didEveningHappen false→true, functions/src/index.ts:5870–5915) writes games: increment(1) for every non-no-show member, which re-populates the rollup rows and so restores champ.players ; the same evening is stamped s2 by the trigger, which gives the season scan its first totalFinished . For a club that plays once a week that is **up to seven days of a club that looks brand new**, and for a club that has stopped playing it is permanent. Goals, assists, wins and mini-games do not come back with it — those are written only by the advanced-mode mini-game commit path, so the common timer-only club stays on goals: 0 for the whole season, and hasScoring (:564) never becomes true. The "all time" scope is the only place where the club's real totals survive, and it is offered only once pastSeasons.length > 0 (:625).

mergeAllTime (src/utils/allTimeTable.ts) is the addition: the live slice first, then every archive, summing thirteen count columns (SUMMED , :35–49) per uid and merging names first-wins so a current member keeps their live name over a frozen copy. Its one piece of care is mergeCoverage (:63–69): csRounds and asRounds are summed **only when both sides have one**, because absent means "never measured" and adding a present one to an absent one would invent coverage and deflate the rate. Everything else is a plain n(prev[f]) + n(row[f]) with n(undefined) = 0 , so an older archive missing a column contributes a confident zero — the older-archive-degrades-into-plausible-zeros finding, reached through the merge rather than through the reader.

The personal season summary

`SeasonSummaryScreen` (`src/screens/profile/SeasonSummaryScreen.tsx` , 951 lines) is where the end-of-season push lands every player who took part. It is titled "סיכום העונה" (*the season summary*) and it answers exactly one question: what happened to **me** between the season opening and its close. Nothing on it is cumulative — the footnote says so in as many words (`he.seasonSummaryFootnote` : "*all the numbers here are for this season only; the cumulative statistics are in the profile*").

Three files do the work. `src/services/seasonSummaryService.ts` (636 lines) fetches; `src/utils/seasonPersonal.ts` (427 lines) computes, and is pure; the screen and `src/components/summary/SeasonShareCard.tsx` (348 lines) render. The service is the only place that knows whether the numbers came from a live collection or from a frozen archive — `buildPersonalSeason` takes one shape either way, and the screen never learns which it got.

Two load paths

`seasonSummaryService.load()` (:416–539) takes `{ groupId, userId, seasonId? }`. The `seasonId` is omitted for "the season running now" and set for a specific one.

Path A — the sealed archive. If `seasonId` is given, the very first read is the archive (:432–446):

```
if (seasonId) {
  const archived = await getDoc(doc(db, 'seasonSummary', `${groupId}__${seasonId}`));
  if (archived.exists()) {
    return fromArchive(archived.data() as Record<string, unknown>, {
      groupId, userId, seasonId,
      seasons: await readSeasons(db, groupId), // may be undefined
    });
  }
}
```

Archive-first is deliberate and load-bearing: a player who has since **left the club** cannot read `/groups/{groupId}` at all, but the push about a season they played still points here. `readSeasons` (:322–335) swallows its own permission error and returns `undefined`, in which case `available` degrades to a one-entry picker holding only this season (:405–407). Everything else — the club's name, the final table, the pair counters, the titles — comes out of the one document. Cost: **two document reads**, one of which is optional.

Path B — the live tables. Falling through means the running season. It reads the club document for `seasons` (bailing with `null` if `seasons.enabled` is false), then three queries in parallel (:465–481):

```
const [statRows, asA, asB] = await Promise.all([
  getDocs(query(collection(db, 'communityPlayerStats'), where('groupId', '=', groupId))),
  getDocs(query(collection(db, 'communityPairStats'), where('groupId', '=', groupId), where('a', '=', uid))),
  getDocs(query(collection(db, 'communityPairStats'), where('groupId', '=', groupId), where('b', '=', uid))),
]);
```

The pair collection is stored once per unordered pair under a sorted key, so "my pairs" is the union of the rows where I am `a` and the rows where I am `b`. On the one club that runs seasons (`HhzIwmjMl1i5HSOGHt3p`, מועדון שזכחת שושי) the collection holds **314 pair documents**, and for `1IdtNEjbEXfiRSqvLrJVn99Nsfi2` (מתן לוי, whose uid sorts first) the `a` query returns 39 and the `b` query 0. So one open of the live path costs 1 group doc + 7 stat docs + 39 pair docs + up to 6 `/users` reads $\approx$ **50 document reads, repeated on every pull-to-refresh**, against 2 for the archive. Many of those 39 pair rows are `quest:...` partners that `orient()` discards a moment later.

What differs between the two paths

	Archive (closed)	Live (running)
Player rows	<code>d.players</code> map, uid $\rightarrow$ row	<code>communityPlayerStats</code> query
Pair rows	<code>d.pairs</code> map	two <code>communityPairStats</code> queries
<code>completedRounds</code>	<code>d.completedRounds</code> (:382)	<code>seasons.playedRounds</code> (:512)
<code>groupName</code>	frozen in the archive	live <code>groups.name</code>
<code>endsAt</code>	<code>d.endsAt</code>	<code>null</code>
<code>roundsCadence</code>	never set	set iff <code>cadence.type === 'rounds'</code> (:515–523)
<code>myTitles</code> / <code>seasonTitles</code>	from <code>d.awards</code>	<code>always []</code> (:526–527)
Peer names	frozen <code>displayName</code> only	<code>/users</code> fetch
<code>peerAvatars</code>	<code>always {}</code> (:404)	avatar from the same <code>/users</code> fetch
Reads	2	$\sim$ 50

Two consequences follow from that table and both are visible to users. Titles do not exist for a running season by design — they are decided when the numbers stop — so the "תארים בעונה" card is gated on `me.hasData && model.closed` (screen :476). And the archive path deliberately makes no `/users` read, so **every peer row on a closed season draws the deterministic fallback disc**. That is the documented price of a summary that still renders after somebody deletes their account (:400–403), not a bug; the bug (`peer-rows-always-show-the-generic-auto-avatar`, P3) was on the *live* path, where the user document was already being fetched for the name and the avatar thrown away. `resolvePeers` (:277–309) now keeps both — but note its `frozen` parameter is dead: the only call site passes `new Map()` (:496), because the archive path resolves its own names inline at :395–399.

Every number on the screen

`buildPersonalSeason` (`seasonPersonal.ts:307–427`) reads **one row** — `players.find(r => r.userId === me)` — and derives the rest. Absent is zero throughout (`num()` , `:147–148`), which is the root of `older–archive–degrades–into–plausible–zeros` (P2): a season sealed before a counter existed renders a confident `0` rather than a dash. Only `csRounds / asRounds` are allowed to be absent, and `playerRow` (`seasonSummaryService.ts:164–165`) preserves that distinction on purpose.

Field	Derivation	Rendered as (he.*)
<code>goals / assists</code>	row fields	שערים / בישולים, lead tiles
<code>contributions</code>	<code>goals + assists</code> (<code>:362</code>)	"שערים + בישולים"
<code>rounds</code>	row <code>rounds</code> — mini-games (משחקונים)	"משחקונים"
<code>evenings</code>	row <code>games</code> — evenings (מחזורים)	"ערבי משחק"
<code>wins / losses / ties</code>	row fields	ניצחונות / הפסדים / תיקו
<code>winPct</code>	<code>wins / rounds</code> (<code>:336</code> , <code>:368</code>)	"אחוז ניצחון"
<code>cleanSheets</code>	row field	"שערים נקיים"
<code>cleanSheetPct</code>	<code>cleanSheets / csRounds</code> (<code>:340</code> , <code>:370</code>)	"אחוז שער נקי"
<code>goalsPerRound</code>	<code>goals / rounds</code> (<code>:376</code>)	"שערים למשחקון"
<code>assistsPerRound</code>	<code>assists / **asRounds**</code> (<code>:377</code>)	"בישולים למשחקון"
<code>ownGoals</code> , <code>penTaken/Scored/Faced/Save d</code>	row fields; tiles hidden at 0 (screen <code>:535–553</code>)	
<code>partialCoverage</code>	<code>rounds > 0 && (csRounds < rounds \ \ asRounds < rounds)</code> (<code>:378</code>)	the note at <code>:564–566</code>
<code>hasData</code>	<code>rounds > 0 \ \ games > 0</code> (<code>:359</code>)	gates the personal half

`rate()` (`:155–157`) returns `null` , **never 0** , for a zero denominator — "an unknown rate is not a zero one" — and the screen prints – for null (`pct / per` , screen `:53–55`).

The three denominators are three different numbers, and that is intentional. `winPct` divides by `rounds` ; `cleanSheetPct` by `csRounds` ; `assistsPerRound` by `asRounds` . Clean sheets have only been recorded since 17.08 and assists since 21.06, both long after clubs started counting mini-games, so for any veteran the three tiles in one grid genuinely cannot be reconciled by arithmetic. The production archive proves it: מתן לוי has `rounds: 26` , `csRounds: 15` , `asRounds: 24` . His grid reads 80% אחוז שער נקי 12 (12/15, not 12/26 = 46%), 1 (1/24) בישולים למשחקון 0.04. `partialCoverage` is true for **all seven players** of s1 — every one has `csRounds < rounds` — so the explanatory note (`he.seasonPartialCoverageNote`) renders for the whole club.

Two honest criticisms of this design. First, `winPct = wins / rounds` counts ties in the denominator, which is the weaker statistic; the code says so at :327–335 and keeps it only because the club's efficiency table has shipped for months computing the same thing under the same Hebrew label. Second, `assistsPerRound` dividing by `asRounds` is the fix for `assists-per-round-denominator-disagrees-with-club-efficiency-tab` (P2) — the shipped binary divides by `rounds` (see the last sub-section), which gave four of the seven s1 players two different "בישולים למשחקון" one tap apart. **There is no test for it.**

`tests/logic/seasonPersonal.test.ts` (411 lines, 30 cases) has a case for the clean-sheet denominator and not one mention of `asRounds`, `assistsPerRound` or `partialCoverage`.

WORKED EXAMPLE — מתן לוי, SEASON 1, FROM THE LIVE ARCHIVE BYTES

```
row: { goals:2, assists:1, rounds:26, games:19, wins:11, losses:9, ties:6,
      cleanSheets:12, csRounds:15, asRounds:24, penTaken:1, penScored:0,
      penFaced:1, penSaved:1, penMissed:1, ownGoals:0 }
```

renders: שערם 2 · בישולים 1 · שערים+בישולים 3 · ערבי משחק 19 · משחקונים 26 · ניצחונות 11 · הפסדים 9 · תיקו 6 · אחוז ניצחון שערם 42% · שערים נקיים 12 · אחוז שער נקי 80% · שערים למשחקון 0.08 · בישולים למשחקון 0.04 · פנדלים שהובקעו 0/1 · פנדלים שנעצרו 1/1. Note $11 + 9 + 6 = 26 = \text{rounds}$ exactly — the win/loss/tie triple does close over the mini-game count on all seven rows, which is the one invariant here that demonstrably holds.

The three ranks

`rankOf` (:258–297) answers "what position does the club table put me in", not "how many are ahead of me". Those differ whenever ties are shared, and the difference was user-reported ("המיקום שלי ... לא נכון").

```
const played = rows.filter((r) => (num(r.rounds) > 0 || num(r.games) > 0) && isReal(r.userId));
const order = [...played].sort((a, b) =>
  pick(b) - pick(a) ||
  num(b.wins) - num(a.wins) || num(b.goals) - num(a.goals) ||
  num(b.assists) - num(a.assists) || a.userId.localeCompare(b.userId));
const i = order.findIndex((r) => r.userId === me);
const anyValue = order.some((r) => pick(r) > 0);
return { rank: i < 0 || !anyValue ? null : i + 1, of: order.length };
```

Three things to read out of it. `of` is "who turned up" — `rounds > 0 || games > 0`, the same test `hasData` makes, and `isReal` (:152) drops `guest`: identities, who are a different person each week. That `|| games > 0` is the fix for `rank-denominator-of-excludes-attendees` (P1): the shipped code filters on `rounds > 0` alone, so a timer-only club — which records **zero** mini-games, ever — gives every player `of: 0` and dashes out seven of the thirteen tiles while the club table one tap away lists everybody. 23 of the 93 `communityPlayerStats` rows in production have `games > 0` and `rounds === 0`; those are exactly the players the shipped denominator erases.

The `anyValue` guard is the fix for a regression the widening itself caused: with `of` now counting attendees, a column in which every player has 0 still produced a sort, which fell all the way through to `userId.localeCompare` and handed three players a **gold medal** for "מקום 1 מתוך 15 בשערים" in a season with no

goals in it. A null rank draws no medal (`rankTint / rankIcon` , screen :119–125) and prints `–` . It fires only when the *whole* column is zero, though: a player with 0 assists in a column where somebody has 5 is still given an ordinal position rather than a blank.

The tie-break mirrors the club table's comparator — and this is where the open P0 lives. `rankOf` breaks ties on wins → goals → assists → uid. `CommunityStatsTable`

(`src/components/community/CommunityStatsTable.tsx:153–156`) sorts on **one key with no tie-break at all**:

```
return [...effPlayers]
  .sort((a, b) => Number(b[sortKey as keyof ChampionshipRow] ?? 0) -
    Number(a[sortKey as keyof ChampionshipRow] ?? 0))
  .slice(0, limit);
```

V8's sort is stable, so ties keep the *incoming* order. For a sealed season that order is whatever

`parseSeasonTable` produced, and at HEAD that is `players.sort((a,b) =>`

`a.uid.localeCompare(b.uid))` (`src/utils/seasonArchive.ts:114`) — a uid sort. So the two surfaces

break ties on **different** keys, and the production archive contains a case where they disagree. Assists in s1 are

`{Nofar 5, Lioz 3, Linoy 2, 1 הלין ,1 איציק ,1 מתן , Eliran 0}` . `rankOf` breaks the three-way tie at 1

on wins first (הלין has 12, the other two 11) and reports **4 הלין**, **5 מתן**, **6 איציק**. The club table, sorting stably

over a uid-ordered input (`1Idt...` , `JoLF...` , `also...`), lists **4 מתן**, **5 איציק**, **6 הלין**. Same document, same

season, two screens, two answers — and the summary's answer is the one that decides whether a bronze medal is drawn and whether the share card prints a placing.

`sealed-season-rank-vs-club-table-nondeterministic` (P0) is worse than that in the **shipped** binary,

where `seasonArchive.ts` has no sort at all and the incoming order is Firestore's map iteration order: two

consecutive reads of the immutable s1 archive returned different key orders, so the silver ring on the club table

moves between refreshes. The HEAD uid sort makes the table *deterministic*; it does not make it *agree*.

The six peer rows

All six come from `communityPairStats` , which the round rollup has been filling per mini-game all along and which the same rollover zeroes — so it is season-scoped for free, with no new counter.

Every read goes through `orient()` (:181–201), because a pair document is stored once under a sorted key and which slot I occupy is an accident of my user id:

```
const meIsA = row.a === me;
...
myWins:      meIsA ? num(row.winsA)      : num(row.winsB),
theirWins:   meIsA ? num(row.winsB)      : num(row.winsA),
iAssistedThem: meIsA ? num(row.assistsAToB) : num(row.assistsBToA),
theyAssistedMe: meIsA ? num(row.assistsBToA) : num(row.assistsAToB),
```

Getting that backwards is invisible — the screen still renders, it just names the wrong person as the one you beat all season. `orient` also returns `null` for a guest partner and for a malformed self-pair, which would otherwise make every player their own best teammate. It was verified over all 21 production pairs and is sound.

`best()` (:211–241) picks the maximum by `pick`, **excluding zero** (if (count <= 0) continue), tie-breaking on a per-row `tiebreak` and then on `o.other < winner.other` — a total order, so the same season always names the same person. The six rows:

Row	Hebrew label	pick	tie-break	detail line shows
partner	הכי הרבה יחד באותה קבוצה	sameTeam	winsTogether	count + winsTogether
nemesis	היריב הכי גדול	against	theirWins	count, myWins vs theirWins
victim	מי הובס הכי הרבה	myWins	against	count only
tormentor	מי ניצח הכי הרבה	theirWins	against	count only
assistedMost	למי הכי הרבה בישולים	iAssistedThem	sameTeam	count
assistedBy	ממי הכי הרבה בישולים	theyAssistedMe	sameTeam	count

`partner`'s `winsTogether` tie-break is deliberate — of two people you played 10 mini-games beside, the one you kept winning with is the better story. `nemesis` leans the other way, towards whoever beat you more.

De-duplication (:402–423). The three rival rows were picked independently, and in a club of regulars the person you face most is usually also the one you beat most and lose to most. On s1 that produced the same name and the same face three times for **5 of the 7 players**, and one of them saw the same person in all four rows he had (`nemesis-victim-tormentor-not-deduplicated`, P1). The fix runs the three picks in order behind a `taken` set:

```
const peer = best(oriented.filter((o) => !taken.has(o.other)), pick, tiebreak)
    ?? best(oriented, pick, tiebreak);
if (peer) taken.add(peer.userId);
```

The fallback to the un-filtered `best` means a player with a single opponent still gets three true rows rather than two blanks.

⚠ **The de-duplication makes two of the three labels false.** Rows 2 and 3 no longer name the maximum — they name the best remaining candidate — while the Hebrew label still claims a superlative. Production, Linoy Levi (CEkRfDs20x...), s1, HEAD code:

- `nemesis` → Lioz Madar, detail "משחקונים זה מול זה — 3 ניצחונות מול 4" (he beat her 4 times);
- `tormentor`, labelled "מי ניצח הכי הרבה" (*who beat me the most*) → מתן לוי, detail "הפסדים 2".

Lioz beat her 4, מתן beat her 2, and both numbers are printed on the same card two rows apart. The old bug produced a repetitive card; the new one produces a self-contradicting one. Neither the label nor the detail line carries any hedge, and the test suite's two dedup cases (:353 , :368) assert only that the rows name different people, never that the row is still true.

△ **The assist rows describe half the season.** `assistsAToB + assistsBToA` summed over all 21 archived pairs is 7. `totals.assists` on the same document is 13, and the seven player rows also sum to 13. So six of the season's thirteen assists are attributed to no pair at all (`assist-peers-account-for-half-the-season-assists` , P2). The visible consequence: Nofar Tzabari won כתר הבישולים with `value: 5` , and one card below, her "למי הכי הרבה בישולים" row reads **אִיצִיק לוי · 2 בישולים**. Three of her five assists are not in the pair data. The legacy undirected `assists` field on the same pair rows sums to 8 — a third number — which is why `archive-duo-reader-has-no-floor` matters: the duo title reads the directional fields and the live scope reads the legacy one.

When both `partner` and `nemesis` are null the card prints `he.seasonPeersEmpty` (screen :658–662), whose copy now names the cause — *"who played with whom is counted only in evenings run from the advanced live screen"* — rather than the old "it will fill itself up", which is a promise a timer-only club can never keep (`peers-empty-promises-it-will-fill` , P2).

Titles, champions, and the "did not play" case

`titlesFor` (:195–210) walks the nine `SEASON_TITLE_KEYS` and keeps the ones whose `winners` array contains me, computing `sharedWith = holders.length - 1`. The duo title is keyed `a__b` , so membership is tested against `w.split('__')` . `allTitlesOf` (:219–243) does the same for the whole club, resolving each winner through the archive's `frozen displayName map` ('-' when absent), and flags `mine` .

s1's awards, live: `topScorer` 10 הֶלֶן צָבָרִי, `topAssister` Nofar 5, `topWinner` Lioz 13, `mostLoyal` 19 מתן, `cleanSheetKing` 13 הֶלֶן, `penaltyKing` 1.0 הֶלֶן, `penaltyKeeper` 1.0 הֶלֶן, `deadlyDuo` null, `mvp` **all seven players** at 6. So the champions card lists eight rows (the null duo is skipped at :228) and every single player holds כתר העונה with `sharedWith: 6` .

The `hasData` split (screen :463–512) is the fix for `did-not-play-told-the-season-had-no-games` (P2). One ternary used to gate both halves, so a player who missed the season got a single card reading *"לא שיחקה בעונה הזאת"* **and lost the champions list with it** — the list that is identical for every reader, is the reason the close push sends the whole club here, and was fetched and thrown away on the way to the render. Now the personal cards stay behind `me.hasData` and the club's do not; the empty line (:491–499) explains the gap and the champions render underneath it. The two strings are distinct: `seasonSummaryNoRounds` ("you haven't played this season yet, so there's nothing to summarise") for a running season, `seasonSummaryNoRoundsClosed` ("you didn't play this season, so you have no summary from it") for a closed one.

`completedRounds` **is the season's EVENINGS**, and it is the note under the standing card: `he.seasonClubRounds(22)` → "מחזורים שוחקו במועדון בעונה הזאת. המיקום מחושב מול מי ששיחק בה 22". The archive has carried `completedRounds: 22` since the archive existed and it was **read by nothing** (`archive-completedrounds-never-read-by-client` , P2); the shipped client substitutes `totals.rounds` , the mini-game count, so the same sealed season reads 22 on the history screen and 37 here, and a timer-only club reads "משחקים שוחקו 0" for a season it played every week of.

SeasonShareCard — what leaves the app as a PNG

The card is rendered off-screen at a fixed `SHARE_CARD_WIDTH = 340` (:30) inside a positioned stage (screen :691–696 ; positioned rather than hidden, because a `display:none` subtree has no layout and captures blank), and `onShare` (screen :330–362) turns it into a PNG with `captureRef` and hands it to `expo-sharing`. `allowFontScaling={false}` is set on every text node: the capture happens before anyone can see it, so at Android "Largest" a 100% used to grow out of its tile and a 30-character name clipped mid-word.

Its contents, three tiles to a row (`PER_ROW = 3`):

```
const mini = me.rounds > 0;
const tiles = [
  { value: String(me.goals),    label: he.statGoals },          // שערים
  { value: String(me.assists),  label: he.statAssists },        // בישולים
  { value: String(me.evenings), label: he.seasonStatEvenings }, // ערבי משחק
  ...(mini ? [ rounds, winPct, cleanSheets, wins, losses, ties ] : []),
];
```

`evenings` first and never conditional; everything after it is a mini-game counter, printed only by a club that has any. That is the fix for `share-card-is-all-zeros-for-a-timer-only-club` (P1): the shipped card prints all eight mini-game numbers unconditionally and omits `evenings` entirely, so 23 of the 93 production stat rows produce a card of eight zeros and a dash to send to the group chat, while the screen behind it says they turned up fourteen times. Even at HEAD a timer-only player's card is `0 · 0 · N` — better, but two of the three headline numbers are still zero.

The rank gate (:95–99): `bestRank` is the numerically lowest of the three ranks, and

```
const worthShowing = !!bestRank && me.ranks.of > 1 &&
  ((bestRank.rank <= 3 && me.ranks.of >= MIN_CLUB_FOR_PODIUM) || bestRank.rank / me.ranks.of <= 0
```

`MIN_CLUB_FOR_PODIUM = 8`. The shipped gate is `bestRank.rank <= 3 || rank/of <= 0.34` with no club-size floor, which printed "3 מתוך 3" — last place — in bold on a card about to be sent (`share-card-rank-gate-still-prints-last-place`, P2). One side-effect of the floor is worth naming: on a 7-player club, the podium arm can never fire, so only ranks 1 and 2 (0.14 and 0.29) clear the top-third arm. לוי's best placing in s1 is **3rd in wins of 7** — 0.43 — so his card carries no placing line at all.

Titles on the card print `{name} · {value}` plus, now, `· במשותף עם עוד N` (209–211:) שחקנים. `share-card-claims-a-shared-title-as-its-own` (P1) is that `sharedWith` was computed all along and dropped here: in the shipped binary all seven holders of s1's כתר העונה send the same exclusive-looking "כתר העונה · ציון 6.0" into the same group chat within a minute of each other.

Two peer lines — partner and nemesis only (:220–235), using `he.seasonSharePartner` /

`seasonShareNemesis`, which were rewritten to say משחקונים rather than משחקים (the card prints "40 משחקונים" above, and calling the same unit two things is `share-card-names-one-unit-two-ways`, P3) and to drop the parentheses, because bidi resolves a closing paren after a Latin name to RTL and mirrors it — four of the seven frozen s1 names are Latin, and the PNG that left the app read "(משחקונים 6)".

A complete HEAD card for מתן לוי · s1: brand row · 1 עונה · pill · his name · שושי · מועדון שכחת שושי · tiles 2 / 1 / 19
then 26 / 42% / 12 then 11 / 9 / 6 · no placing line · במשותף · 6.0 ציון · כתר העונה · 19 מחזוריים · כתר העונה · 19 מחזוריים ·
משחקונים · היריב הגדול: איציק לוי · 10 משחקונים · 10 Lioz Madar · עם עוד 6 משחקונים · הכי הרבה יחד

⚠ None of the fixes above is in a shipped binary

Both 1.1.9 store binaries (Android vc235, iOS build 103) were built from 4de80d0 . Every fix described in this section landed after it and exists only in git. `git diff --stat 4de80d0 HEAD : seasonPersonal.ts`
`+92/-10, seasonSummaryService.ts +88/-21, SeasonShareCard.tsx +97/-29`. What users are actually running today:

```
// 4de80d0:src/utils/seasonPersonal.ts
:246 const played = rows.filter((r) => num(r.rounds) > 0 && isReal(r.userId)); // no `|| games`
:331 assistsPerRound: rate(assists, rounds), // not asRounds
:339 nemesis: best(oriented, (o) => o.against, (o) => o.theirWins), // no dedup
:340 victim: best(oriented, (o) => o.myWins, (o) => o.against),
:341 tormentor: best(oriented, (o) => o.theirWins, (o) => o.against),
// 4de80d0:src/services/seasonSummaryService.ts
:341 completedRounds: num(totals.rounds), // 37, not the archive's 22
:461 completedRounds: num(clubSnap.data()?.rounds), // an extra read of communityStats
```

Read against production s1 that means the live app tells מתן לוי his three rivals are all איציק לוי — including "מי
ניצח הכי הרבה: איציק לוי" about a man he beat 5–3 — divides his one assist by 26 instead of 24, and captions the
club's season "37" where the hall of fame says "22". An expert reading HEAD is reading a version no user has.

The screens, and what each one reads

⚠ Before you compare this to the app on your phone

Almost nothing described below is in a store binary. Both 1.1.9 artifacts — Android versionCode 235, iOS build 103 — were built from commit `4de80d0` ("1.1.9", 2026-09-18 08:54 +0300). Every seasons UI commit landed after it:

e35394a	21:40	The last six labels that still called a משחק a משחקון
bfc60db	21:34	(availability, unrelated)
e89b926	21:19	1.1.10 – and the date range that was never actually fixed
8235851	21:14	Round 4: the blast radius...
5531a21	19:02	Seasons: the rest of the audit, and the six regressions the fixes caused
bc6a562	17:03	Seasons: the numbers people actually read
ae2c042	16:53	Seasons: the six defects that made the feature unusable
8a3b9ef	14:34	The hall of fame becomes a ceremony
1672d6b	09:01	A picker needs something to pick
4de80d0	08:54	1.1.9 ← both store binaries are HERE

`git diff --stat 4de80d0 HEAD -- src/` is 27 files, +3,240 / –467. Five files that this section describes **do not exist at all** in the shipped binary: `src/Utils/seasonCardVariant.ts`, `src/Utils/seasonMedalTier.ts`, `src/Utils/seasonScope.ts`, `src/components/community/SeasonMedal.tsx`, `src/components/community/SeasonPoster.tsx`. `app.json` now says `"version": "1.1.10"` and no build of it exists. Firestore `appConfig/android` says `latestVersion: "1.1.9"`; `appConfig/ios` says `"1.1.7"`. This is ROUND4 finding #1, and it colours everything here: where a defect below is marked *shipped*, a user can see it today; where it is marked *HEAD only*, it is a defect waiting for a build.

The shipped hall of fame is a different screen: 231 lines, a plain white card per season with a title row, a date line, `he.seasonHistoryLine(22, 37, 7) → "22 משחקונים · 37 משחקונים · 7 שחקנים"`, and nine flat rows. No poster, no medals, no tiers, no streak badges, no void ribbon, no variant. Its titles are still מלך (`topScorer: 'השערים' מלך`, `mvp: 'שחקן העונה'` at `4de80d0:src/i18n/he.ts:2148–2156`); HEAD renamed six of them to כתר.

The one club with real data

Everything below is worked against `HhzIwmjMl1i5HSOGht3p` (מועדון שכחת שושי), the only club that has ever run a season. Read live:

source	value
<code>groups/.../seasons</code>	<code>{enabled:true, currentId:'s2', currentNo:2, count:1, playedRounds:0, roundsAtStart:10, startedAt:1789679141952, reopenedAt:1789727847680, cadence:{type:'rounds', targetRounds:24}}</code>

source	value
seasonCard s/...s1	{no:1, startsAt:1789547794676 (2026-09-16T08:36Z), endsAt:1789661526768 (2026-09-17T16:12Z), completedRounds:22, totals:{rounds:37, goals:27, assists:13}, players:7, winners:[8]} — no endedEarly, no partialData, no awardsDenominator
seasonSummary/...s1	7 player rows, games ∈ {19,18,18,18,18,18,17}, awards.deadlyDuo: null, originalTarget.targetRounds: 22, roundsAtStartOfSeason: 7
all 23 terminal games	3 stamped s1, 1 stamped s3 (an orphan — the club has never had an s3), 19 unstamped, 0 stamped s2
all 7 communityP layerStats	every counter 0 — the close zeroed them

The decision functions

seasonCardVariant(s) — src/utils/seasonCardVariant.ts:25-57. Sorts a finished season into 'full' | 'noTitles' | 'void':

```
const nothingRecorded =
  s.players === 0 && s.winners.length === 0 && s.completedRounds === 0 &&
  s.totals.rounds === 0 && s.totals.goals === 0 && s.totals.assists === 0;
if (nothingRecorded) return 'void';
return s.winners.length === 0 ? 'noTitles' : 'full';
```

It is deliberately **not** keyed on totals.rounds alone (mini-games are advanced-mode only) and no longer on endsAt - startsAt — on s1 that span is 31.6 hours for three months of football, so a 48h "void" window would have swallowed the club's only real season. Ledger finding *variant-tests-pass-against-the-exact-regression-the-file-warns-about* stands: all five cases in tests/logic/seasonCardVariant.test.ts also pass against an implementation keyed on totals.rounds, so the test suite does not defend the property the file's comment is about.

heroSeasonId(list) — :65-70. The season the list opens on and the only one that gets confetti: first 'full', else first 'noTitles', else null. For this club → 's1'.

heroWinner(s) — :79-101. HERO_ORDER = ['mvp', 'topScorer', 'topWinner'], take the first whose names.length ≤ 2; otherwise fall back to the title with the **fewest** holders. On s1, mvp has seven names, so it is skipped and topScorer (10, הלן צברי) becomes the poster subject.

medalTier(key, value, completedRounds) — src/utils/seasonMedalTier.ts:103-123, with per-title scales at :57-67. Counting titles have their own three-stop steps; the two penalty titles use RATE_STEPS = [0.7, 0.85, 1]; mvp uses MVP_STEPS = [6.6, 7.6, 8.8]; mostLoyal divides by the season's own length against [0.6, 0.8, 1]. step() compares with ≥.

`titleStreak(seasons, index, key)` — :141–177 . Walks towards older seasons while the season numbers stay consecutive and the holder set is identical. **It compares by NAME**, because a `SeasonWinner` is `{key, names, value}` and no uid survives the archive (the file says so). A player who renames themselves silently ends their own streak, and nothing can distinguish that from a new holder. The names are joined on `after trim()` and `sort()` , which closes the older `['ג ב', 'א']` vs `['ג', 'ב א']` collision.

SeasonsCard — the club screen

`src/components/community/SeasonsCard.tsx` , mounted at `CommunityDetailsScreen.tsx:902` with `{groupId, seasons: group.seasons, isMember: isMember || isAdmin}` . It reads **only** the `seasons` block of the group document already in memory — no query, no read.

Three states:

- 1. `!isMember` → `null` (:55). Every collection behind its buttons is membership-gated.
- 2. `seasons.enabled !== true` and `count > 0` (:62–77) → a stub card: `he.seasonsOffButArchived` plus the "עונות קודמות ותארים" button. `enabled !== true` and `count === 0` → `null` .
- 3. Running (:132–187).

The running card's fields:

pixel	source
2 עונות • עונה	<code>he.seasonsCardTitle</code> , <code>he.seasonNumberLabel(seasons.currentNo ?? 1)</code> (:137)
ⓘ	<code>he.seasonsCardInfo</code> (:143) — the only place in the app that defines מחזור for the reader
big number 0	<code>seasons.playedRounds ?? 0</code> (:151) — rounds cadence only
מתוך 24 מחזורים	<code>he.seasonsCardOfTarget(cadence.targetRounds)</code> (:153)
progress bar	<code>playedRounds / targetRounds</code> , <code>clamped</code> (:118–127); turns <code>colors.success</code> at ≥ 1
נשארו 24 מחזורים	<code>he.seasonsCardRemaining(max(0, target - played))</code> (:87)
days-left line	date cadence only, from <code>endsOn</code> or <code>endsAt</code> (:104–113)
history button	shown iff <code>seasons.count > 0</code> (:177)

For a date-cadence season, `progress` is derived from `startedAt` and `daysLeft` rather than from the cadence (:123–126), so it is only as honest as `startedAt` — and `startedAt` is stamped when seasons are **switched on**, not at the season's first evening. On this club `startedAt = 1789679141952 = 2026-09-17T21:05Z`, five hours after s1 closed and a day before the reopen at `2026-09-18T10:37Z`.

Today this card renders `0 / מתוך 24 מחזורים` , an empty bar and `נשארו 24 מחזורים` — which is correct for s2 and sits one tap away from a stats screen that will tell you the club has played 22 evenings. The ledger's *card-promises-an-evening-of-grace-that-is-an-hour* is fixed: the at-target string is now `העונה הגיעה ליעד ותיסגר` ' .
`מעצמה בשעה הקרובה` .

SeasonsSettings — the admin controls

`src/components/community/SeasonsSettings.tsx`, 1,150 lines, mounted as `extraAdvanced` inside the club-edit form (`CommunityEditScreen.tsx:181`). It is the only surface that writes anything.

State seeding (:230–306) is all derived from the club document: `live = seasons.enabled === true` ; `cadence` from `seasons.cadence.type` ; `months / rounds` from the stored target **whenever valid**, not only when it matches a chip (:253–262); `customMonths / customRounds` open pre-expanded when the stored value is off-preset. `firstTime = (count ?? 0) === 0 && !live` (:309). `thisSeasonNo = live ? currentNo : count + 1` (:313) — numbering continues across a disable/enable, which is why every string here is parameterised.

`history` (:314–355) is the number every validation and every confirmation line is measured against, and it comes from **two different places**:

```
if (!firstTime) {
  setHistory(live ? Math.max(0, seasons?.playedRounds ?? 0) : 0);
  return;
}
// firstTime only:
gameService.getCommunityStats(groupId).then(st => setHistory(st?.totalFinished))
```

For a live club that is `seasons.playedRounds` — the mirror, `0` today. For a first activation it is the **unscoped** 200-document evening scan — 22 for this club. A failed read sets `historyFailed` and leaves `history` at `null` rather than `0`, and every button below is disabled while it is `null` (:944–947); `0` would have told the confirmation sheet the club had never played, one press before the server sealed 22 evenings.

`plan` (:419–441) is one `planActivation(...)` object that drives the red line, the button's enabled state and every row of the sheet. `planErrorText` (:171–195) maps its six refusal codes onto Hebrew that names the actual season number — `he.seasonsErrSeasonEndOf(no)` etc. The ledger's P0 *live-club-cannot-use-date-cadence* is fixed at HEAD by `hasHistory` (`seasonActivation.ts:160–171`), which derives the end date itself instead of demanding a control that only renders under `firstTime` ; ROUND4 #21 (that branch returning `noNextStartsOn`) is fixed in the same place. **Both fixes are HEAD only** — a club on season 2 running 1.1.9 still cannot choose a date cadence.

Controls, top to bottom: the toggle with its `InfoTip` (:637–667); `he.seasonsCadenceQuestion` and two chips; length chips `MONTH_CHOICES = [3,6,12]` / `ROUND_CHOICES = [24,48,96]` plus a `מותאם אישית` chip opening a `Stepper` (`ROUND_STEP` is now `1`, closing the ledger's *rounds-stepper-lattice*); a three-line date preview for a non-first-time date club (:770–790); the `firstTime`-only history block with `לצרך לעונה` / `הנוכחית / לסגור ולהתחיל מאפס` and, under `cadence==='date' && !sealHistory`, the season-1 end-date chips built from `SEASON1_MONTH_CHOICES = [1,2,3,6]`.

Three destructive buttons:

- `הפעל עונות` opens `SeasonConfirmSheet` (:990–1010); only the sheet's own button calls `seasonService.enable`.

- `saveTarget` (:513–523) is a **bare one-tap write** — no sheet, no confirmation. `targetChanged` (:401–408) compares only `c.months !== months` for a date club, so the ledger's *date-season-cannot-be-renewed-at-the-same-length* still holds: a date club that wants the same length again cannot press the button at all.
- `endNow` (:590–620) builds its body from `seasons.playedRounds`. For this club today that is `he.seasonsConfirmSealedNow(0)` → "עונה 2 · 0 מחזורים קיימים – תיסגר כעת", followed by `he.seasonsEndConfirmBody`. The server's own comment on `endSeasonNow` claims this client shows the admin which titles are about to be awarded; it does not — the titles are not computed anywhere this component can reach.
- `reopenLast` (:525–588) now reads the season's own card **before** opening the dialog, and only warns about an immediate re-close when `!last.endedEarly || last.completedRounds >= target`. It is gated on `live && count > 0` (:979), which closes the ledger's case of it sitting next to "הפעל עונות" on a seasons-off club. It still sits directly beneath the red end-season button, same size, same shape.

`SeasonConfirmSheet` (`src/components/community/SeasonConfirmSheet.tsx`) renders one `Row` per present field of the plan, so a plan missing a field silently drops a line — which is exactly how `ROUND4 #21` manifested.

CommunityStatsScreen and its scope picker

`src/screens/communities/CommunityStatsScreen.tsx`, 1,439 lines. `type Scope = {k:'current'} | {k:'all'} | {k:'season', id} (:60).`

The load order matters (:182–197): the **group document is read first**, because the evening scan has to know which season it is counting.

```
const scope = g?.seasons?.enabled && g.seasons.currentId
  ? { currentId: g.seasons.currentId, currentNo: g.seasons.currentNo ?? 1 }
  : undefined;
gameService.getCommunityStats(groupId, scope)
```

`inSeason` (`src/utils/seasonScope.ts:28–36`) is `stamp ? stamp === season.currentId : season.currentNo === 1`. On this club `currentNo` is 2 and **no game carries** `s2`, so the season-scoped half of `getCommunityStats` returns `totalFinished 0`, `organizationRate 0`, `activeThisYear 0`, `longestStreak 0`, `topPlayers []`, while the lifetime block returns `22 / 96% / 7 / streak 22` held by `1IdtNEjb...`.

Which of the two each pixel reads, at HEAD:

row	line	reads
hero tile מחזורים	:769	<code>scopedCard ? scopedCard.completedRounds : scope==='all' ? lifetime.totalFinished : stats.totalFinished</code> → <code>season-scoped, 0</code>
organisation donut	:908–913	<code>lifetime.organizationRate</code> → <code>96%</code> , gated on <code>lifetime.totalFinished + totalCancelled > 0</code>

row	line	reads
longest streak	:943–951	lifetime.longestStreak → 22, name from lifetime.longestStreakUid
פעילים השנה	:981	lifetime.activeThisYear → 7
club badges + level	:500–514	lifetime.totalFinished, champ.totalGoals + archivedGoals (0 + 27)
everything else	derived	the slice / champ rollup, which a close zeroes

That row-by-row split is ROUND4 #13, and it is HEAD only: the next build will read 0 מחזורים, then 96% שחקנים היו פעילים השנה, then 22 מחזורים ברצף, then 7 שחקנים — ROUND4 #2, the bug that started the audit, live on phones right now. ROUND4 #12 (the name prefetch still asking for the season-scoped uid) is fixed at :279–280, which now adds both uids.

The picker itself (:604–641) renders when seasons.enabled || count > 0, and the chip row only when count > 0. Chips: עכשיו N עונה, then כל הזמנים (only when pastSeasons.length > 0), then one per closed season. Selecting a past season fetches seasonSummary/{groupId}__{seasonId} through seasonHistoryService.table; כל הזמנים fetches every archive and mergeAllTime s them with the live rows, and bails to {k:'current'} if any archive is missing (:316). scopeLoading (:375) forces viewChamp to null while the archive is in flight, so the running season's numbers can never render under a closed season's heading.

Two things the picker deliberately hides outside scope==='current': the chemistry section (:866) and the club achievements (:1035). Two things it deliberately shows only outside it: "7 שחקנים שיחקו בעונה" from derived.players.length (:958) and the N סיכום עונה button (:714–727).

SeasonHistoryScreen, SeasonPoster, SeasonMedal — the hall of fame

src/screens/communities/SeasonHistoryScreen.tsx (700 lines at HEAD, 231 shipped). Data: seasonHistoryService.list(groupId) → getDocs(query(collection(db,'seasonCards'), where('groupId','==',groupId))), sorted b.no – a.no. It reads the cards, not the archives — a five-season sixty-player set of archives is over a megabyte and this screen needs a date, three numbers and nine names.

State machine (:294–343): seasons === null → SoccerBallLoader. A failed refresh sets failed and returns without touching seasons, so the list survives; a failed first load additionally sets seasons to []. failed && seasons.length > 0 renders the amber he.seasonHistoryRefreshFailed above the list; failed && empty renders he.seasonHistoryLoadFailed; not-failed and empty renders he.seasonHistoryEmpty. viewLogged (:311) makes SeasonHistoryViewed fire once per visit rather than once per pull. The list is a FlatList with initialNumToRender: 2 — twelve sealed seasons in a plain ScrollView meant ~228 SVG roots and ~336 gradients in one frame.

Header crest (:438–463): `he.seasonHallTitle` = היכל התהילה , `he.seasonHallClosed(1)` = עונה אחת , `then totals.rounds =  $\Sigma$  completedRounds = 22 מחזורים` and `totals.mini =  $\Sigma$  totals.rounds = 37 משהקונים` (the mini tile is suppressed at 0).

Worked example — what s1 renders today, field by field. `seasonCardVariant(s1)` = 'full' (players 7, winners 8). `heroSeasonId` = 's1' → `celebrate` = true → `ConfettiBurst` + `LightSweep` on every open. `heroWinner` skips `mvp` (7 names) and returns `topScorer`.

`SeasonPoster` (`src/components/community/SeasonPoster.tsx`):

- ghost numeral 1 at 168pt, `season.no` (:113)
- tint `seasonTitleTint('topScorer')` = `colors.primary` ; ghost icon `football` at 84pt
- pill `he.seasonTitleNames.topScorer` (כתר השערים)
- hero name `הלן צברי` at 35pt, capped at `MAX_HERO_NAMES = 2`
- value row: `he.seasonHeroValue('topScorer', 10)` → big 10 , unit `שערים` ; `share` = `round(10/27*100)` = 37, so the unit line reads (182–184:) `שערים · 37% מכל שערי המועדון` . `share` is computed only for `topScorer` and only when `totals.goals > 0` . Note the denominator is `totals.goals = 27` , which excludes the season's `guestGoals: 4` — the club actually scored 31.
- stat strip: 22 `משחקונים` , 37 `מחזורים` , 7 `שחקנים` (suppressed at 0),

`Cabinet` (`SeasonHistoryScreen.tsx:124–250`) draws nine fixed slots in `SEASON_TITLE_KEYS` order, three to a shelf, with an engraved socket where a title went unclaimed. Per slot: `SeasonMedal` (`ring` = tier, `core` = title tint, `ribbon` = title tint, `×N` badge when `streak > 1`), then the title name, then `names.slice(0,2).join(' · ')` , then `he.seasonTitleSharedWith(n-2)` when more, then `he.seasonTitleValue` , then `TIER_NAME[tier]` in words. The whole slot carries one `accessibilityLabel` and `SeasonMedal` is hidden from the accessibility tree (`SeasonMedal.tsx:51–54`).

The tiers s1 actually produces, with `completedRounds = 22` :

title	holder(s)	value	scale	tier
topScorer	הלן צברי	10	count [8,16,28]	כסף
topAssister	Nofar Tzabari	5	count [6,12,22]	ארד
mvp	all seven players	6	eveningScore [6.6,7.6,8.8]	ארד
topWinner	Lioz Madar	13	count [10,20,34]	כסף
mostLoyal	מתן לוי	19	19/22 = 0.864 vs [0.6,0.8,1]	זהב
cleanSheetKing	הלן צברי	13	count [5,11,20]	זהב
penaltyKing	הלן צברי	1.0	rate [0.7,0.85,1]	פלטינה
penaltyKeeper	הלן צברי	1.0	rate [0.7,0.85,1]	פלטינה
deadlyDuo	—	—	—	לא חולק

Two of these are worth the reviewer's attention. **The two platinum medals were won on two penalties each** (`penTaken: 2` , `penScored: 2` ; `penFaced: 3` , `penSaved: 3`) — `RATE_STEPS` has no volume floor, while `src/utis/penaltyStats.ts` , which decides the same-named crown on the club card, uses a Wilson lower

bound written specifically to stop a one-shot 100%. **The headline title wears the weakest metal:**

`eveningScore` is clamped to `[6,10]` and `6.0` is *also* the sentinel for "no mini-games recorded", so the seven-way `mvp` at exactly 6.0 is both structurally bronze and structurally shared. **ROUND4 #5** — `awardsDenominator` making `mostLoyal` permanently platinum — was resolved by deleting the field and the code that wrote it; `:159–170` records that both the finding and the first fix were wrong, and the divisor is now `season.completedRounds`. Ledger *shared-title-slot-names-nobody* is fixed: the names print and the suffix is additional.

The footer (`:281–289`) is `formatRange(startsAt, endsAt)` plus the `נסגרה ידנית / נתונים חלקיים` chips. **Neither chip renders on s1**, because the card carries neither field — and the ledger's *partialdata-only-on-the-sealnow-path* says s1 genuinely is partial (19 of 22 evenings predate the feature, assists only from 21.06, clean sheets from 17.08).

`formatRange` (`:71–101`) is where **ROUND4 #19** was fixed and a deeper problem was not. Both ends of s1 format to `2026 ספט`, so `from === to`, so the screen now falls through to `he.seasonRangeDays(formatDay(start), formatDay(end))` → `"16 בספט – 17 בספט"` printed directly under `22 מחזורים`. That is an improvement on the old `"2026 ספט – 2026 ספט"` and on the wrong first fix `"2026 ספט עד ספט"`, but it is still a 31.6-hour caption on a season holding twenty-two weekly evenings, because `startsAt` is the moment seasons were switched on. The dates are honest about the wrong event.

The 'void' variant renders `VoidRibbon` (`:112–122`) — one dashed grey line, `he.seasonVoidLine(no, day)` = `יום < - נסגרה בלי שנרשם בה מחזור < N עונה`. The 'noTitles' variant keeps the poster (hero `null` → the 35pt line is just `N עונה`) and replaces the cabinet with `he.seasonHistoryNoTitles`.

SeasonSummaryScreen **and** SeasonShareCard

`src/screens/profile/SeasonSummaryScreen.tsx`, model from `src/services/seasonSummaryService.ts`. Two sources, chosen by one rule (`:425–446`): **if a `seasonId` was asked for, the archive is tried first** — `seasonSummary/{groupId}__{seasonId}` — precisely so a player who has left the club and can no longer read `/groups` can still open the season they played. Only then does it fall back to the group document plus the live `communityPlayerStats` / `communityPairStats` rows.

`completedRounds` (`:382`, `:512`) is the one number this screen and the hall of fame must agree on. Closed → `d.completedRounds` from the archive (22). Running → `seasons.playedRounds`. Both were previously `totals.rounds / communityStats.rounds`, which is why the same season read 22 on one screen and 37 on another, and `0 משחקים` for every timer-only club. **HEAD only.**

Render order (`:403–699`): hero (`N עונה · club name`, then either `he.seasonsProgressRounds(played, target)` for a running rounds season or `formatRange`, then `(העונה הסתיימה)`; the season picker chips from `seasonChoices(seasons)` when `available.length > 1`, each 44pt tall; then five cards gated on two independent conditions. `me.hasData` (`seasonPersonal.ts:359`) is `rounds > 0 || games > 0` — attendance **or** mini-games, so a timer-only club no longer tells every player they played nothing. The champions card is gated on `model.closed` **only**, so a non-participant still gets `אלופי העונה`; the personal cards are gated on `hasData`.

The thirteen tiles (:519–534) read straight off `PersonalSeason` . Their denominators deliberately differ and the screen says so: `goalsPerRound` divides by `rounds` , `assistsPerRound` by `asRounds` , `cleanSheetPct` by `csRounds` , and `partialCoverage` (`rounds > 0 && (csRounds < rounds || asRounds < rounds)`) prints `he.seasonPartialCoverageNote` underneath. Four of the seven `s1` rows have `asRounds < rounds` (e.g. הֵלֶן: `rounds` 27, `asRounds` 25, `csRounds` 17), so the note fires for them.

The standing card's three tiles use `rankOf` (`seasonPersonal.ts:258–297`), whose denominator is now `rounds > 0 || games > 0` and whose rank is `null` unless **some** player has a non-zero value in that column — the fix for "three players got a gold medal for 1st place in goals decided by uid alphabetical order". Under them, `he.seasonClubRounds(model.completedRounds)` .

`SeasonShareCard` (`src/components/summary/SeasonShareCard.tsx`) is the only artefact of this feature that leaves the app. Fixed 340pt, rendered off-screen at `top: -10000` (:941–945), captured with `captureRef` , `allowFontScaling={false}` on **every** text. Tile set (:107–122): goals, assists and ערבי משחק **unconditionally**, then the six mini-game tiles only when `me.rounds > 0` — 23 of the 87 production stat rows have evenings and no mini-games, and those players used to get a card of eight zeros. `worthShowing` (:95–99) now requires `of > 1` **and** either top-3 in a club of ≥ 8 or the top third, closing both 3 מקום 3 מתוך 30 and 27 מקום 30 מתוך 30 . Titles print `sharedWith` (:209–211), so the seven holders of `s1`'s כתר העונה no longer each send an identical exclusive-sounding claim. **All four of those are HEAD only**; 1.1.9 ships the mini-game-only tile set, `bestRank.rank <= 3` with no club-size floor, and no `sharedWith` .

SeasonTitlesShelf — the profile

`src/components/profile/SeasonTitlesShelf.tsx` . Reads `users/{uid}/seasonTitles` — a collection written only by `closeSeason` — and sorts `b.at - a.at || a.id.localeCompare(b.id)` (:64), because every title from one season shares a millisecond. Each doc carries `groupName` **frozen at the moment it was won**, so the row survives the club being renamed or the player leaving. Unknown `titleKey`s are skipped (:53). It renders **nothing at all** while loading and nothing when the player holds none (:87). For this club the collection holds 14 docs across 7 users, seven of which are `__s1__mvp` at value 6.

The Hebrew: מחזור, מחזור, and the strings the data can contradict

The feature's vocabulary lives in `src/i18n/he.ts` . The two units:

- **מחזור** = one whole evening. `he.seasonStatRoundsShort`: 'מחזורים', `he.communityStatsEvenings`: 'מחזורים', `he.seasonStatEvenings`: 'ערבי משחק'.
- **משחקון** = one mini-game inside it, recorded only in advanced mode. `he.seasonStatMiniShort`: 'משחקונים', `he.seasonStatRounds`: 'משחקונים', `he.communityStatsMiniGames`: 'משחקונים'.

`he.seasonsCardInfo` is the only string that defines the distinction for a user: "מחזור הוא ערב משחק שלם, לא משחקון" — "בולד בתוך" — "a מחזור is a whole game-evening, not a single mini-game inside it." The 1.1.9 binary calls mini-games משחקים ("games") in at least three places the HEAD copy pass corrected: `communityStatsMiniGames` was 'משחקים', `seasonClubRounds` was ``${rounds}`` בעונה הזאת במועדון שוחקו במשחקים, `totals.rounds` (so "37 משחקים" for a season whose hall of fame said "37 משחקונים", and "0 משחקים" for every timer-only club), and `seasonPeerPartnerDetail` said משחקים .

The nine titles are defined once, in a `TITLE` constant **above** the `he` object (`he.ts:34-44`), because an object literal cannot read its own keys while it is being built — which is exactly how the `מלך` → `כתר` rename shipped half-done and produced ROUND4 #26: `הלן צברי` was `כתר השערים` on her hall-of-fame card and `מלך השערים` four times over on the club stats screen for the same season, one tap apart. At HEAD the club-stats leader labels point at `TITLE` (`he.ts:1356-1376`), so the two agree. **HEAD only**.

Strings whose claim the data can contradict, still true at HEAD:

- `he.seasonRangeDays` on `s1` → `16 בספט' 17 - בספט' over 22 מחזורים`. The range is real; the event it measures is not the season.
 - `he.seasonRangeUntil: (to) => 'מתחילת המועדון עד ' + to` — "from the club's beginning until X". It is reserved for a season 1 with no `startsAt`, and the file now carries a ⚠️ doc-comment saying so. It reached the same-month case once and is one refactor away from doing it again.
 - `he.seasonHistoryNoTitles` — "תואר ניתן רק למי שהגיע לפחות לחצי מערבי המשחק של העונה", "a title goes only to someone who attended at least half the season's evenings". The gate is `games >= ceil(D/2)` where `D = max(players.games)`, not the season length: on `s1` that was `ceil(19/2) = 10` of `22` evenings = 45%. The copy promises half; the code enforces half of whatever the best attendee managed.
 - `he.seasonsToggleInfo` no longer promises nine titles (`s1` awarded eight; six of the nine need mini-games), but `he.seasonHistoryNoTitles` still blames attendance first for a season whose counter simply never advanced.
 - `he.communityStatsScopeClosedInfo` — "אחוז ההתארגנות, הרצפים, הכימיה בין השחקנים ותארי המועדון נמדדים על המועדון" — "the organisation rate, the streaks, the chemistry and the club titles are measured over the club's whole life... and so they are shown only in the running season." At HEAD those three rows genuinely do read `lifetime`, so the tooltip is now true — but the `מחזורים` hero tile eight rows above it does not, and nothing on the card says which of the two scopes any given number is in.
 - `he.seasonTitleNames.mvp` = `כתר העונה`, "the crown of the season", printed on seven people at once on the only season ever closed.
 - Gender: seven of the nine titles were `מלך` ("king"); the rename to `כתר` ("crown") is the only available fix because no gender field exists anywhere in the app. `he.seasonPeerVictimDetail` and `seasonPeerTormentorDetail` drop the pronoun entirely for the same reason, and `he.seasonHeroValue` returns `'הצלחה בפנדלים'` rather than `'מהפנדלים שבט'`. `communityStatsMostLoyal` was the adjective `הכי מתמיד`; it now points at `TITLE.mostLoyal`.
 - `he.seasonPeersEmpty` — "מי שיחק עם מי נספר רק במחזורים שמנוהלים במסך הלייב המתקדם" — replaces an older `"זה יתמלא מעצמו"` ("this will fill itself in"), a promise a timer-only club can never see kept.
 - `he.seasonsConfirmSealedNow(0)` → `"0 מחזורים קיימים - תיסגר כעת 0"`: what this club's admin would read on the end-season dialog today, against a card that says the club has played 22.
-

Notifications, security rules, triggers and cost

Everything here is infrastructure the seasons feature borrowed rather than built, and most of the defects are at the seams — where a new notification type, a new collection and a new sweep were bolted onto plumbing that already had rules of its own. Line numbers are HEAD (e35394a , 2026-09-18 21:40 +0300); where production and HEAD differ, that is said explicitly.

The announcement: `announceSeasonClosed`

`functions/src/index.ts:15324–15373` . One push per player, fanned out after the archive is sealed, never before.

```
// functions/src/index.ts:15332–15361
const snap = await db.collection('seasonSummary').doc(`${groupId}__${seasonId}`).get();
if (!snap.exists) return;
const players = (snap.data()?.players ?? {}) as Record<string, { rounds?: unknown; games?: unknown };
const ops: Promise<unknown>[] = [];
for (const [uid, row] of Object.entries(players)) {
  const rounds = typeof row?.rounds === 'number' ? row.rounds : 0;
  const evenings = typeof row?.games === 'number' ? row.games : 0;
  if (rounds <= 0 && evenings <= 0) continue;
  ops.push(createNotificationOnce({
    type: 'seasonSummary', recipientId: uid,
    entityType: 'group', entityId: groupId, reason: `season-${seasonId}`,
    payload: { groupId, seasonId, seasonNo, groupName },
  }));
}
```

Three deliberate choices, all defensible:

1. **The recipient list is the archive, not the club roster.** Somebody who played the first half of the season and then left the club is still told what they did; a member who never turned up is not sent a screen of zeros. The archive's `players` map is the only list that encodes "played this season".
2. **The filter is `rounds || games`, in that order.** `rounds` counts משחקונים (mini-games) and is written only by the advanced live screen. A club running the plain timer archives `rounds: 0` for every player of every season it will ever play, so filtering on `rounds` alone means a timer-only club closes its season and tells *nobody*. `games` is evenings attended and every club records it. The fix is in — in *this* function. It is not in the security rules (below), which is the whole of finding **season-archive-rule-gates-on-minigames**.
3. **Best-effort.** Wrapped in a `try`, failures counted and logged, never rethrown. A failed push must not roll back an archive that is already sealed and a live table that is already zeroed.

Point 3 has a hole: `createNotificationOnce` **never rejects** — every internal failure path returns `{ wrote: false, skipped: '...' }` — so the `Promise.allSettled / failed > 0` accounting at `index.ts:15363-15368` is dead code, and can only ever print `0/N`. That is **season-fanout-failure-accounting-dead**: a fan-out where every recipient silently failed and one where every recipient succeeded log identically.

Deterministic ids: `dedupeIdFor` and the cooldown buckets

`functions/src/notificationDedup.ts` is a hand-maintained mirror of `src/services/notificationDedup.ts` — the functions build has its own `tsconfig` `rootDir` and cannot import app source, so the two are kept in lockstep by hand and by a byte-comparison test.

```
// functions/src/notificationDedup.ts:105-121
export function dedupeKeyFor(input: DedupeInput): string {
  return `${input.type}:${input.recipientId}:${input.entityType}:${input.entityId}:${input.reason}`
}
export function dedupeIdFor(input: DedupeInput, nowMs: number = Date.now()): string {
  const cooldown = cooldownMsFor(input.type);
  const bucket = Math.floor(nowMs / cooldown);
  const raw = `${dedupeKeyFor(input)}__b${bucket}`;
  const safe = raw.replace(/[^A-Za-z0-9:_-]/g, '_');
  return safe.length > 480 ? safe.slice(0, 480) : safe;
}
```

The document id is the dedupe key plus a bucket number, and the write is `ref.create()` — atomic, failing with `AlreadyExists` (gRPC code 6) if the id is taken. That is the entire dedupe mechanism for `seasonSummary`: no unread-suppression query (`STRICT_UNREAD_DEDUP`, `index.ts:305`, holds only `gameCanceledOrUpdated`) and no aggregation (`AGGREGATE_ON_DUPLICATE`, `index.ts:352`, only `playerCancelled` and `gamePlayersJoined`). Its cooldown is **seven days** (`notificationDedup.ts:95`), and the bucket is a global epoch-week — `Math.floor(now / 604800000)` — so boundaries fall on Thursday 00:00 UTC.

Worked example, from production. The real close on `HhzIwmjMl1i5HS0Ght3p` (מועדון שכחת שושי) ran at `createdAtMs = 1789661534040 = 2026-09-17 16:12:14 UTC`. `1789661534040 / 604800000 → 2959`. The seven documents written are literally named:

```
seasonSummary:1IdtNEjbEXfiRSqvLrJVn99Nsfi2:group:HhzIwmjMl1i5HS0Ght3p:season-s1__b2959
```

Bucket 2959 opened 2026-09-17 00:00 UTC and closes 2026-09-24 00:00 UTC. Any re-close of `s1` inside that window — which is exactly what an undo-then-reclose is — mints the same seven ids, hits `AlreadyExists`, and pushes nobody (**reclose-after-undo-notifies-nobody**). The key itself is sound: `season-s1` sits in the `reason`, so a *different* season reaches everybody. It is the seven-day bucket that makes a same-season retry unreachable for up to a week.

Why nobody has ever received a season-summary push

This is the PO: `season-push-type-not-implemented-in-prod`.

`onNotificationCreated ( index.ts:1874–1998 )` is an `onDocumentCreated` trigger on `notifications/{id}`. Its shape:

```
// index.ts:1941–1951
const canonical = await canonicaliseNotificationPayload(notif.payload);
const message = buildMessage(notif.type, canonical);
if (!message) {
  await snap.ref.update({ delivered: true, deliveredAt: Date.now(), skipped: 'type-not-implemented' });
  return;
}
```

`buildMessage ( index.ts:660–1122 )` is a `switch` over the type with `default: return null`. Its `seasonSummary` case, `index.ts:1101–1112`:

```
case 'seasonSummary': {
  const no = typeof payload.seasonNo === 'number' ? payload.seasonNo : 0;
  const club = typeof payload.groupName === 'string' ? payload.groupName : '';
  return {
    title: no ? `עונה ${no} נגמרה` : 'נגמרה', // "Season {n} is over"
    body: club
      ? `מוכן – שערים, בישולים, ומי שיחק איתך הכי הרבה ${club} סיכום העונה שלך ב`
      : 'סיכום העונה שלך מוכן – שערים, בישולים, ומי שיחק איתך הכי הרבה',
  };
}
```

("Your season summary at {club} is ready — goals, assists, and who played with you the most.")

The case exists in source. It did not exist in the **deployed** reader on 17.09. The writer (`announceSeasonClosed`) had been redeployed twice that day; `onNotificationCreated` had not been redeployed since 07.09. Production proves it — all seven documents, read today:

field	value
type	seasonSummary
delivered	true
skipped	type-not-implemented
stats	(absent)
read	false
deliveredAt	1789661535069 – 1789661535113 (≈1.0 s after creation)

Seven documents, seven `type-not-implemented`. Six of the seven recipients had live FCM tokens at the time. The season-summary push — the thing the entire close builds towards — has been delivered to **zero people, ever**.

Two aggravations:

- **It is unreplayable (season-push-unreplayable)**. The docs are stamped `delivered: true`, so even if `onDocumentCreated` re-fired it would return at `index.ts:1879`. And `onDocumentCreated` does not re-fire on an update in any case — the same trap that hid 39 undelivered `teamsGenerated` documents for eleven weeks (ROUND4 #3). Recovery needs either a manual re-mint with a different id, or waiting out bucket 2959.
- **The bail happens *after* `canonicaliseNotificationPayload`**, which reads `groups/{groupId}` per notification (`index.ts:1848–1868`). The 17.09 close therefore spent seven reads of the same club document building a message it immediately threw away.

gcloud functions describe puts `onNotificationCreated`'s last update at **2026-09-18T18:16:54Z**, i.e. after commit 8235851 (18:14 UTC) — so the reader in production today does contain the case. Nothing has closed a season since, so this is still untested against a real delivery.

Delivery: `deliverBatch`, the dormant gate, and the toggle that does not exist

`deliverBatch` (`index.ts:1511–1789`) turns recipients into FCM tokens and sends. Three gates, in order, per recipient:

```
// index.ts:1548–1578 (abridged)
const prefKey = type === 'approved' || type === 'rejected' ? 'approvedRejected'
  : type === 'friendRequest' || type === 'friendRequestAccepted' ? 'friendRequest'
  : type;
for (const user of recipients) {
  if (user.notificationPrefs?.[prefKey] === false) { skippedPref++; continue; }
  if (dormantGated) {
    const ls = typeof user.lastSeenAt === 'number' ? user.lastSeenAt : 0;
    if (ls > 0 && ls < dormantCutoff) { skippedDormant++; continue; }
  }
  const userTokens = (user.fcmTokens || []).filter(t => typeof t === 'string' && t.length > 0);
  if (userTokens.length === 0) { skippedNoToken++; continue; }
  ...
}
```

The dormant gate. `DORMANT_PUSH_CUTOFF_MS` is 21 days; `DORMANT_SUPPRESSIBLE` (`index.ts:326–347`) lists the suppressible types. `seasonSummary` is deliberately **excluded**, with a comment arguing that the person a closing season is most for is exactly the one who played the first half and drifted off, and that it fires once per season. Sound in isolation; unsound in combination with the next paragraph.

The preference key does not exist. `prefKey` for `seasonSummary` falls through to the 1:1 branch, i.e. the key `seasonSummary`. But `defaultNotificationPrefs` (`src/types/index.ts:405–424`) has no such key, and `readNotificationPrefs` (`src/firebase/firestore.ts:453–463`) rebuilds the object by iterating the *default* key set:

```
const out: NotificationPrefs = { ...defaultNotificationPrefs };
(Object.keys(out) as (keyof NotificationPrefs)[]).forEach((k) => {
  if (typeof o[k] === 'boolean') out[k] = o[k] as boolean;
});
```

A stored `seasonSummary: false` is discarded on read and can never be written back, and the settings screen builds its switches from the same list, so there is no switch. Net effect: `seasonSummary` is the one type in the app that **ignores the dormant cutoff and has no off switch anywhere**. A comment at `index.ts:1539–1547` says exactly this. Same for `eveningSummary` (ROUND4 #20).

The stats block. After delivery, `index.ts:1990–1997` writes:

```
await snap.ref.update({
  delivered: true, deliveredAt: Date.now(),
  stats: { ok: totalOk, failed: totalFailed, skippedPref, skippedNoToken, skippedDormant },
});
```

`skippedDormant` was added only in the round-4 pass. Before it, 279 of the 1,206 notification documents carrying a stats block (23%) read `{ok:0, failed:0, skippedPref:0, skippedNoToken:0}` — a deliberately-suppressed announcement and a silently-broken dispatcher were indistinguishable (ROUND4 #14). There is no in-app notification inbox, so this block is the only durable record that a push existed; the corresponding Cloud Run log line ages out after 30 days.

What the seven recipients would have looked like had the reader worked, from a live read of `/users/{uid}` and `/users/{uid}/private/push`:

recipient	tokens	lastSeenAt	outcome
מתן לוי	4	2026-09-18	delivered
Lioz Madar	1	2026-09-16	delivered
Linoy Levi	1	2026-08-29	delivered
איציק לוי	1	2026-09-14	delivered
Nofar Tzabari	1	2026-09-04	delivered
Eliran Tzabari	1	2026-09-19	delivered
הלן צברי	0	2026-06-05	skippedNoToken

Note the last row: 104 days dormant, and *not* suppressed by the dormant gate (because `seasonSummary` is exempt) — she is skipped only because she happens to hold no token. None of the seven has a `notificationPrefs` map at all except מתן לוי, whose map has no `seasonSummary` key, because it cannot.

The deep link

src/navigation/navigationRef.ts:359–373:

```
case 'seasonSummary': {
  const seasonId = typeof data.seasonId === 'string' ? data.seasonId : undefined;
  if (groupId) {
    nav.navigate('ProfileTab', { screen: 'SeasonSummary', initial: false, params: { groupId, seasonId } });
    return true;
  }
  return false;
}
```

`SeasonSummary` is registered in all three stacks that can host it (`ProfileStack.tsx:162`, `CommunitiesStack.tsx:150`, `GameStack.tsx:172`), so the known nav-registration trap — `navigate()` silently no-ops for a screen missing from the active stack — does not apply. The push carries `groupId` and `seasonId`, so the route resolves. The screen then loads through `seasonSummaryService.load` (`src/services/seasonSummaryService.ts:411–540`), which is **archive-first** when a `seasonId` is given (`:433`) precisely so a departed player never touches `/groups/{id}`, which they can no longer read. That design is correct, and it is exactly what the rules then break.

Security rules

Two traps this repository has been bitten by, both of which are load-bearing here.

Trap 1 — a `list` query does not bind path wildcards. In `match /x/{docId}`, a query (as opposed to a single-document `get`) evaluates the rule with `docId` unbound — it reads as `null`. Any expression touching it raises a *Null value error*, and a raised error denies the **entire query**, not just one document. The workaround throughout this file is to answer `list` from a **field** on the document and to add a separate `allow get` for the by-id path.

Trap 2 — `.get(k, default)` defaults only on ABSENT, never on present-null. A document holding `k: null` returns `null`, not the default; the comparison downstream (`null > 0`, `uid in null`) then raises, and the request is denied. This bit production before, on `guestsOpenAt: null` blocking non-admin guest-add.

/SEASONSUMMARY/{DOC} — FIRESTORE.RULES:2121–2164

```
allow read: if isGroupMemberSafe(resource.data.get('groupId', ''));
allow get:  if isGroupMemberSafe(resource.data.get('groupId', '')) ||
  (request.auth != null &&
    request.auth.uid in resource.data.get('players', {}) &&
    (resource.data.players[request.auth.uid].get('games', 0) > 0 ||
     resource.data.players[request.auth.uid].get('rounds', 0) > 0));
allow write: if false;
```

The split is Trap 1 applied correctly, and keeping the participant clause *out of* `list` is subtler and also correct: a `list` is evaluated per document and the whole query dies if any one fails, so a participant clause there would deny a member's *entire* season history the moment it contained one season they sat out.

The defect: `games` in that clause is younger than the deployment. HEAD has `games > 0 || rounds > 0`. The previous commit (`f446474`) has `rounds > 0` alone, and the `games` half landed in `8235851` at 2026-09-18 21:14 +0300. Both server sides — the close's participant filter and the push fan-out above — were corrected first. So there is a window in which the server pushes a season summary to every ex-member of a timer-only club and the deep link lands on `permission-denied`: the precise failure the clause exists to prevent. That is **season-archive-rule-gates-on-minigames**, confirmed independently three times in round 3 and again as ROUND4 #8. I could not read the live ruleset from this session (the Firebase Rules API returns 403 for these credentials), so whether the deployed ruleset is `f446474` or `8235851` is an **open question for the reviewer** — but the source history establishes that the two sides were out of step for at least four days.

The clause has not fired in production yet, and the honest reason is that the only club that has ever closed a season is an *advanced-mode* club: all seven archived rows carry `rounds` between 24 and 26 alongside `games` between 18 and 19. The rules gate passes on `rounds` for every one of them. The trap is waiting for the first timer-only club.

Trap 2 is **latent, not live**, here. `closeSeason` (`functions/src/seasonRollover.ts:367–380`) writes every `PLAYER_SEASON_FIELDS` entry through `num()`, which coerces to a real number, so `games` and `rounds` are always present integers. `csRounds / asRounds` are deliberately *omitted* when absent (`seasonRollover.ts:376–378`) rather than written as `0` — which is the right shape for `.get(k, default)` and would still be safe if a rule ever read them. The exposure is one `?? null` away: a writer that ever emits `games: null` turns `allow get` into a hard deny for every participant, with no error the client can explain.

`/SEASONCARDS/{DOC}` — `FIRESTORE.RULES:2105–2109`

```
allow read: if isGroupMemberSafe(resource.data.get('groupId', ''));
allow get:  if isGroupMemberSafe(resource.data.get('groupId', ''));
allow write: if false;
```

`read` already means `get + list`, and both statements carry the **identical** expression, so `allow get` is a no-op. Harmless, but the comment above claims the pair exists for the list/get split — true of `/communityStats` (`:2093–2094` , where the two expressions genuinely differ: field vs path wildcard) and false here.

Note also that the card is **members-only, with no participant clause**: a departed player can open the sealed archive of a season they played but cannot see the club's list of seasons, which makes the hall of fame a member-only surface by construction.

`/CLUBRECORDS/{GROUPID}` — `FIRESTORE.RULES:1813–1815`

```
match /clubRecords/{groupId} { allow read, write: if false; }
```

Fully server-only. This holds `eveningsSealed`, the all-time counter `seasons.roundsAtStart` is seeded from. Because the client cannot read it, season progress had to be mirrored onto the club document as `seasons.playedRounds` (`index.ts:5219–5232`) — a second copy of a number, and the origin of a good deal

of the "22 on one screen, 0 on the next" family of defects. The rule is right; the mirror it forces is the cost.

GROUPS.SEASONS — FIRESTORE.RULES:334–341 AND 376–396

Create: `!( 'seasons' in request.resource.data )` — a club cannot be *born* holding a seasons block. Without it, anyone could plant `enabled: true` with an arbitrary `count`, and the hourly sweep would adopt the club without `enableClubSeasons` ever having run.

Update: the whole map is pinned.

```
( !affectedKeys().hasAny(['seasons']) ||
  ( 'seasons' in resource.data &&
    request.resource.data.seasons == resource.data.seasons ) )
```

Server-owned and immutable from the client, which is correct: four callables exist because enabling can seal a club's whole history and closing archives, zeroes every stat row and awards nine permanent titles. Note that this is a whole-map equality comparison, and the live cadence genuinely contains present-nulls (`{"targetRounds": 24, "months": null, "endsAt": null, "endsOn": null, "startsOn": null}` on the real club today) — map equality handles nulls fine, so Trap 2 does not apply.

GAMES.SEASONID — NOWHERE

`grep -n seasonId firestore.rules` returns **zero hits**. The stamp that decides which season an evening belongs to is not mentioned in the rules at all. Consequences (**games-seasonid-unpinned**):

- `allow create ( :794–822 )` validates a named handful of fields and says nothing about extras, so a member can create a game already carrying any `seasonId` they like.
- The organiser/admin branch of `allow update ( :1172–1244 )` can "mutate ANYTHING except terminal lifecycle states" plus `createdBy` and `groupId` — `seasonId` included.
- The server stamp is **write-once by absence-check** (`index.ts:5803–5812 : !(after as { seasonId?: string }).seasonId`), so a client-planted stamp is never corrected. It is accepted as authoritative by both `src/utils/seasonScope.ts` (client) and `functions/src/seasonCounters.ts` (server).

This is not hypothetical. Production, today, on the one club that runs seasons (`seasons = { currentNo: 2, currentId: 's2', count: 1 }` — so `s3` does not exist and never has):

date	game	status	seasonId
2026-06-28 → 2026-09-15 (19 games)	...	finished	(none)
2026-09-16	yA48XsyCEP	finished	s1
2026-09-17	nRHEgxVt3j	finished	s1
2026-09-17	TZ7IWEJWWw	finished	s1
2026-09-17	DTNscolRoj	finished	s3

`DTNscolRoj` is stamped with a season the club has never reached, produced by a close/close/reopen sequence on 17.09. It belongs to no scope: the client's `inSeason()` rule (`seasonScope.ts:28–36`) says "stamped → must equal `currentId`", so it is excluded from `s2`; `s1`'s archive is sealed and cannot gain it; there is no `s3`

card or archive to hold it. The stamp is write-once and finished games are client-read-only, so **nothing in the app can repair it (seasonid-stamp-orphan-unrepairable)**. It is also live: the seal's `tableZeroedAt / seasonRoundsAtStart` `split ( index.ts:5050-5106 )` exists *because* of this document — the code comment names it.

The nineteen unstamped games are not a bug in themselves; "your history becomes season 1" is what enabling the feature promises, and both sides implement it. But the client implemented it months before the server did, which is **unstamped-season-1-rule-disagrees-with-the-server-by-19**.

Triggers

ONGAMEROSTERCHANGED — INDEX.TS:5295

An `onDocumentWritten('games/{gameId}')` trigger and the largest function in the app, firing on **every** write to a game: join, cancel, arrival check-in, admin edit, status flip, mini-game commit. End to end it handles deletion bookkeeping (`gameDeletions` + the cancellation fan-out, `:5355-5428`), capacity notices, waitlist promotion and `spotOpened` , the pending-joiner buffer, holiday notices, the registration-open announcement, the join-request fan-out to admins — and four blocks that belong to seasons.

Block 1 — the club document, fetched once. `:5452-5476` .

```
let stampedSeasonId = '';
let groupSnapOnce: Promise<admin.firestore.DocumentSnapshot> | null = null;
const groupOnce = (): Promise<admin.firestore.DocumentSnapshot> => {
  if (groupSnapOnce) return groupSnapOnce;
  const p = db.collection('groups').doc(String(after.groupId ?? '')).get();
  groupSnapOnce = p;
  p.catch(() => { if (groupSnapOnce === p) groupSnapOnce = null; });
  return p;
};
```

Four consumers wanted `groups/{groupId}` and each opened it for itself, so one sealed evening read the same club document **three times** in a single invocation (**group-doc-read-three-times-per-sealed-evening**). Fixed. Note the deliberate detail: the *success* is memoised, never the failure — `??=` cached the rejected promise, so one transient blip cost the invocation its season stamp, its progress mirror *and* the join-request push, three blocks that used to fail independently (ROUND4 #22).

Block 2 — the season stamp. `:5760-5851` .

```

const stampStatusChanged = before?.status !== after.status;
const stampRetryPending = (after as { seasonStampRetry?: boolean }).seasonStampRetry === true;
if (after.groupId && (stampStatusChanged || stampRetryPending) &&
    !(after as { seasonId?: string }).seasonId &&
    (after.status === 'active' || after.status === 'finished')) {
  const gSnap = await groupOnce();
  const seasons = gSnap.data()?.seasons;
  if (seasons?.enabled && seasons.currentId) {
    await event.data!.after.ref.update({ seasonId: seasons.currentId, ...clearRetry });
    stampedSeasonId = seasons.currentId;
  }
}

```

The status-transition gate is the fix for the second half of **group-doc-read-three-times-per-sealed-evening**: with the absence-check alone, a club that runs no seasons never gets a stamp written, so `!after.seasonId` stays true for ever and *every* write to an active or finished game re-read the club document to be told the same thing again. 191 of the 195 clubs in production have seasons off. The `seasonStampRetry` marker restores the retry the transition gate removed — an evening that loses the group read on both transitions would otherwise never be stamped, and an unstamped evening is filed under season 1 by both sides. No game in production currently carries the marker.

Block 3 — the seal's season mirror. :5050–5232, inside `sealRoundSummary`. Reads `groupOnce()`, decides via the evening's stamp whether it credits the *running* season, and if so increments `groups/{id}.seasons.playedRounds` on the same batch that seals the evening.

Block 4 — close-on-target. :5269–5291, after the batch commits.

```

if (seasonRoundsAtStart !== null && liveSeasons) {
  const playedNow = typeof liveSeasons.playedRounds === 'number' && liveSeasons.playedRounds >= 0
    ? liveSeasons.playedRounds + 1
    : eveningsSealed + 1 - seasonRoundsAtStart;
  const gName = (await args.groupOnce()).get('name');
  await closeSeasonIfRoundsTargetMet(groupId, gameId, { name: ..., seasons: liveSeasons, played: playedNow });
}

```

A rounds season closes the instant the evening that meets it is sealed, not at the top of the next hour. Only a rounds cadence can close this way; a date cadence is the sweep's job.

```
export const cronEvery60Min = onSchedule(
  { schedule: 'every 60 minutes', timeZone: 'Asia/Jerusalem', timeoutSeconds: 540, memory: '512MiB' },
  async () => {
    await runSweep('cleanupStaleGames', runCleanupStaleGames);
    await runSweep('sendPromotePrompts', runSendPromotePrompts);
    await runSweep('holidayGameNotices', runHolidayGameNotices);
    await runSweep('dailyCleanup', runDailyCleanupIfDue);
    await runSweep('clubActivity', runClubActivityIfDue);
    await runSweep('seasonRollovers', runSeasonRollovers); // ← LAST
  },
);
```

Six sweeps, **sequential**, one 540-second budget. The seasons sweep runs last and inherits whatever the other five leave; `runSweep` swallows each sweep's exception so one failure cannot starve the rest, but it cannot give back time. Two structural notes: `'every 60 minutes'` is an *interval*, not a cron expression, so `timeZone` is inert (P3 `cron-is-an-interval-not-a-clock`); and the timeout was raised from the 60-second default specifically because a close is a transaction per player plus a transaction per pair plus a batch of titles.

`runSeasonRollovers` (:16055–16283) defends its position with two constants: `SEASON_SCAN_PAGE = 200` (:16026) and `MAX_CLOSES_PER_SWEEP = 5` (:16042). Without the second, one sixty-player club with a few thousand pair documents can spend the whole remainder and every club after it in the scan is silently never reached.

The scan itself (**sweep-scan-fetches-the-whole-group-document**, fixed at HEAD):

```
let q = db.collection('groups')
  .where('seasons.enabled', '==', true)
  .select('name', 'seasons')
  .orderBy(admin.firestore.FieldPath.documentId())
  .limit(SEASON_SCAN_PAGE);
if (cursor) q = q.startAfter(cursor);
```

This reduces bytes, not billed reads. Firestore charges one document read per document returned regardless of projection; `.select()` trims the payload (2,259 bytes → 666 measured) and `.limit()` + cursor bound memory, but the read count for a club with seasons enabled is unchanged at one per sweep pass. The cost table below counts reads, not bytes, and is not improved by this fix.

due-but-blocked-is-invisible-forever, also fixed at HEAD: a due season that `clubIsQuiet` refuses used to emit one `console.log` per hour, for ever, while a season that *threw* went into the `errors` inbox a person reads. The fix stamps `seasons.dueBlockedSince` the first hour a due season cannot close and, past `DUE_BLOCKED_STUCK_MS` (24 h), calls `reportServerError({operation: 'seasonRolloverBlocked'})` — reported, never thrown, so the sweep still reaches every other club. Verified today: the production `errors`

collection holds **21 documents**, none of them seasons-related and none with `platform: 'server'` , which is consistent with the old behaviour and means this path has never yet produced a row. The club's live `seasons` map carries no `dueBlockedSince` .

Cost

Counts are Firestore **document reads**. The branch this work sits on is named `perf/firestore-read-costs` , so this is the ledger it exists for.

Today's scale, read live on 2026-09-19: **195 groups**, 1 with `seasons.enabled == true` , 95 games (91 finished), 93 `communityPlayerStats` rows, 717 `communityPairStats` rows, 675 users, 1,231 notifications.

PER SEALED EVENING

club state	reads attributable to seasons
seasons OFF (191 clubs)	2 — one <code>groups/{id}</code> on the <code>active</code> transition, one on <code>finished</code> , both via <code>groupOnce</code> and both answering "no". Before the transition gate this was one read per <code>write</code> to the game while active or finished: dozens per evening, for ever.
seasons ON, target not met	1 — <code>groupOnce</code> , now shared by the stamp, the seal mirror and the close check. Was 3.
seasons ON, target met	1 (<code>groupOnce</code>) + <code>clubIsQuiet</code> : 2 queries ($\leq 2 + \leq 3$ documents) and up to 3 <code>roundSummaries</code> gets, minus the just-sealed evening. Then a full close.

Writes per sealed evening with seasons on: one `seasonId` stamp (once in the evening's life) and one `seasons.playedRounds` increment folded into the existing seal batch.

PER CLOSE

Measured against the real club and then extrapolated. `closeSeason`
(`functions/src/seasonRollover.ts:306-...`) opens with three parallel reads:

```
const [psSnap, csSnap, pairSnap] = await Promise.all([
  db.collection('communityPlayerStats').where('groupId', '==', groupId).get(),
  db.collection('communityStats').doc(groupId).get(),
  db.collection('communityPairStats').where('groupId', '==', groupId).get(),
]);
```

step	reads, real club (7 players)	reads, 60-player club
<code>performSeasonClose</code> pre-flight <code>groups/{id}</code>	1	1
<code>communityPlayerStats</code> where <code>groupId</code>	7	60
<code>communityStats/{id}</code>	1	1
<code>communityPairStats</code> where <code>groupId</code>	314	~2,700
<code>getAll</code> of <code>/users</code> for display names	7	60

step	reads, real club (7 players)	reads, 60-player club
seasonSummary latch get	1	1
per-player wind-back transactions (1 read each)	7	60
communityStats wind-back transaction	1	1
sealedEveningsOf → clubRecords/{id}	1	1
lifecycle advance transaction	1	1
announceSeasonClosed → seasonSummary re-read	1	1
onNotificationCreated ×N: canonicalise groups/{id}	7	60
onNotificationCreated ×N: loadUsers (root + private/push)	14	120
total	≈363	≈3,067

The pair row dominates and is almost pure waste (**guest-pairs-make-every-close-read-15x-what-it-archives**). Of the club's 314 pair documents, 293 are guest pairs — a guest id is minted fresh for every game, so the collection gains a permanent document per stranger per night and nothing ever deleted one. The archive keeps none: the log for the real 17.09 close reads "kept 0, dropped 293 guest, 21 empty" — 314 read, zero archived. HEAD deletes guest pairs during the wind-back (seasonRollover.ts:926–948); the 314 are still there today because that code was committed after the only close.

Two more line items are pure overhead: the seven canonicalise reads of one club document (one per notification, all identical, all discarded when buildMessage returned null), and the /users read per close that exists only to stamp closedBy / closedByName — two fields **read by nothing (closedby-write-only)**.

PER SWEEP RUN

runSeasonRollovers costs, per hour:

- **1 read per club with** seasons.enabled == true — the paged scan. Today: 1.
- Clubs not yet due stop there: a date cadence is answered from the document in hand, and completedRoundsOf is called only for a rounds cadence, and then only via the playedOnce() memo (:16114–16124) so the value is computed once instead of the two identical derivations it used to do.
- A due club adds clubIsQuiet : 2 game queries plus up to 3 roundSummaries gets ≈ 5–8.
- A due-and-quiet club adds a full close (table above).

scale	sweep reads/hour	sweep reads/day
today (1 enabled club, not due)	1	24
1,000 enabled clubs, none due	1,000	24,000
1,000 enabled clubs, 5 due and quiet	1,000 + 5×~363 ≈ 2,815	~67,600

24,000 reads/day is the floor at 1,000 clubs, it buys nothing on the ~999 not due, and it is the number `.select()` does *not* reduce. The obvious improvement — a `where('seasons.cadence.endsAt', '<=', now)` filter — is not implemented, and cannot cover a rounds cadence, whose due-ness is not a field on the club document.

The 5-close ceiling interacts badly with scale: at 1,000 clubs on a weekly rhythm, more than five will routinely be due in the same hour, and a club deferred behind a busier neighbour has no priority or fairness mechanism — the scan order is document-id ascending, for ever. Nothing tests any of this (**nothing-tests-the-sweep**: zero tests for `runSeasonRollovers`, `clubIsQuiet`, or the ordering).

PER STATISTICS-SCREEN OPEN

Season-scoped club statistics come from `gameService.getCommunityStats` (`src/services/gameService.ts:949–1124`), whose single query is:

```
const q = query(col.games(),
  where('groupId', '=', groupId),
  where('status', 'in', ['finished', 'cancelled']),
  orderBy('startsAt', 'desc'),
  limit(200));
```

Up to 200 game documents per open, season-scoped or not — the season filter is applied in memory afterwards, via `inSeason()`. On the real club that is 23 reads; on a club with three years of history it is the full 200, every time the screen is opened, with no cache. Plus, on the same screen: the club document, `communityStats/{groupId}`, and a `communityPlayerStats where groupId` scan (7 today, 60 on a large club).

The hall of fame is cheap by design: `seasonHistoryService.list` (`src/services/seasonHistoryService.ts:116–145`) reads `seasonCards`, not archives — 2,473 bytes per card against 20,596 bytes for the same season's archive, with the winners already resolved. One read per finished season.

The personal season summary (`seasonSummaryService.load`) is two-shaped: a **closed** season is one read of `seasonSummary/{groupId}__{seasonId}` plus a peer-name resolve; the **running** season is `groups/{id}` + `communityPlayerStats where groupId` + two `communityPairStats` queries (`a == me`, `b == me`), so it scales with roster size rather than with the season.

Finally, the rules themselves bill reads. `isGroupMemberSafe` (`firestore.rules:101–106`) performs an `exists()` **and** a `get()` on `groups/{gid}` — two document accesses per evaluation, cached within a single request. So every `seasonCards` list, every `seasonSummary` get and every `communityStats` read adds two billed reads on top of the documents returned. On the archive's `allow get` the participant clause is free by comparison: membership is a key in the document being read, so it costs nothing extra.

What is tested, what is not, and the open questions

How the suites are run, and by whom

There are two independent test surfaces and no automation connecting them to anything.

The unit suite is Jest. `jest.config.js` is five lines:

```
module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  roots: ['<rootDir>/tests'],
  moduleNameMapper: { '^@/(.*)$': '<rootDir>/src/$1' },
  transform: { '^\\.\\.\\.tsx?$': ['ts-jest', { isolatedModules: true }] },
};
```

`roots` is `tests/` only, and the transform matches `.ts / .tsx`. The rules suite under `tests/rules/` is written in `.mjs` against Node's built-in `node:test` runner, so `npx jest` does not see a single one of its files. Running `npm test` at the repo root reports a green run while 146 security assertions have not been executed.

The rules suite is its own npm project (`tests/rules/package.json`) with its own `node_modules` :

```
"test": "node --test --test-concurrency=1 --test-reporter=spec *.test.mjs"
```

`--test-concurrency=1` is load-bearing. Five of the eight suites (`firestore` , `antiHijack` , `clubIsolation` , `oldClientCompat` , `roundHistoryAccess`) share `projectId: 'demo-soccer'` and each calls `clearFirestore()` in its setup; run in parallel they wipe each other's fixtures. `seasonSummary.test.mjs` is one of the three that took its own project id (`rules-season-summary`), so the seasons rules tests are at least isolated from that hazard.

There is **no CI**. There is no `.github/` , no workflow file, no `.husky/` , and no non-sample hook in `.git/hooks` . Nothing runs Jest, the rules suite, or `tsc` unless a person types the command. `npm run typecheck` (`tsc --noEmit`) is clean at HEAD for both the app and `functions/` , but Metro — the React Native dev server — does not typecheck, which is how three seasons defects of exactly the class a compiler catches reached the production `errors` collection on 18.09 from the developer's own emulator:

fingerprint	message	surface
toxitz	<code>_he.he.seasonsCardOfTarget</code> is not a function (it is undefined)	<code>SeasonsCard</code>
1ytrra f	Cannot read property 'totalFinished' of undefined	<code>CommunityStatsScreen</code>
3lov71	Property 'winsPlaceIsFactual' doesn't exist	<code>assistantInsightsService</code>

All three are `appVersion 1.1.5` with a `10.0.2.2:8081` Metro bundle URL, so they are dev-emulator reports rather than store users — but they are the only seasons entries the error inbox has ever held, and they are all "a symbol that did not exist yet". The production `errors` collection holds 21 documents in total and **not one of them comes from the seasons backend**, which is a fact about the backend's error reporting rather than about its correctness: the hourly sweep's failure path is `console.log`, and only a `throw` reaches the inbox.

The seasons unit suite, file by file

Twenty-three Jest files touch seasons; together they are 362 assertions and they all pass today (2.7s). Every one of them was written between 13.09 and 18.09.2026 — that is, *after* the feature shipped in 1.1.7 on 16.09 and mostly *during* the audit. Season 1 of the only club that runs seasons was closed by this code at `2026-09-17T16:12:06Z`, at which point the close had no functional test of any kind.

File	n	What it pins	What it leaves free
<code>seasonCounterReconciliation.test.ts</code>	38	All four answers to "how many <code>מחזורים</code> " agree on the real club: the client games scan, <code>inSeason</code> , the counter subtraction, and <code>countSeasonParticipants</code>	One club's fixture. A second club shape is not exercised
<code>seasonMedalTier.test.ts</code>	40	The 60/80/100% steps, the 6–10 MVP range, all nine titles gradable, <code>titleStreak</code>	Nothing renders a medal in a test
<code>seasonAwards.test.ts</code>	32	Eligibility <code>ceil(D/2)</code> , penalty floors, ties shared, the MVP scale floor, the nine-key set	The choice of <code>D</code> — the denominator is an argument here
<code>seasonPersonal.test.ts</code>	29	<code>buildPersonalSeason</code> : rates, ranks, peer orientation, guest exclusion, three distinct rival rows	<code>seasonSummaryService</code> , which chooses the inputs
<code>seasonCardVariant.test.ts</code>	22	full / card / ribbon / void classification; a timer-only club is never void; <code>heroWinner</code> , <code>heroSeasonId</code>	The screen that renders the variant
<code>seasonLifecycle.test.ts</code>	22	<code>isSeasonDue</code> , the quiet seam, rounds remaining, mid-season retarget validation, month clamping	The client copy only; the server has its own
<code>seasonActivation.test.ts</code>	21	<code>planActivation</code> for every cadence × choice × history combination, including the pre-picker client	The settings screen that builds the input
<code>seasonDates.test.ts</code>	21	Month-end clamping, leap years, club-midnight vs UTC, three years of arithmetic	—
<code>seasonCloseNumbers.test.ts</code>	20	<code>windBackRow</code> , <code>restoreRow</code> , <code>seasonPairKey</code> , <code>awardsDenominatorOf</code> , <code>sealedCardEvenings</code>	Which documents <code>closeSeason</code> reads, in what order, in which batch
<code>eveningPlayedMirror.test.ts</code>	17	Five client/server mirror pairs: byte identity and differential execution	The deployed bundle
<code>seasonSweep.test.ts</code>	12	<code>seasonFinishLine</code> and <code>isSeasonDue</code> from <code>functions/src/seasonCounters.ts</code>	The sweep itself, <code>clubIsQuiet</code> , <code>performSeasonClose</code>

File	n	What it pins	What it leaves free
<code>seasonOpensWithOffset.test.ts</code>	11	All five season-opening object literals stamp <code>roundsAtStart</code> and <code>playedRounds</code>	Whether the values are right
<code>groupSeasonsReader.test.ts</code>	10	The client deserializer keeps every field the server writes, including <code>targetHistory</code>	—
<code>groupHydrate.test.ts</code>	10	A failed club refresh does not empty the store	—
<code>seasonRolloverFields.test.ts</code>	9	<code>__seasonFields</code> — the exact list of counters a close zeroes, and their pairings	That the list is applied
<code>seasonArchiveTable.test.ts</code>	8	<code>parseSeasonTable</code> over a hand-copied archive fixture	Key-set parity with the writer
<code>seasonChoices.test.ts</code>	7	<code>lastClosedSeason</code> , the season picker order	—
<code>seasonScopedStats.test.ts</code>	7	<code>inSeason</code> — including "unstamped means season 1" — over the real club's documents	—
<code>seasonAwardsMirror.test.ts</code>	6	The title rules agree client/server at denominators 0, 1, 2, 13, 19, 20, 22, 40	—
<code>seasonCadenceClean.test.ts</code>	5	Every cadence literal nulls the other kind's fields and declares its <code>type</code>	—
<code>sealedCounterAtomicity.test.ts</code>	5	Neither evening counter is ever written as <code>x + 1</code>	The race itself
<code>seasonCardParticipants.test.ts</code>	5	<code>countSeasonParticipants</code> on a timer-only club	—
<code>seasonSeedHistory.test.ts</code>	5	<code>seasonSeed</code> starts a season at zero regardless of club history	—
<code>allTimeTable.test.ts</code>	9	Live season + archives merge; coverage denominators are dropped, not invented, when one side never measured	—

The fixture that most of the newer files share, `tests/fixtures/realClub.ts`, is a hand-copied snapshot of the one club that runs seasons, taken on 18.09.2026: 22 evenings of which 19 carry no `seasonId`, one `unverified` evening, one cancelled, `eveningsSealed: 10`, `roundsAtStart: 7`, a sealed card of `completedRounds: 22` beside `totals.rounds: 37`. Its own header explains why it exists — every fixture before it described a club that does not exist. Nothing re-verifies it against production; it will drift silently.

Measured coverage of the server modules

File	% Stmts	% Branch	% Funcs	Uncovered Line #s
functions/src	45.14	33.99	72.41	
seasonActivation.ts	97.22	100	100	179
seasonAwards.ts	92.45	83.33	87.09	249–250
seasonCounters.ts	97.5	97.14	100	75
seasonDates.ts	85.24	69.23	88.88	74–76, 128–135
seasonParticipants.ts	100	100	100	
seasonRollover.ts	12.62	5.50	28.00	308–1026, 1072–1252
seasonSeed.ts	100	100	100	

`seasonRollover.ts` is 1,253 lines. The uncovered ranges are exactly `closeSeason` (declared at line 307) and `reopenSeason` (line 1067). Everything the file has a test for is the arithmetic helpers above them — `windBackRow` (62), `restoreRow` (84), `seasonPairKey` (104), `awardsDenominatorOf` (133), `sealedCardEvenings` (155) and the `__seasonFields` list (1034). The two functions that actually mutate a club's history run 5.5% branch-covered.

`functions/src/index.ts` is 17,081 lines and is **never imported by any test**. The seasons lifecycle occupies roughly lines 15,405–17,081 of it, plus the stamp block at 5,802–5,850, the push fan-out at 15,333–15,367 and the notification body at 1,101. All of it is reached only as text.

The MIRROR tests, and what they cannot see

Cloud Functions cannot import from the app's `src/`, so six modules exist twice. Each server copy carries a marker line and is required to be byte-identical below it:

```
const MARKER =
  '/* ---- everything below this line is a copy of the client file ----';
const copied = server
  .slice(server.indexOf(MARKER) + MARKER.length)
  .replace(/^\\n+/, '')
  // Cloud Functions have no `@/` alias, so that one line is allowed to
  // differ – and ONLY that one.
  .replace(/from '\\./seasonDates'/g, "from '@/utils/seasonDates'");
expect(copied).toBe(client);
```

`eveningPlayedMirror.test.ts:32` declares five pairs — `eveningPlayed`, `seasonParticipants`, `seasonSeed`, `seasonDates`, `seasonActivation` — and `seasonAwardsMirror.test.ts:23` covers the sixth, `seasonAwards`.

Two defects in this mechanism were found by the audit and both are fixed at HEAD, but the second is the one a reviewer should understand:

1. `mirror-each-ignores-its-own-params` (**LEDGER**). The `describe.each` table declared five module pairs and the body hard-coded `src/utils/eveningPlayed.ts`. Four of the five mirrors were never compared; the suite reported ten green assertions that were two assertions run five times. The comment at lines 47–55 now records this.
2. **Byte identity is not evidence.** Two identical copies of a *wrong* rule pass a diff. Until 18.09 nothing in either file ever *executed* the server copy, so the server half of six rules had no test at all — only a diff. Both files now also load both modules and run them side by side: 480 evening shapes through `eveningPlayStateWithReason`, every club-document shape through `seasonSeed`, three years of month ends through `seasonDates`, every settings-screen submission through `planActivation`, and the real club's seven rows through `computeSeasonAwards` at eight different denominators.

The limitation that remains is structural and the reviewer should not overlook it. **The mirror tests read** `functions/src/`. **Production runs** `functions/lib/`, the `tsc` output, which is gitignored (`.gitignore:58`). Nothing in the repo compares the deployed bundle against either copy. This is not hypothetical: ROUND4 #1 established that both 1.1.9 store binaries were built from commit `4de80d0` while the backend they talk to was HEAD — twenty client files and +2,636 lines apart — and the audit's only method for checking what was actually deployed was downloading the function bundles and grepping them. A green mirror run proves `src/utils/x.ts` and `functions/src/x.ts` agree in the working tree. It says nothing about what the user's phone and the running function are doing.

A third limitation: agreement is not correctness. `seasonAwardsMirror.test.ts:65` runs the real club's sealed season through both copies and asserts they produce the same answer. They do. The answer they agree on gave שחקן העונה ("player of the season") to all seven members of the club at a value of 6.0, which is the evening-score sentinel for "no mini-games were recorded". Both halves were wrong in perfect agreement.

The SOURCE-GREP guards

Three rules cannot be unit-tested from a Node process — a Firestore merge into a nested map, a read-modify-write race, and a field missing from one of five inline object literals. The repo guards them by reading the backend as a *string* and asserting its shape. The three guards are `seasonCadenceClean.test.ts`, `seasonOpensWithOffset.test.ts` and `sealedCounterAtomicity.test.ts`.

As shipped, all three read exactly one file — `functions/src/index.ts` — with hand-written regular expressions that terminated on indentation. The cadence guard's original matcher was:

```
/(?:cadence|next) = \{[\s\S]*?\n {4,6}\};/
```

Two failure modes follow directly. **Indentation-sensitive:** the `\n {4,6}\};` terminator means a literal nested one level deeper, or written on a single line, is simply not matched. **Single-file:** a writer moved into `seasonRollover.ts` — or into a file created tomorrow — is invisible. Both are the same flaw: *the guard describes where the code is today rather than what it must not do*. And a guard that matches nothing asserts nothing while reporting green: the cadence file reports four passing tests over an empty array just as happily as over four literals.

tests/fixtures/serverSource.ts (added 18.09) replaces both properties. serverFiles() walks all of functions/src/ , skipping node_modules and lib , and returns each file's text with a repo-relative path so a failure names something greppable. objectLiterals(marker) finds the opening brace after the marker and then counts braces to the close, so indentation is not part of the rule:

```
const open = file.text.indexOf('{', m.index + m[0].length - 1);
let depth = 0, i = open;
for (; i < file.text.length; i += 1) {
  if (file.text[i] === '{') depth += 1;
  else if (file.text[i] === '}') { depth -= 1; if (depth === 0) break; }
}
```

codeLines() strips comments so a guard looking for a forbidden write does not fire on the comment explaining why it is forbidden — and, more importantly, is not satisfied by one.

What is still fragile, and a reviewer should treat these as weaker than the tests around them:

- **The guards are count assertions.** expect(cadenceLiterals.map(c => c.where)).toHaveLength(4) and expect(blocks.map(b => b.where)).toHaveLength(5) . A sixth season-opener fails the count, which is the intent — but it fails on arithmetic, not on behaviour, and the fix ("update the number") is one character away from the fix that defeats the guard.
- **They match identifiers and string literals.** /type: 'rounds'/ , /currentId:/ , /playedRounds:\s*eveningsSealed/ . Renaming a field, or building the literal by spread from a helper, quietly removes code from the guard's view. seasonOpensWithOffset.test.ts already has to special-case ...seasonSeed(for this reason.
- sealedCounterAtomicity.test.ts:63 is the only guard that proves it can see its own subject ("and the guard is reading the file the seal is actually in"). The other two do not. That assertion is the pattern the other guards need.
- They read functions/src/ , with the deployment gap described above.

The honest inventory of untested code

Path	Where	Functional test
runSeasonRollovers — the hourly sweep	index.ts:16055	none
clubIsQuiet — the close permission gate	index.ts:15466	none
performSeasonClose — archive + titles + push + lifecycle	index.ts:15790	none
closeSeasonIfRoundsTargetMet — the on-seal close	index.ts:15943	none

Path	Where	Functional test
<code>closeSeason</code> — the wipe and the archive	<code>seasonRoll over.ts:307</code>	none (helpers only)
<code>reopenSeason</code> — the undo	<code>seasonRoll over.ts:1067</code>	none (helpers only)
The <code>seasonId</code> stamp trigger + <code>seasonStampRetry</code> marker	<code>index.ts:5802</code>	none
<code>playedEveningsOfSeason</code> / <code>completedRoundsOf</code>	<code>index.ts:15599 , 15695</code>	none
The six callables (<code>enableClubSeasons</code> 16286, <code>disableClubSeasons</code> 16613, <code>updateSeasonTarget</code> 16627, <code>reopenLastSeason</code> 16848, <code>endSeasonNow</code> 16971)	<code>index.ts</code>	none
<code>onNotificationCreated</code> and the <code>seasonSummary</code> push body	<code>index.ts:1874 , 1101</code>	none
The push fan-out	<code>index.ts:15333–15367</code>	none
<code>seasonSummaryService.ts</code> (636 lines), <code>seasonService.ts</code> (274)	<code>src/services/</code>	none
<code>SeasonsSettings.tsx</code> (1,150), <code>SeasonSummaryScreen.tsx</code> (951), <code>SeasonHistoryScreen.tsx</code> (699), <code>SeasonPoster</code> , <code>SeasonShareCard</code> , <code>SeasonsCard</code> , <code>SeasonMedal</code> , <code>SeasonConfirmSheet</code> , <code>SeasonTitlesShelf</code>	<code>src/screens</code> , <code>src/components</code>	none — there is no component-render test in this repo at all

Stated plainly, because it is the single most important sentence in this section: **the only unattended path in the app that destroys production data has no functional test**. The phrase is the codebase's own, from the comment on the cron that carries it (`index.ts:13589`):

```

export const cronEvery60Min = onSchedule(
  {
    schedule: 'every 60 minutes',
    timeZone: 'Asia/Jerusalem',
    // The default is 60 seconds, and this job now contains the season sweep –
    // the only unattended path in the app that destroys production data.
    timeoutSeconds: 540,
    memory: '512MiB',
  },
  async () => {
    await runSweep('cleanupStaleGames', runCleanupStaleGames);
    /* ... four more ... */
    await runSweep('seasonRollovers', runSeasonRollovers);
  },
);

```

Every sixty minutes, unattended, that function may zero a club's `communityPlayerStats`, `communityStats` and `communityPairStats`, write a `seasonSummary` with `create()` (which is one-shot), write nine `seasonTitles` onto players' profiles, and push every participant. `clubIsQuiet` is the only thing standing between a club and that happening mid-evening, and the fix that unblocked 24 of 30 clubs from a permanent `unsealedGame` refusal — the `endedBy !== 'admin' && endedBy !== 'auto'` skip for legacy closes at `index.ts:15557` — landed with no test exercising it. The resume path that exists because a close can be killed halfway is likewise untested; so is the archived-latch that is supposed to stop a concurrent seal-close and sweep from resetting the *new* season (LEDGER `perform-close-writes-next-season-even-when-nothing-was-archived`).

Two more absences worth naming. Nothing tests the Hebrew vocabulary: the latest commit at HEAD (`e35394a` , "The last six labels that still called a משחק a משחק") was, in its own words, "caught on the device, not in the diff" — the club stats screen printed "37 משחקים" (37 *games*) beside "22 מחזורים" (22 *evenings*) while the hall of fame one tap away called the same 37 "משחקונים" (*mini-games*). `copyDirectionality.test.ts` bans horizontal arrows in user-facing strings and `format.test.ts` pins the Hebrew plural forms of the season countdown; neither can catch a unit word. And nothing reconciles a *sealed* archive against the current rules: the s1 card still names seven MVPs at 6.0 because it was written before the scale floor existed, and `seasonAwards.test.ts:327` now asserts that cannot happen again — for future seasons only.

The rules test suite

```

brew install openjdk@21 # firebase-tools requires JDK 21+
export PATH="/opt/homebrew/opt/openjdk@21/bin:$PATH"
firebase emulators:start --only firestore --project demo-soccer
cd tests/rules && npm install && npm test

```

One practical trap the README does not mention: `firebase.json` declares no `emulators` block, so the CLI also starts the Emulator UI on port 4000 and **aborts the whole run** if that port is taken (`Error: Could not start Emulator UI, port taken`). Adding `"emulators": { "ui": { "enabled": false } }` locally is the workaround.

Run against HEAD on 19.09.2026 the suite reports:

```
i tests 146    i suites 14    i pass 142    i fail 4    i duration_ms 7476
```

The four failures, and what each actually is:

1. `games: self can join an open community game ( firestore.test.mjs:179 )` — expects success, gets `PERMISSION_DENIED: Unable to evaluate the expression as the maximum of 1000 expressions to evaluate has been reached. for 'update' @ L824` . Not a rules bug in the ordinary sense: the `/games allow update at firestore.rules:824` is one long OR chain and Firestore stops evaluating a rule after 1,000 expressions. The rules file documents this at line 826 — the publish-split branch was moved to the front *because* "on a real game document (82 fields) the branches below burn the whole budget and the request is denied before this one is ever reached. That is what actually failed in production."
2. `OLD-CLIENT manual-offer cancel ( oldClientCompat.test.mjs:61 )` — the same ceiling, same rule, same line.
3. `groupsPublic: admin of canonical group can create the public mirror` — a stale test. It writes `memberCount: 2; firestore.rules:627` caps a create at `<= 1` . The test predates the cap.
4. `notifications: cannot fake an "approved" push to inflate a join` — a test that contradicts its own comment. It `assertFails` on a `spotOpened` create, then explains three lines later that "spotOpened IS in the client whitelist — that's intentional". Stale, not a hole.

So two of the four are test rot and two are a **live production constraint that the seasons rules sit directly downstream of**. The expression ceiling is not confined to the failures. Over the whole run, forty requests logged `maximum of 1000 expressions` : twenty-two of them against `@ L343` (the `/groups allow update`) and eighteen against `@ L824` . Every one of the seven tests that prove `groups.seasons` cannot be touched from a client (not an admin adding it, not an admin rewinding one that exists, and an ordinary member certainly does not, not deleting the field, not one nested number, not riding along on a self-leave, not riding along on a join request) **passes with a budget-exhaustion error in the log**. The deny is real, but it is not proof the rule denied — it is proof the engine gave up. If the `seasons` immutability clause were removed tomorrow, those seven tests would very likely still pass. That is the most serious weakness in the rules coverage of this feature and it has not previously been reported.

What the seasons rules suite does pin, and pins well — 40 tests in 12 groups, all passing, with its own project id:

- A club member reads a `seasonSummary` by id and by list, answered from the `groupId` **field**, never the path wildcard (the `list -does-not-bind` trap that once made `communityStats` unreadable).
- A **departed participant** reads a season they played, by id, on `games > 0 || rounds > 0` . The `games` half exists because the rule previously authorised on `rounds` alone and every ex-member of a timer-only club was pushed a season summary and denied when they opened it (LEDGER `seasonsummary-participant-rounds-only` ; confirmed three times independently). `seasonSummary.test.mjs:131` — "and a timer-only season, where nobody has a single `משחקון`" — is the regression test, and it passes.

- A departed player may *not* list, may not read a later season their zeroed row still appears in, and may not read a season they never played.
- `seasonCards` is member-only including the list path; nobody writes a card, an archive or a title; nobody deletes one; nothing is public to the internet.
- A player cannot award themselves a title, on their own profile or anybody else's, nor overwrite one they did win.
- A club cannot be **born** holding a `seasons` block (`allow create` at `firestore.rules:324` ends ! (`'seasons' in request.resource.data`)), which closed the door on a client planting `enabled: true` and having the hourly rollover adopt it.

What it leaves free:

- `seasonId` on `/games` is **not pinned anywhere**. `grep -c seasonId firestore.rules` = 0 outside two comments, and no rules test touches it. A member can create a game pre-stamped with any season id and the server's stamp is write-once, so it never corrects it. The resulting orphan (a game stamped `s3` on a club that knows only `s1` / `s2`) belongs to no scope and is unrepairable: finished games are client-read-only.
- Only one club shape. The fixtures are one group, one member, one outsider, one departed player.
- The `/groups` update rule is tested only through the budget ceiling described above.

The open questions

These are the decisions that are genuinely unresolved — not defects with a known fix, but forks where more than one answer is defensible and the code has silently taken one. They are ordered by how much of the feature moves when the answer changes.

1. Should the awards denominator be the season's length or the best attendance? Today it is the best attendance: `awardsDenominatorOf ( seasonRollover.ts:133 )` returns `Math.max(...players.games)`, which on the one real season is **19** against a season length of **22**. The gate is `games >= ceil(D/2)`, so 19 gives a threshold of 10 (45% of the season) where the Hebrew copy promises half; and `minPenaltyAttempts(19) = 2` against `(22) = 3`, which is the entire reason מלך הפנדלים went to a player with a 2-of-2 record instead of a 2-of-3 one. *Season length* matches the copy and makes "attended every evening" meaningful, but the numerator (`games`, counted since 22.06) and that denominator (`eveningsSealed`, born 25.08) come from different eras — on the club the gate was calibrated against, 22 would admit 25 of 29 players to a gate meant to admit 13, including seven who turned up twice. *Best attendance* keeps both sides in the same units but is self-referential: `mostLoyal` is the maximum of the same array, so the loyalty winner always clears its own gate and `winner.value / D` is identically 1.0 — which is why `awardsDenominator` was written to the card for half a day and then removed, and why platinum on כתר ההתמודה is currently unreachable for anyone who missed one night. A third answer exists: keep attendance for *eligibility* and length for every *share* the app displays, which is roughly where the code has landed by accident, but has never been stated as the rule.

2. Should the MVP have a sample-size gate? `mvpAvg` is `eveningScoreSum / eveningScoreCount` (`seasonRollover.ts:516`) with no minimum on the count, ranked by `leaders(..., MVP_SCALE_FLOOR = 6)`. On the real club, `eveningScoreCount` runs 2–3 against 17–19 evenings attended, and all seven members tied at exactly 6.0 — the sentinel the scale returns for "played no mini-games". *No gate* is the status quo and is simple. *The same* `ceil(D/2)` gate the other titles use is consistent, but on any club whose evening-score

accumulator has been live for less than a season it awards nobody, and the card's current Hebrew blames the players for that. *Shrinking toward the club mean* (an empirical-Bayes prior) ranks honestly but costs explainability: the number that decided the title stops being the number on the player's own card.

3. Should a season's statistics be the live scan or the sealed card? Both exist and they disagree. The card is deterministic, cheap, survives account deletion, and is protected — `allow write: if false` in the rules, `create()` on the server. It is also frozen with whatever was wrong when it was written: seven MVPs at 6.0, and per-player `games` short by exactly their attendance at three evenings that carry no `finishCredited` marker. The live scan is self-correcting — fix the rule and history fixes itself — but cannot answer for a deleted game, changes its answer as evenings age out of the 200-document window, and costs reads on every screen. A third option, *card by default with an admin-visible recompute*, breaks the write-once property both layers currently enforce. Note that the property is already not quite true: `seasonSummary/HhzIwmjMl1i5HS0GHt3p__s1` has `createTime 2026-09-17T16:12:08Z` and `updateTime 2026-09-18T11:41:41Z` — it was repaired by hand a day later, which is exactly the operation the design forbids.

4. Should guest pairs be archived? 293 of the 314 `communityPairStats` rows on a seven-player club are guest pairs (93%). The close reads all 314, archives none of the guests, zeroes them, and the undo can only restore what was archived — so a reopen loses them permanently. *Archive them* makes the undo lossless, at the cost of document size (`MAX_ARCHIVED_PAIRS = 1200`, ~250–300 bytes per entry against Firestore's 1MB ceiling) and of freezing a "partner" who is a different human next week, since a guest identity is per-evening. *Delete them at close* removes the 15× read amplification — one close read 314 documents and archived zero — at the cost of the club's lifetime chemistry history. *Never create them* is cheapest of all but the live evening's chemistry card is the feature they exist for. The question is really "is a guest a person or an event", and the data model has not decided.

5. Should `eveningsSealed` be backfilled? `clubRecords.eveningsSealed` is 10 on a club that has played 22 evenings, because the counter began on 26.08.2026; `seasons.roundsAtStart` is 7, and $10 - 7 = 3$ is the number two separate shipped code paths archived for a season everyone had watched reach 22. *Backfill it per club from the games scan* makes the number mean what its name says, but it is the denominator of every running rounds-cadence season: moving it moves every club's finish line at once, and the `playedRounds` mirrors seeded from it would all need re-seeding in the same operation. *Leave it and rename it* ("evenings since 26.08") is honest and cheap but leaves every rounds target measured in a unit the card does not show. *Retire it in favour of stamped games* removes the era problem but reintroduces the one the counter was created to solve: deleting a game would silently walk a season's progress backwards.

6. Should the three counters be collapsed into one derivation? There are three answers to "how many מחזורים": `clubRecords.eveningsSealed` (lifetime counter), `seasons.playedRounds` (per-season mirror), and a scan over stamped games filtered by `didEveningHappen`. `seasonCounterReconciliation.test.ts` exists precisely because nothing compared them and all three disagreed. *One derivation from the games* is the truthful answer and costs an unbounded scan plus sensitivity to deletion. *Keep the mirror as the display number and make the counter internal* is the smallest change and leaves two numbers that can still drift, held together by a single test over a single club's fixture. *Event-source it* — one immutable document per credited evening in a `seasonEvenings` subcollection — gives idempotency, a repairable history and a cheap count, at the price of a migration and one extra write per seal.

7. Should `מחזור` and `משחקון` be one unit in the data model? They are two today (`completedRounds` vs `totals.rounds` : 22 and 37 on the same season), and the most recent commit at HEAD is the sixth pass at fixing labels that confused them. *Keep both* is correct and requires the copy discipline no test enforces. *Collapse to evenings* makes every timer-only club — the common club — coherent and throws away the advanced mode's per-mini-game record. *Derive the display unit from the club's mode* removes the confusion at the cost of a screen that reads differently for two clubs looking at the same number.

8. Should ending a season early be destructive at all? `endSeasonNow` and the sweep take the same irreversible path, and `reopenLastSeason` exists as the compensating transaction — with no time bound, no already-undone guard, and a `seasonWoundBack` stamp that survives it (all 314 pair rows currently carry `seasonWoundBack: 's2'` while s2 is the *running* season, which will make s2's close archive the chemistry and skip the wind-back). The alternative is a non-destructive "sealed" state: compute the archive, leave the live counters alone, and subtract only when the next season's first evening is credited. That makes undo a no-op and removes the resume path entirely, at the cost of every live table reading as cumulative across the seam — which is the behaviour seasons were introduced to end.

The parts nothing else owns

Eleven sections describe what a season *does* with its numbers. This one describes where those numbers come from, who is allowed to change them after the fact, and what happens to a season when the evening underneath it is deleted, confirmed late, or corrected. All line numbers are HEAD (e35394a). Production figures were read live on 2026-09-19 from soccer-app-52b6b , and the scale numbers are: 195 groups , 93 communityPlayerStats rows across 12 clubs, 717 communityPairStats , 95 games, 1 seasonSummary , 1 seasonCards .

The two functions that write every counter a season archives

closeSeason does not compute anything. It copies communityPlayerStats , communityStats and communityPairStats into a document and subtracts them back out. Everything a season *is* was written earlier by two functions, and no other section names either of them.

commitRoundStats — functions/src/index.ts:13800 , a callable, one invocation per משחקון (mini-game) committed from the advanced live screen. It is the sole writer of fifteen of the nineteen PLAYER_SEASON_FIELDS and of all seven CLUB_SEASON_FIELDS :

target	fields	line
communityPlayerStats/{g}__{uid}	goals , assists , ownGoals	14141–
	rounds , cleanSheets , csRounds , asRounds	14337, 14356-14357
	wins / losses / ties	14440–
	penTaken/penScored/penMissed/penFaced/penSaved/penConceded	14200–
communityStats/{g}	rounds , goals , guestGoals , ownGoals , tiedRounds , shootoutRounds , scorelessRounds	14162-14189
communityPairStats/{g}__{a}__{b}	assists only	14474

CLUB_SEASON_FIELDS is that list at 14166-14186, field for field, in order. The archive's totals map is a transcript of one sb.bump call.

Note the coverage denominators ride the *same* write as the metric they divide, at zero extra Firestore operations — which is why they exist at all, and why their absence on an old row is information rather than an omission:

```
// functions/src/index.ts:14337-14358 — one bump per on-field player per mini-game
sb.bump(cpsRef(uid), { groupId, userId: uid, updatedAt: now }, {
  rounds: 1,
  ...(clean ? { cleanSheets: 1 } : {}),
  csRounds: 1,
  asRounds: 1,
});
```

`rollUpClubPairs` — `functions/src/index.ts:4695-4788`, and this is the one that surprises. `commitRoundStats` writes exactly **one** of the ten `PAIR_SEASON_FIELDS`. The other nine — `sameTeam`, `winsTogether`, `lossesTogether`, `cleanSheetsTogether`, `against`, `winsA`, `winsB`, `assistsAToB`, `assistsBToA` — are written once **per evening**, after the evening is credited, by a separate rollup that re-reads the mini-games from a subcollection:

```
// index.ts:6466-6496, inside the creditedNow block of onGameRosterChanged
const rhSnap2 = await db.collection('games').doc(gameId).collection('roundHistory').get();
const rounds2: ChemistryRound[] = rhSnap2.docs.map(...);
await rollUpClubPairs({ gameId, groupId: gid, at: after.startsAt, rounds: rounds2 });
```

`rollUpClubPairs` is latched by `communityPairRollups/{groupId}__{gameId}` (:4710-4716), chunks its writes at 450 ops, and computes the pair totals with `pairsFromRounds` (`functions/src/clubChemistry.ts:95-146`) — collapsing the whole evening in memory first, because writing pairs per mini-game adds n^2 operations to a batch already near Firestore's 500 ceiling, *and the idempotency latch is inside that batch*, so an overflow would lose the round's statistics permanently. It also stamps `communityStats.chemistrySince` the first time it runs (:4772-4777).

Two consequences the other sections observe as symptoms without naming the cause.

Guests. `pairsFromRounds` treats a guest as a full participant — its comment says so: *"they were on the pitch, the pass was real... they simply have no account, which matters for titles, not for what happened."* `closeSeason` then drops every guest pair (`isReal`, `seasonRollover.ts:31`, :436). So the collection that grows without bound and the collection the archive keeps are governed by two deliberate and *opposite* decisions, taken in two files, neither of which cites the other. 293 of the live club's 314 pair documents are the result.

The assist gap. Section 09 reports that the archive's directional pair assists sum to 7 against `totals.assists: 13`, and calls the cause unexplained. It is the two writers' two windows. I read every `roundHistory` document of all 23 surviving games of `HhzIwmjMl1i5HSOGht3p` and counted the goals that carry both a scorer and a different assister and are not own goals:

```

roundHistory documents across the 23 games : 31
committedRounds latch documents           : 35
communityStats.rounds sealed into s1      : 37
assisted goals recoverable from roundHistory: 7    (0 of them involve a guest)
archive pairs, assistsAToB + assistsBToA   : 7    ← exact match
archive pairs, legacy undirected `assists` : 8
archive players[].assists / totals.assists: 13 / 13

```

Four mini-games were committed with no `roundHistory` document at all — three on `WhqMQzLznMgP` (28.06) and one on `5EQKVHge5mtP` (29.06), the club's two oldest evenings, which predate the subcollection — and two more belong to games that have since been deleted (below). The pair rollup can only see what `roundHistory` holds, so those mini-games contributed **no chemistry at all**: no `sameTeam`, no `against`, no directional assist. The legacy `assists` is 8 rather than 7 because `commitRoundStats` writes it at commit time from the payload in hand, so it saw one mini-game the rollup did not. Three numbers, three windows, one Hebrew word.

Nothing in the repository reconciles `communityStats.rounds` (37) against the `committedRounds` latches (35) or `roundHistory` (31). `totals.rounds: 37` is the number `he.seasonHistoryLine` prints as "37 משחקונים" on a permanent card.

`games` is credited without asking which season it is

The nineteenth `PLAYER_SEASON_FIELD`, and the only one every club records, is written somewhere else again — in `onGameRosterChanged`, on the `didEveningHappen` `false→true` transition (`index.ts:5868–5919`):

```

const wasHappened = didEveningHappen(before);
const isHappened  = didEveningHappen(after);
if (!wasHappened && isHappened && after.groupId && after.players?.length) {
  for (const uid of after.players) {
    if (arrivals[uid] === 'no_show') continue;
    batch.set(db.collection('communityPlayerStats').doc(`${gid}__${uid}`),
      { groupId: gid, userId: uid, games: increment(1), updatedAt: Date.now() }, { merge: true })
  }
  batch.create(db.doc(`games/${gameId}/finishCredited/once`), { at: Date.now() });
}

```

Three things follow, and the third is not written down anywhere.

1. **The `finishCredited/once` marker is the audit trail for section 08's 22-vs-19.** I probed it on all 23 games: **19 exist**. The four without one are `2SMrlCGH` (06.07), `MMtE8J6H` (16.07), `FCa5UtSd` (04.08) — the three `happened` evenings that never advanced past `liveMatch.phase: 'organizing'` — and `DTNscolR` (17.09), which is `notHappened`. 22 happened evenings minus 3 uncredited = the 19 in the archive, exactly.
2. No-shows are excluded here, and the client scan excludes them too (`gameService.ts:1206–1210`), so the two do **not** diverge on that axis. The whole of the 3-evening gap is the missing marker.

3. **There is no season check.** The `mine test ( index.ts:5092–5106 )` gates `seasons.playedRounds` and `roundSummaries.seasonEvenings`; it does not gate this. An evening whose stamp names an archived season, or an orphan season, still adds `games: +1` to the *running* season's live rows. `games/DTNscolRojYDf0ZmT0sF` is stamped `s3` on a club that has only ever had `s1` and `s2`, is `playVerified: false`, and has no `finishCredited` marker — so it is one admin tap away from crediting seven attendances into season 2's live table while season 2's own progress counter (`playedRounds`, currently `0`) refuses to move and no season's archive will ever contain the night.

`setEveningPlayed` — how an evening joins a season days later

`index.ts:13704–13790`. Admin-only (`adminIds`, not the creator), and the read and write are in one transaction because two admins can be looking at the same screen and a "no" is unrecoverable.

```
const correctingAMistap = state === 'notHappened' && cur.playVerified === false && played;
if (state !== 'unverified' && !correctingAMistap) return { changed: false, state };
tx.update(ref, { playVerified: played, playVerifiedBy: uid, playVerifiedAt: Date.now(), ... });
```

The transition is a plain field write with **no status change**, which is precisely why the credit block above keys on `didEveningHappen` rather than on status. It is also why the season *stamp* cannot be written by it: the stamp's gate is `before.status !== after.status ( index.ts:5802–5810 )`, so a late-confirmed evening that was never stamped stays unstamped for ever and is filed under season 1 by every reader. On a club past season 1 that is an evening credited to the live table and attributed to a sealed archive. ROUND4 §15 covers what the *seal* then computes from today's totals; the seasons half of it is untested and unnamed.

The fourth copy of the season-membership rule

`inSeason` (client) and `eveningInSeason` (server) are documented as the rule. There is a third copy, and it is the only place in the product where `game.seasonId` *refuses* an action:

```
// functions/src/index.ts:14924–14937, loadRetroGameContext – shared by addRetroGoal + removeRetroGoal
const gameSeason = typeof game.seasonId === 'string' ? game.seasonId : '';
const seasons = grp.seasons as { enabled?: boolean; currentId?: string; currentNo?: number } | undefined;
if (seasons?.enabled) {
  const belongsTo = gameSeason || (seasons.currentNo === 1 ? seasons.currentId ?? '' : 's1');
  if (belongsTo !== seasons.currentId) {
    throw new HttpsError('failed-precondition',
      'closedSeasonGame: this evening belongs to a season that has already closed');
  }
}
```

Retro goals write straight into `communityPlayerStats.goals/assists`, `communityStats.goals` and `users.stats ( index.ts:14985–15015 )` with no round attached, so correcting a June goal after a close would credit it to the running season while the sealed archive stayed wrong — and `removeRetroGoal` would decrement a counter the close has already set to zero, since Firestore's `increment` goes negative happily. The guard is right to exist. Three things about it:

- It spells the unstamped→season-1 rule a **fourth** way, by hard-coding the literal 's1' rather than comparing season numbers. It happens to agree with the other three because ids are always s\${no} , but nothing pins that.
- It is inert when seasons.enabled is false, so a club that switched seasons off can still retro-credit goals into a table a close zeroed.
- Its admin gate is game.createdBy === uid || adminIds.includes(uid) (:14899), which is **wider** than every seasons callable's requireClubAdmin (index.ts:15710–15721, adminIds only). A non-admin who created the game can move the club's season counters.
- closedSeasonGame is mapped by seasonRefusalText (seasonService.ts:119) to he.seasonBlockedClosedGame , but it is thrown from a **match** screen, not a seasons screen — RetroGoalsSheet.tsx:47 has its own copy of the substring test.

Production: games/fnBXPo76y0lm (14.07.2026) carries one retroGoals document. That evening is unstamped, so belongsTo is 's1' and currentId is 's2' — **an admin can no longer remove that retro goal**, and the counter it added was archived into s1 and then wound back out of the live table. games/DTNscolRoj , stamped s3 , is refused for ever by construction.

One correction this forces on section 07. That section states that for a timer-only club "eight of the nine titles are decided on fields that are permanently zero, and only mostLoyal can ever be awarded", and calls the Hebrew comment's figure of three wrong. addRetroGoal writes goals and assists onto communityPlayerStats **without** writing rounds , so a club that never opens the advanced screen can still accumulate season goals and assists and can award topScorer and topAssister . Three is the right number; one is not.

Deleting an evening

onGameRosterChanged 's deletion branch (index.ts:5355–5428) writes a gameDeletions/{gameId} audit row and fans out a cancellation push. It touches **no** counter: not clubRecords.eveningsSealed , not seasons.playedRounds , not communityPlayerStats , not communityStats , not communityPairStats . Section 03 states this in the abstract. It has happened four times on the one club that runs seasons:

doc	deletedAt (UTC)	rosterCount	source
QKMcTm2oa0BpD8z5bNMj	2026-08-04 10:58:29	7	manual
HVJ1QfFk3SWN4TyAGMjF	2026-09-02 06:35:26	6	manual
oi6Iz1F308lkbHrv56vy	2026-09-15 06:53:00	0	manual
Iyj4md7E46cbeYL8v61T	2026-09-17 15:13:42	6	manual

All four by the same admin (YIZlKWbvvjae3oqgIoAMr9nzQEi1 , Eliran Tzabari). The last one was deleted **59 minutes before season 1 closed** at 16:12:06Z. Whatever that evening contributed to communityStats.rounds , communityStats.goals and every player's row was still there at 16:12 and was sealed into a write-once archive; the client's games scan lost it the moment the document went. This is the mechanism behind two numbers section 03 calls coincidental: the counters and the scan on this club drift *both* ways at once — the counters are blind to twelve evenings before 26.08, and the scan is blind to four deletions.

`promoteOrphanToGroup` is the one other path that destroys seasons inputs. Its artefact purge (`index.ts:9322–9358`) deletes every `communityPairStats` row, every `eveningStandings` row, every `roundSummaries` and `communityPairRollups` marker, and `clubRecords/{groupId}` — the document holding `eveningsSealed`, which is the zero mark every `roundsAtStart` is measured from. It does not touch `seasonSummary`, `seasonCards`, `users/*/seasonTitles` or `groups.seasons`. A personal group cannot hold a seasons block today (`firestore.rules:334–341`), so the combination is unreachable; it is one rule change away from a club whose archives outlive the counter they were measured against.

The season picker

`src/utils/seasonChoices.ts` — 52 lines, pure, and the only thing that decides which seasons a person is offered on `SeasonSummaryScreen` and whether `SeasonsCard` draws its history button.

```
const MAX_SEASON_CHOICES = 200;
export function seasonChoices(seasons: GroupSeasons): SeasonChoice[] {
  const out = [{ no: seasons.currentNo, id: seasons.currentId, closed: false }];
  const closed = Math.max(0, Math.min(MAX_SEASON_CHOICES, seasons.count ?? 0));
  for (let no = closed; no >= 1; no -= 1) {
    if (no === seasons.currentNo) continue; // disable/re-enable keeps numbering
    out.push({ no, id: `s${no}`, closed: true });
  }
  return out;
}
```

It derives ids from `count` alone and never reads an archive, so **the picker asserts that `seasonSummary/{g}__s{n}` exists for every $n \leq \text{count}$** . That assertion is exactly what a reopen breaks in the other direction: the reopen decrements `count` (`index.ts:16962`), so the discarded successor disappears from the list with no trace, which is correct — but nothing removes a chip for an archive that was deleted by hand or never written. `CommunityStatsScreen` handles the miss by bailing the whole scope back to `{k: 'current'}` (`:316`); `SeasonSummaryScreen` handles it by silently rendering the **running** season under the closed season's heading (`seasonSummaryService.ts:429–456`). Two screens, two different answers to one missing document.

`lastClosedSeason` (`:48–52`) is `seasonChoices(...).find(c => c.closed)`, with a first-line guard that returns `null` for a club that has neither `enabled` nor a `count`. It is what gates the "שלי N סיכום עונה" button.

Observability: eight events, and the three that do not exist

event	where	payload
<code>SeasonsEnabled</code>	<code>SeasonsSettings.tsx:502</code>	<code>groupId</code> , <code>cadence</code> , <code>target</code> , <code>sealedHistory</code>
<code>SeasonsDisabled</code>	<code>:479</code>	<code>groupId</code>
<code>SeasonTargetChange</code> <code>d</code>	<code>:516</code>	<code>groupId</code> , <code>cadence</code> , <code>target</code>

event	where	payload
SeasonReopened	:557	groupId, seasonNo
SeasonEndedEarly	:612	groupId, seasonNo
SeasonHistoryViewed	SeasonHistoryScreen.tsx:319	groupId, seasons
SeasonSummaryViewed	SeasonSummaryScreen.tsx:308	—
SeasonSummaryShared	:349	—

Every one is client-side and every one is an **admin action or a screen open**. There is no event for a season closing by the hourly sweep, none for a close-on-seal, none for the summary push. The two mechanisms that close seasons without a human — the two that also destroy data — emit nothing to analytics, log to `console.log`, and (per section 11) have never written a row to the `errors` collection. The only durable record that an automatic close happened is the archive it creates.

The mock layer, and why QA cannot see the common club

`USE MOCK DATA` short-circuits all three seasons services: `seasonService` returns success without calling anything (:234–266), `seasonHistoryService` returns `mockHistory()` / `mockTable()` (:151–233), `seasonSummaryService` returns `mockSeasonSummary()` (:549–).

Every mock player in every one of the three builders has `rounds > 0` — 22, 24, 20, 12 on the closed season; 41, 44, 38, 30 on the running one — and every mock pair has non-zero `sameTeam` and directional assists. **The timer-only club is unreachable in mock mode**. That is the club the product notes call common; it is the club for which `rank-denominator-of-excludes-attendees`, `share-card-is-all-zeros-for-a-timer-only-club`, `seasonsummary-participant-rounds-only` and `season-push-body-promises-three-things-a-timer-club-has-none-of` were all written. Mock QA cannot reproduce any of them.

The mock is also the **only** place two archive fields are ever exercised: `mockHistory()` sets `partialData: true` (`seasonHistoryService.ts:161`), and no production archive carries the field at all, so the "נתונים חלקיים" chip has only ever rendered against a fixture. `mockTable()` deliberately includes one uid that is *not* in the mock roster (`'u_left_the_club'`, :224) to exercise the frozen-name path — a good instinct that the real fixture (`tests/fixtures/realClub.ts`) does not copy.

Three small things with no owner

`src/utils/seasonTitleIcon.ts` (51 lines) holds `SEASON_TITLE_ICON` and `SEASON_TITLE_TINT`, total `Record<SeasonTitleKey, ...>` maps so a tenth title is a compile error. The tints are declared usable **only on a disc, never as text colour, where they fail contrast** — a constraint nothing enforces. It is the fourth total record keyed on `SeasonTitleKey`, after `SCALE` (`seasonMedalTier.ts:57`), `TITLE` (`he.ts:34`) and `SEASON_TITLE_KEYS` itself.

`parseSeasonTable` **does not read** `completedRounds`. `FinishedSeasonTable` (`seasonArchive.ts:35–48`) has seven totals, a players array, a duo and a names map — and no season length, no `no`, no `awards`, no `endsAt`. So the archive has two client readers with two disjoint field sets: `seasonHistoryService.table` → `parseSeasonTable` for the stats table, and `seasonHistoryService.list` → `fromCard` over `seasonCards` for everything else. A screen that wants the length of a closed season must read the **card**, never the archive it is looking at.

`he.seasonRangeUntil` ("מתחילת המועדון עד X" — "from the club's beginning until X") is now reachable only when `startsAt === 0`, which only the `sealNow` activation branch produces (`index.ts:16478`). No production season has it. ROUND4 §19 is closed; the string is now dead code for every club that carried its history rather than sealing it.

13. Defect register

Every defect named anywhere in this document, collected. **83** in total: 2 P0, 10 P1, 11 P2, 60 carrying no explicit severity in the section that raised them.

Each slug is unique and searchable — find the full argument, the code quotation and the production evidence by searching this document for the slug itself. The one-line gloss here is the sentence the section opened its case with, not a summary of it.

Severity, where a section assigned one: **P0** the feature is unusable or produces a wrong result for every club; **P1** a wrong number or lost data in an ordinary flow; **P2** a wrong number in an edge case, or a correctness risk not yet observed.

#	Severity	Defect	\$	What the section says
1	P0	live-club-cannot-use-date-cadence	2	live-club-cannot-use-date-cadence (P0). Until commit ae2c042 (18.09 16:53)
2	P0	sealed-season-rank-vs-club-table-nondeterministic	9	sealed-season-rank-vs-club-table-nondeterministic (P0) is worse than that in the shipped
3	P1	club-totals-zeroed-not-subtracted	1	then violates fifty lines from the bottom. Ledger club-totals-zeroed-not-subtracted. Live:
4	P1	disable-reenable-zeroes-running-season	2	disable-reenable-zeroes-running-season (P1). It does not. Re-enabling runs the
5	P1	enable-sealnow-archives-counter-not-games	2	enable-sealnow-archives-counter-not-games (P1). completedRounds: played where
6	P1	season-id-reissued-with-live-stamps	2	season-id-reissued-with-live-stamps (P1). count advances only on a <i>close</i> , so
7	P1	quiet-legacy-evening-blocks-club-forever	5	quiet-legacy-evening-blocks-club-forever, P1-really-P0 The legacy floor at :15557
8	P1	sweep-closes-a-club-that-turned-seasons-off	5	sweep-closes-a-club-that-turned-seasons-off, P1 This check is the fix for that
9	P1	nemesis-victim-tormentor-not-deduplicated	9	the same person in all four rows he had (nemesis-victim-tormentor-not-deduplicated, P1). The fix

#	Severity	Defect	§	What the section says
10	P1	rank-denominator-of-excludes-attendees	9	each week. That games > 0 is the fix for rank-denominator-of-excludes-attendees (P1): the
11	P1	share-card-claims-a-shared-title-as-its-own	9	(:209-211). share-card-claims-a-shared-title-as-its-own (P1) is that sharedWith was computed
12	P1	share-card-is-all-zeros-for-a-timer-only-club	9	club that has any. That is the fix for share-card-is-all-zeros-for-a-timer-only-club (P1): the
13	P2	assist-peers-account-for-half-the-season-assists	1	archive-duo-reader-has-no-floor, assist-peers-account-for-half-the-season-assists.
14	P2	older-archive-degrades-into-plausible-zeros	1	older-archive-degrades-into-plausible-zeros: an archive written by an older build yields confident
15	P2	reenable-inherits-stale-reopenedat	2	reenable-inherits-stale-reopenedat (P2). The sealNow branch writes
16	P2	rounds-clamp-uses-a-different-history	2	rounds-clamp-uses-a-different-history (P2). The plan refused a target below
17	P2	season-plan-refusals-unmapped	2	season-plan-refusals-unmapped (P2). git show 4de80d0:src/services/seasonService.ts
18	P2	partialdata-never-set-on-a-rollover-close	5	partialdata-never-set-on-a-rollover-close, P2 partialData marks an archive whose
19	P2	resume-subtracts-recomputed-row	5	resume-subtracts-recomputed-row, P2 — with one exception, and it is the whole of the
20	P2	archive-completedrounds-never-read-by-client	9	(archive-completedrounds-never-read-by-client, P2); the shipped client substitutes totals.rounds,

#	Severity	Defect	§	What the section says
21	P2	assists-per-round-denominator-disagrees-with-club-efficiency-tab	9	assists-per-round-denominator-disagrees-with-club-efficiency-tab (P2) — the shipped binary divides
22	P2	peers-empty-promises-it-will-fill	9	timer-only club can never keep (peers-empty-promises-it-will-fill, P2).
23	P2	share-card-rank-gate-still-prints-last-place	9	(share-card-rank-gate-still-prints-last-place, P2). One side-effect of the floor is worth naming:
24	—	archive-duo-reader-has-no-floor	1	archive-duo-reader-has-no-floor, assist-peers-account-for-half-the-season-assists.
25	—	awards-denominator-derivation-untested	1	awards-denominator-derivation-untested, repaired-completerounds-silently-contradicts-the-sealed-titles.
26	—	evenings-22-on-one-screen-0-on-the-next	1	evenings-22-on-one-screen-0-on-the-next). Note the stamp still reads s1 even though s2 has since
27	—	games-seasonid-unpinned	1	(ledger games-seasonid-unpinned, emulator-proven with four successful writes).
28	—	orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id	1	undo-close-orphans-the-season-stamps, orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id.
29	—	partialdata-only-on-the-sealnow-path	1	partialdata-only-on-the-sealnow-path.
30	—	reopen-inherits-the-successor-seasons-target-history	1	reopen-inherits-the-successor-seasons-target-history: an array element cannot say which season it
31	—	repaired-completerounds-silently-contradicts-the-sealed-titles	1	awards-denominator-derivation-untested, repaired-completerounds-silently-contradicts-the-sealed-titles.

#	Severity	Defect	§	What the section says
32	—	season1-range-prints-a-31-hour-lie	1	season1-startsat-is-the-enable-moment, season1-range-prints-a-31-hour-lie. Worse, the
33	—	season1-startsat-is-the-enable-moment	1	season1-startsat-is-the-enable-moment, season1-range-prints-a-31-hour-lie. Worse, the
34	—	stale-pair-woundback-stamp-skips-next-wipe	1	ran after this reopen. Ledger: stale-pair-woundback-stamp-skips-next-wipe, ROUND4 §9. This is armed
35	—	target-history-never-shown-to-anyone	1	reader was fixed; nothing renders it even now (ledger target-history-never-shown-to-anyone).
36	—	undo-close-orphans-the-season-stamps	1	undo-close-orphans-the-season-stamps, orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id.
37	—	approved-rounds-target-is-not-the-one-written	2	finding approved-rounds-target-is-not-the-one-written from the other side.
38	—	reenable-wipes-targethistory	2	trail of every finish-line move the club ever made — finding reenable-wipes-targethistory.
39	—	end-season-now-ignores-playedrounds	3	card said was 22. (LEDGER end-season-now-ignores-playedrounds; fixed, the fix is
40	—	group-doc-read-three-times-per-sealed-evening	3	group-doc-read-three-times-per-sealed-evening). And with the gate added, a game that
41	—	inseason-tested-as-a-private-copy	3	inseason-tested-as-a-private-copy).
42	—	mirror-each-ignores-its-own-params	3	see LEDGER mirror-each-ignores-its-own-params: the describe.each declares five
43	—	no-test-reconciles-the-three-counters	3	assertion that was missing — LEDGER no-test-reconciles-the-three-counters, which is
44	—	nothing-tests-the-sweep	3	the suite only inside comments (LEDGER nothing-tests-the-sweep).

#	Severity	Defect	§	What the section says
45	—	reopen-season1-drops-the-19-unstamped-evenings	3	reopen-season1-drops-the-19-unstamped-evenings).
46	—	seasonid-stamp-orphan-unrepairable	3	seasonid-stamp-orphan-unrepairable, orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id).
47	—	unstamped-season-1-rule-disagrees-with-the-server-by-19	3	unstamped-season-1-rule-disagrees-with-the-server-by-19).
48	—	cadence-with-no-finish-line-never-closes	4	will ever close. This is cadence-with-no-finish-line-never-closes. The sweep at least now
49	—	date-retarget-restarts-the-clock	4	date-retarget-restarts-the-clock. The date branch writes startsOn: todayIn(now) and
50	—	date-season-cannot-be-renewed-at-the-same-length	4	date-season-cannot-be-renewed-at-the-same-length. The save button is gated on
51	—	endson-endsat-one-day-apart	4	(endson-endsat-one-day-apart). Verified by executing both algorithms with TZ=UTC, as Cloud
52	—	reopen-extends-a-date-season-by-a-full-length	4	(reopen-extends-a-date-season-by-a-full-length). rebaseCadence is the function for opening
53	—	settings-screen-validates-a-live-target-against-zero	4	settings-screen-validates-a-live-target-against-zero is fixed at HEAD. Per ROUND4 finding 1,
54	—	target-change-can-close-a-season-with-no-confirmation	4	target-change-can-close-a-season-with-no-confirmation is now <i>half</i> fixed, and the fix
55	—	target-history-entry-carries-no-season-identity	4	season's history to another (target-history-entry-carries-no-season-identity). Not the
56	—	target-history-erased-at-close	4	target-history-erased-at-close. performSeasonClose (:15911) and endSeasonNow
57	—	chemistry-since-not-restored	6	chemistrySince (chemistry-since-not-restored). The close sets it to the close instant

#	Severity	Defect	§	What the section says
58	—	club-seasons-block-written-only-after-the-restore-succeeds	6	club-seasons-block-written-only-after-the-restore-succeeds. The order is: run
59	—	pair-wipe-skipped-on-next-close-after-reopen	6	That is pair-wipe-skipped-on-next-close-after-reopen, and it is armed in production right
60	—	reclose-after-undo-notifies-nobody	6	reclose-after-undo-notifies-nobody. announceSeasonClosed routes through
61	—	reopen-never-restores-guest-pair-chemistry	6	Guest pair chemistry (reopen-never-restores-guest-pair-chemistry). A guest id is minted
62	—	reopen-playedrounds-ignores-unstamped-games	6	(reopen-playedrounds-ignores-unstamped-games). 4be7f1c filtered on seasonId == lastId
63	—	undo-chains-backwards-with-no-limit	6	undo-chains-backwards-with-no-limit. There is no time bound and no already-undone guard.
64	—	undo-leaves-dangling-season-summary-pushes	6	The notification documents (undo-leaves-dangling-season-summary-pushes). createNotificationOnce
65	—	cleansheetking-ranks-on-count-while-the-club-table-ranks-on-rate	7	The same split exists for clean sheets. Defect — cleansheetking-ranks-on-count-while-the-club-table-ranks-on-rate. The season title is leaders(eligible, p => p.cleanSheets) — a raw count. The club'...
66	—	deadlyduo-together-is-collected-and-never-read	7	deadlyduo-together-is-collected-and-never-read. SeasonPairLine.together (line 61, "Rounds the two were on the same side") is populated at seasonRollover.ts:536 from p.sameTeam and is never read by ...
67	—	eligibility-denominator-is-max-attendance	7	eligibility-denominator-is-max-attendance. Two numbers describing "how long was this season" live in one archive document (19 and 22) and neither is labelled as the other's alternative; awardsDenom...
68	—	loyalty-crown-is-structurally-platinum	7	loyalty-crown-is-structurally-platinum (fixed at HEAD, and worth the reader's attention as a worked example of the identity). For about a day, seasonRollover.ts wrote awardsDenominator; seasonEveni...

#	Severity	Defect	§	What the section says
69	—	mvp-medal-can-never-leave-bronze	7	MVP_STEPS carries an honest comment (lines 72–86) admitting the original 6.5, 7.5, 8.5 was written against an assumed 1–10 scale that the clamp makes impossible, and that a "correction" to 7, 8, 9 ...
70	—	mvp-title-held-by-the-entire-club	7	mvp-title-held-by-the-entire-club. With the original floor = 0, the sentinel cleared the floor and every eligible player tied at exactly 6.0. Production s1 is exactly that:
71	—	penalty-titles-crowned-on-two-kicks	7	penalty-titles-crowned-on-two-kicks. A season crown at a perfect 100% off two kicks is what the archive actually contains, and medalTier('penaltyKing', 1.0, ...) grades it platinum (RATE_STEPS = 0.7,...
72	—	appearances-rollup-undercounts-the-scan	8	appearances-rollup-undercounts-the-scan. mostLoyal deliberately reads the scan for the running
73	—	archived-goals-silently-zero	8	archived-goals-silently-zero. Lifetime club goals are assembled on the client as
74	—	closed-season-tooltip-promises-lifetime	8	closed-season-tooltip-promises-lifetime is true of HEAD and false of every shipped binary. At
75	—	compare-card-divides-season-goals-by-lifetime-nights	8	playerCompareService — compare-card-divides-season-goals-by-lifetime-nights is fixed at
76	—	limit-200-window-is-a-lifetime-label	8	limit-200-window-is-a-lifetime-label. The doc-comment at :934-947 is honest that this is a
77	—	did-not-play-told-the-season-had-no-games	9	The hasData split (screen :463-512) is the fix for did-not-play-told-the-season-had-no-games
78	—	peer-rows-always-show-the-generic-auto-avatar	9	(:400-403), not a bug; the bug (peer-rows-always-show-the-generic-auto-avatar, P3) was on the
79	—	share-card-names-one-unit-two-ways	9	share-card-names-one-unit-two-ways, P3) and to drop the parentheses, because bidi resolves a closing
80	—	cron-is-an-interval-not-a-clock	11	(P3 cron-is-an-interval-not-a-clock); and the timeout was raised from the 60-second

#	Severity	Defect	§	What the section says
81	—	perform-close-writes-next-season-even-when-nothing-was-archived	12	Every sixty minutes, unattended, that function may zero a club's communityPlayerStats, communityStats and communityPairStats, write a seasonSummary with create() (which is one-shot), write nine sea...
82	—	seasonsummary-participant-rounds-only	12	- A departed participant reads a season they played, by id, on games > 0 rounds > 0. The games half exists because the rule previously authorised on rounds alone and every ex-member of a timer...
83	—	season-push-body-promises-three-things-a-timer-club-has-none-of	13-gaps	season-push-body-promises-three-things-a-timer-club-has-none-of were all written. Mock QA cannot